스프링부트4와시큐리티쿡북

8장 · 2026-11-11

1장. REST API 프로젝트 준비하기
2장. 엔티티와 더미 데이터 만들기
3장. JWT로 토큰 만들고 검증하기
4장. 공통 응답과 예외 처리 만들기
5장. 스프링 시큐리티와 JWT 필터 만들기
6장. SecurityFilterChain 설정하기
7장. 회원 API 만들기
8장. JWT 인증 전체 흐름 확인하기

1장. REST API 프로젝트 준비하기

01

1. REST API와 SSR

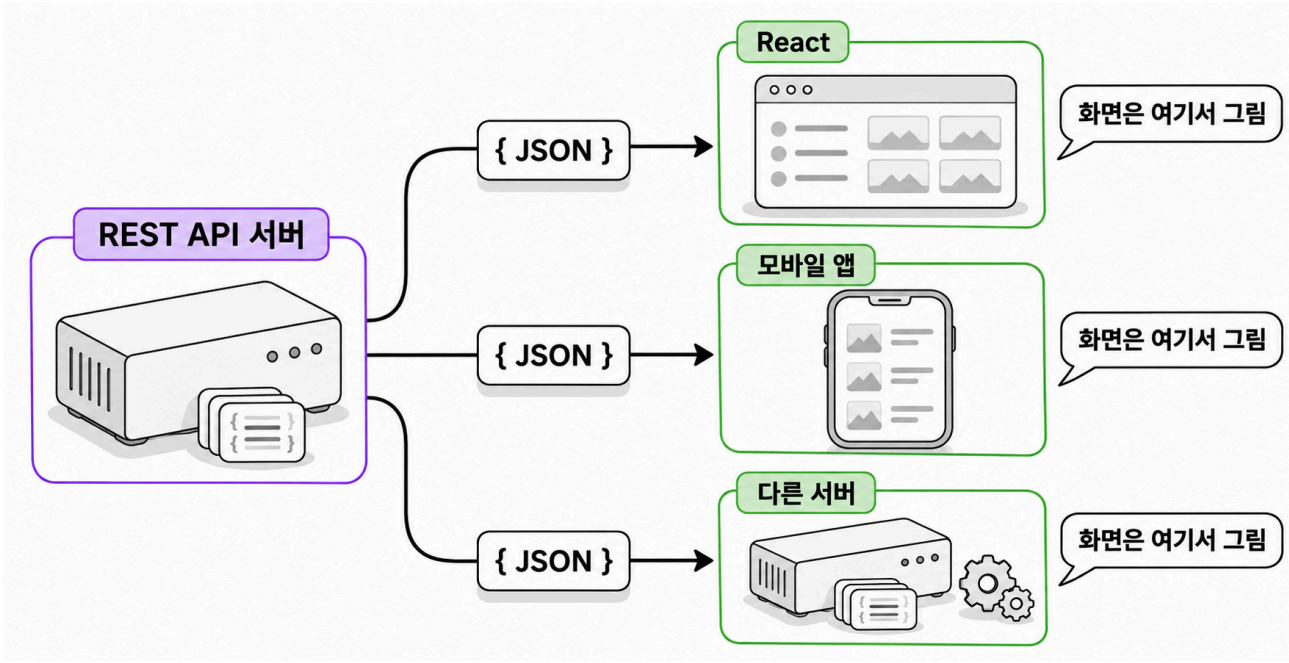
1.1 REST API의 정의

REST(Representational State Transfer)는 로이 필딩(Roy Fielding)이 2000년 박사학위 논문에서 소개한 웹 아키텍처 원칙입니다. **REST API**는 이 원칙에 따라 클라이언트와 서버가 데이터를 주고받는 방식입니다. 원래 REST에는 클라이언트-서버 분리, 무상태, 캐시, 균일 인터페이스 같은 제약이 여러 개 있습니다. 이 책에서는 실습에 필요한 세 가지 약속만 지킵니다.

약속	내용	예
자원은 URI로 표현	다룰 대상을 주소로 가리킵니다	`/api/users/me`
행위는 HTTP 메서드로 표현	주소가 같아도 메서드에 따라 하는 일이 달라집니다	GET 조회 · POST 생성 · PUT 수정 · DELETE 삭제
응답은 데이터	서버는 HTML 화면 대신 JSON 데이터를 돌려줍니다	`{"id":1,"username":"ssar"}`

JSON으로 응답하는 것은 REST의 필수 조건이 아닙니다. 이 책이 고른 방식입니다.

서버가 데이터만 돌려주면 화면은 데이터를 받은 쪽에서 그립니다. 그래서 서버 하나가 브라우저의 React 화면, 모바일 앱, 다른 서버의 요청에 모두 응답할 수 있습니다.



하나의 REST API 서버와 여러 클라이언트

1.2 SSR 응답과 REST 응답 비교

Mustache나 Thymeleaf로 만들어 본 게시판은 **SSR(Server Side Rendering)** 방식입니다. 서버가 템플릿에 데이터를 채워 완성된 HTML 문서를 보냅니다. REST API 방식에서는 서버가 데이터를 JSON으로 바꿔 보내면 할 일이 끝납니다.

구분	SSR	REST API
서버 응답	HTML 문서	JSON 데이터
화면을 그리는 곳	서버(Mustache, Thymeleaf)	클라이언트(React, 앱)
응답을 쓸 수 있는 클라이언트	주로 브라우저	브라우저, 모바일 앱, 다른 서버
컨트롤러	`@Controller` + 뷰 이름 반환	`@RestController` + 객체 반환

두 컨트롤러 코드는 반환값만 다릅니다. SSR 컨트롤러는 뷰 이름을 반환하고, 스프링은 그 이름으로 템플릿을 찾아 HTML을 만듭니다.

코드 조각

Java

```java
@Controller
public class UserViewController {

    @GetMapping("/users/me")
    public String me(Model model) {
        model.addAttribute("username", "ssar");
        return "user/me";   // templates/user/me.mustache 로 HTML을 만든다
    }
}
```

REST 컨트롤러는 객체를 반환합니다. `@RestController`가 붙은 클래스에서는 스프링이 반환된 객체를 JSON으로 바꿔 응답 본문에 씁니다.

코드 조각

Java

```java
@RestController
public class UserApiController {

    @GetMapping("/api/users/me")
    public Map<String, Object> me() {
        return Map.of("id", 1, "username", "ssar");   // JSON으로 바뀌어 응답된다
    }
}
```

같은 사용자 정보를 요청해도 클라이언트가 받는 응답 본문은 다음처럼 다릅니다.

코드 조각

HTML

```html
<!-- SSR 응답 -->
<html><body><h1>ssar 님의 정보</h1></body></html>
```

코드 조각

JSON

```json
{"status":200,"msg":"성공","body":{"id":1,"username":"ssar"}}
```

REST 응답에 있는 `status` · `msg` · `body`는 이 책에서 쓸 공통 응답 형식입니다. 응답마다 형식이 다르면 클라이언트가 응답을 해석하기 어렵습니다. 그래서 4장에서 모든 응답을 이 형식으로 맞춥니다.

---

2. 프로젝트 만들기

2.1 start.spring.io 설정

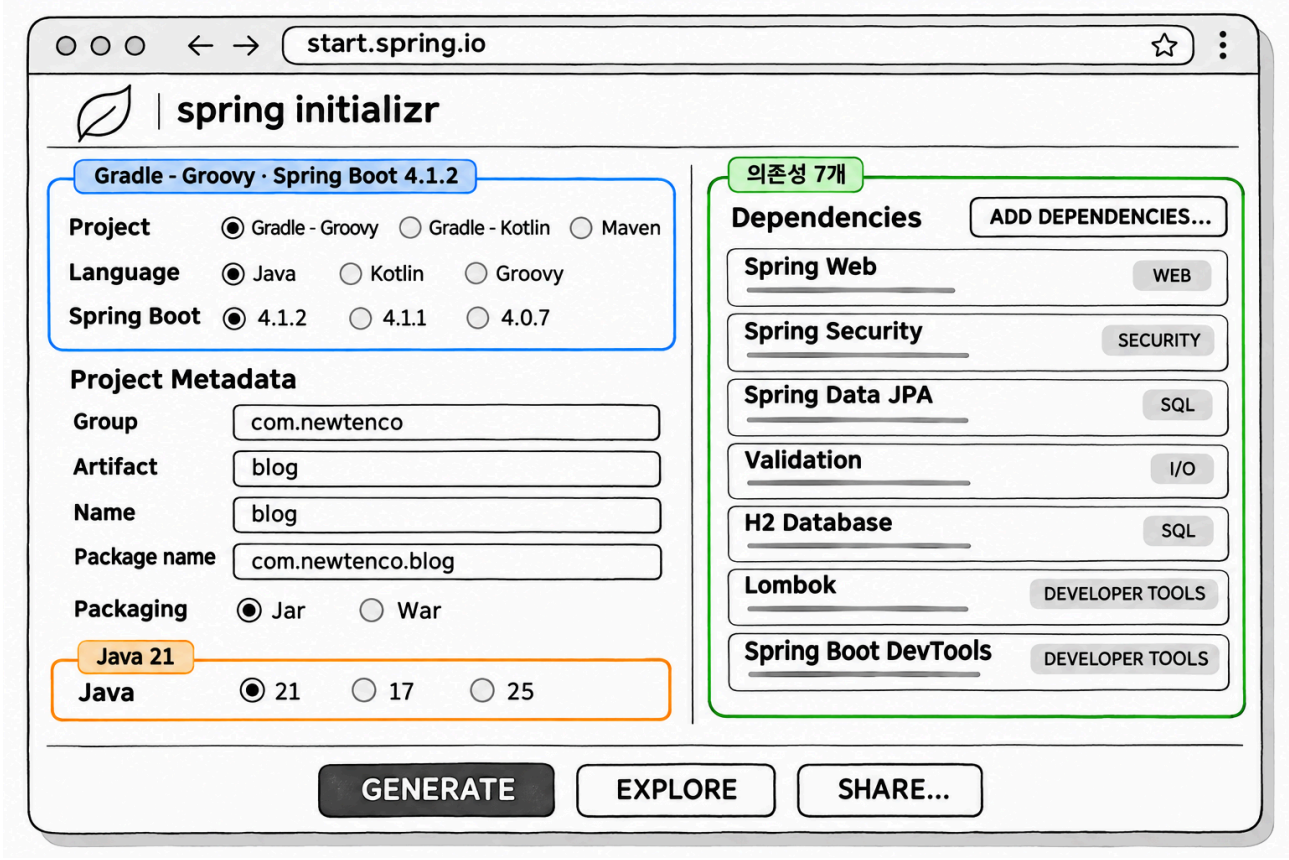
start.spring.io에서 아래 값으로 프로젝트를 만듭니다. 내려받은 압축 파일을 풀고 IntelliJ IDEA로 엽니다.

항목	값
Project	Gradle - Groovy
Language	Java
Spring Boot	4.1.2
Group	com.newtenco
Artifact	blog
Packaging	Jar
Java	21
Dependencies	Spring Web, Spring Security, Spring Data JPA, Validation, H2 Database, Lombok, Spring Boot DevTools

각 의존성을 이 책의 어디에서 쓰는지는 다음과 같습니다.

의존성	쓰는 곳
Spring Web	REST 컨트롤러와 JSON 응답
Spring Security	필터 체인, 인증 · 인가, BCrypt
Spring Data JPA	User · Board · Reply 엔티티와 Repository
Validation	회원가입 요청의 `@Valid` 검증

의존성	쓰는 곳
H2 Database	개발용 메모리 DB
Lombok	getter · 생성자 · 빌더 자동 생성
Spring Boot DevTools	코드를 바꾸면 서버 자동 재시작



위 표의 값을 모두 고른 start.spring.io 화면

생성형 예시 · 실제 실행 안 함

2.2 java-jwt 의존성 추가

JWT(JSON Web Token)를 만들고 검증하는 라이브러리는 start.spring.io 목록에 없습니다. 그래서 `build.gradle`에 직접 적습니다. 이 책은 Auth0의 **java-jwt**를 씁니다. 토큰 생성과 검증을 메서드 몇 개로 끝낼 수 있어서 3장에서 만들 `JwtUtil`의 코드가 짧아집니다.

코드 조각

GROOVY

```
implementation 'com.auth0:java-jwt:4.5.0'
```

`dependencies` 블록에 이 줄을 넣으면 `build.gradle`은 다음과 같이 완성됩니다. start.spring.io가 만든 파일과 비교해서 빠진 줄이 있으면 추가합니다.

예제 1-1 — 프로젝트 빌드 설정

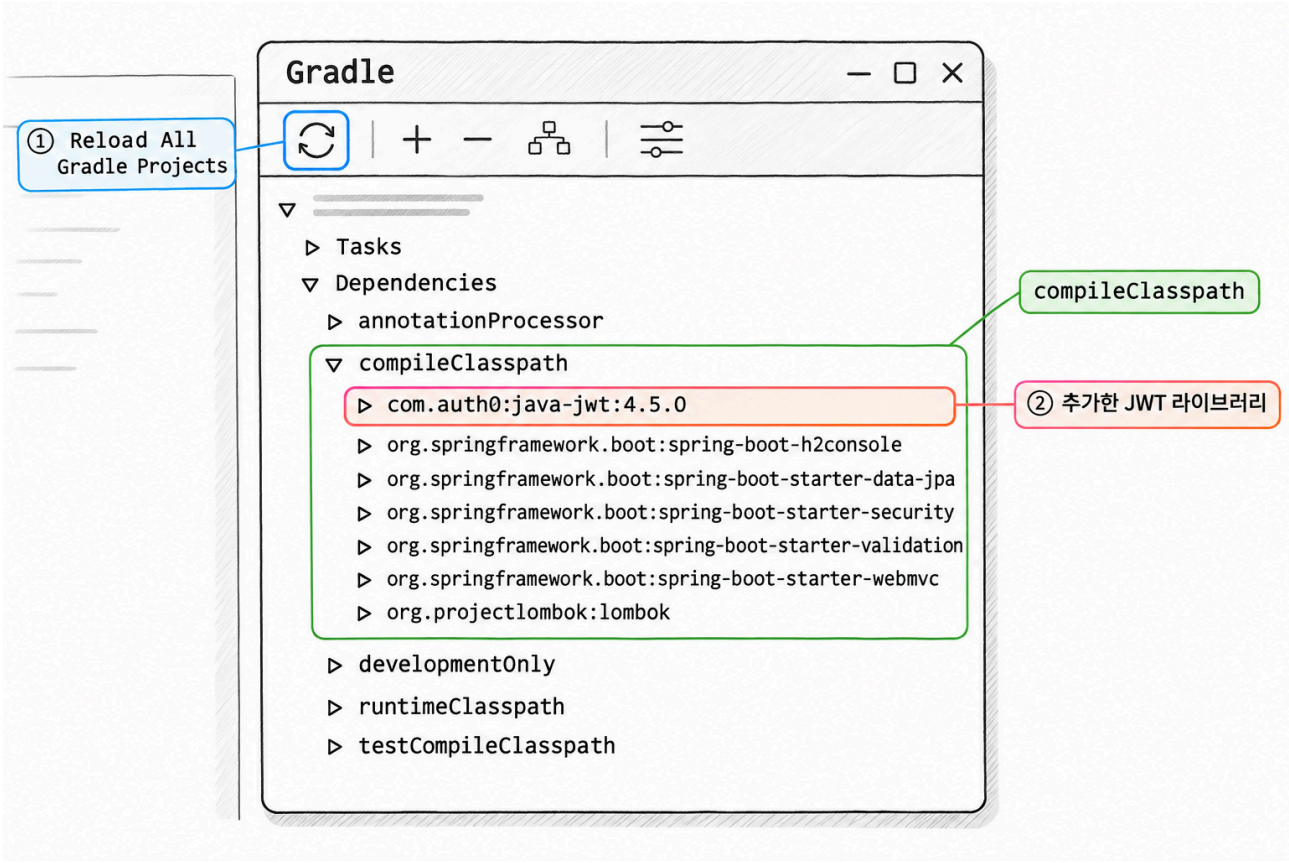
파일명: `build.gradle`

GROOVY

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.2'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'com.newtenco'
version = '0.0.1-SNAPSHOT'
description = 'Spring Boot 4 + Security + JWT REST API'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    // JWT 라이브러리 (auth0)
    implementation 'com.auth0:java-jwt:4.5.0'

    implementation 'org.springframework.boot:spring-boot-h2console'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-security'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    compileOnly 'org.projectlombok:lombok'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    runtimeOnly 'com.h2database:h2'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-security-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-validation-test'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testCompileOnly 'org.projectlombok:lombok'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testAnnotationProcessor 'org.projectlombok:lombok'
}

tasks.named('test') {
    useJUnitPlatform()
}
```

실행 결과

IntelliJ 오른쪽 Gradle 창에서 Reload All Gradle Projects 버튼을 누르면 의존성을 내려받습니다. 다 내려받으면 Gradle 창의 Dependencies 목록에 `com.auth0:java-jwt:4.5.0`이 나타납니다.



IntelliJ Gradle 창에서 Dependencies › compileClasspath를 펼친 화면

생성형 예시 · 실제 실행 안 함

2.3 스프링부트 4에서 달라진 의존성

스프링부트 3 프로젝트의 의존성과 예제 1-1을 비교하면 이름이 바뀐 것과 새로 생긴 것이 보입니다. 스프링부트 4는 자동 설정을 기능별 모듈로 잘게 나눴고, 그에 맞춰 스타터 이름도 바뀌었습니다.

스프링부트 3	스프링부트 4	달라진 점
`spring-boot-starter-web`	`spring-boot-starter-webmvc`	웹 스타터의 이름이 바뀌었습니다. 예전 이름은 사용 중단 예정입니다
(스타터에 포함)	`spring-boot-h2console`	H2 콘솔 자동 설정이 별도 모듈로 분리되었습니다

스프링부트 3	스프링부트 4	달라진 점
`spring-boot-starter-test` 하나	`...-webmvc-test`, `...-data-jpa-test`, `...-security-test` 등	테스트 스타터도 기능별로 나뉘었습니다

실습에 바로 영향을 주는 것은 `spring-boot-h2console` 입니다. 이 의존성이 없으면 `spring.h2.console.enabled=true` 를 설정해도 `/h2-console` 주소가 열리지 않습니다. 스프링부트 3 자료를 보고 설정만 따라 하면 여기서 막히기 쉽습니다.

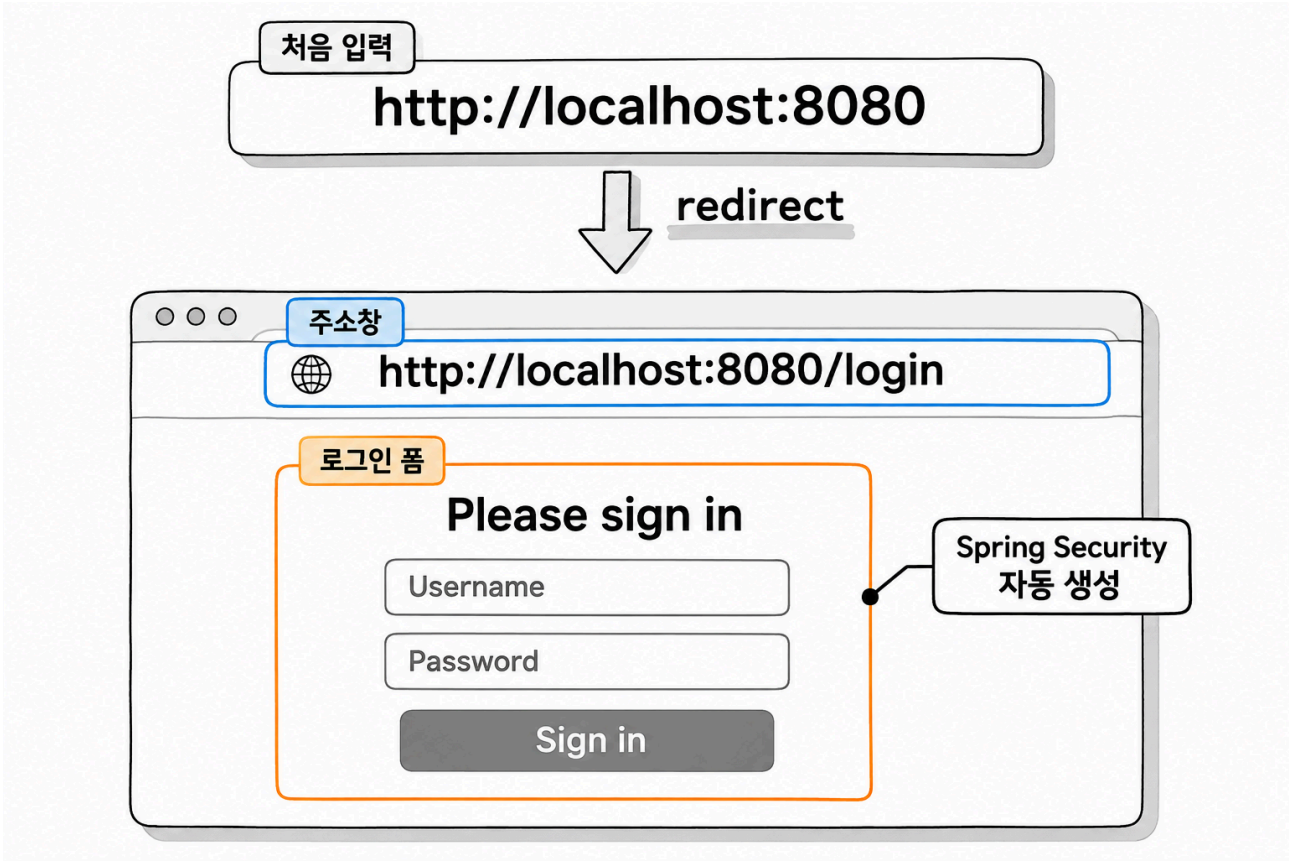
인터넷에서 찾은 스프링부트 3 예제를 가져올 때도 주의해야 합니다. `spring-boot-starter-web` 같은 예전 스타터 이름은 예제 1-1에 나온 이름으로 바꿔 씁니다.

---

3. 시큐리티 기본 동작 확인하기

3.1 모든 요청이 로그인 화면으로 가는 이유

설정 파일이나 클래스를 하나도 만들지 않은 상태에서 `BlogApplication` 을 실행합니다. 그리고 브라우저로 `http://localhost:8080` 에 접속합니다. 컨트롤러를 만들지 않았는데도 "Please sign in" 로그인 화면이 나타납니다. `http://localhost:8080/hello` 처럼 없는 주소로 접속해도 같은 화면으로 이동합니다.



브라우저에서 http://localhost:8080에 접속한 화면

생성형 예시 · 실제 실행 안 함

클래스패스에 `spring-boot-starter-security`가 있으면 스프링부트가 보안 기본 설정을 자동으로 등록합니다. 개발자가 보안 설정을 하나도 하지 않았을 때 이 기본 설정이 쓰이며, 규칙은 "모든 요청은 인증된 사용자만 통과"입니다. 인증되지 않은 브라우저 요청이 들어오면 스프링 시큐리티는 요청을 컨트롤러로 넘기지 않고 `/login`으로 리다이렉트합니다. 이 로그인 화면도 시큐리티가 만들어 줍니다.

보안 라이브러리는 기본값을 "모두 막기"로 둡니다. 개발자가 설정을 깜빡해도 정보가 새지 않게 하려는 것입니다. 6장에서 `SecurityFilterChain`을 직접 등록하면 이 기본 설정이 이 책의 규칙으로 바뀝니다. 요청이 컨트롤러에 닿기 전에 어디서 막히는지는 5장의 필터 체인에서 설명합니다.

3.2 임시 사용자와 비밀번호

로그인 화면에 넣을 값은 서버를 실행한 콘솔에 출력되어 있습니다.

실행 결과

출력

```
Using generated security password: 8e557245-73e2-4286-969a-ff57fe326336

This generated password is for development use only. Your security configuration must be
updated before running your application in production.
```

등록된 사용자 정보가 없어서 스프링부트가 임시 사용자 하나를 메모리에 만든 것입니다. 이 사용자의 아이디는 `user`이고 비밀번호는 무작위 값입니다. 비밀번호는 서버를 다시 실행할 때마다 바뀌므로 위 값과 독자의 콘솔에 나온 값은 다릅니다.

아이디에 `user`, 비밀번호에 콘솔에 나온 값을 넣고 로그인하면 다음 화면이 나옵니다.

실행 결과

출력

```
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallback.
... There was an unexpected error (type=Not Found, status=404).
```

로그인은 통과했지만 `/` 주소를 처리할 컨트롤러가 없어서 404가 나온 것입니다. 이 화면에서 요청은 시큐리티 검사를 먼저 통과해야 컨트롤러를 찾으러 간다는 것을 알 수 있습니다.

이 프로젝트에서는 시큐리티의 기본 로그인 기능을 쓰지 않습니다. 7장에서 로그인 API를 직접 만들고, 요청마다 JWT를 검사하는 필터로 인증합니다. 6장에서 `SecurityConfig`를 작성한 뒤에도 "Using generated security password" 로그는 계속 출력됩니다. 쓰지 않는 임시 사용자의 비밀번호이므로 무시해도 됩니다.

---

4. 프로파일 설정하기

4.1 공통 설정과 dev 설정 나누기

프로파일(profile)은 실행 환경에 맞는 설정 묶음을 골라 쓰게 해 주는 스프링 기능입니다. 개발할 때는 H2 메모리 DB를 쓰고 SQL 로그를 자세히 보는 편이 좋습니다. 실제 서비스에서는 다른 DB를 쓰고 로그를 줄여야 합니다. 설정을 한 파일에 모두 적으면 배포할 때마다 값을 고쳐야 합니다. 그래서 공통 값과 개발용 값을 파일 두 개에 나눠 적습니다.

파일	담는 값	적용 시점
`application.properties`	어느 환경에서나 같은 값, 사용할 프로파일 이름	항상
`application-dev.properties`	개발 환경에서만 쓰는 값(H2, SQL 로그, 개발용 CORS 출처)	`dev` 프로파일이 활성화되었을 때

`application.properties`에 `spring.profiles.active=dev`를 적으면 스프링부트는 파일 이름 규칙에 따라 `application-dev.properties`도 함께 읽습니다. 두 파일에 같은 키가 있으면 프로파일 파일의 값이 쓰입니다. 운영 환경이 생기면 `application-prod.properties`를 만들고 활성 프로파일을 `prod`로 바꿉니다.

4.2 H2 · JPA · data.sql 설정

`src/main/resources`에 설정 파일 두 개를 만듭니다. `spring.sql.init.data-locations`가 가리키는 `db/data.sql`의 내용은 2장에서 채웁니다. 다만 이 파일이 없으면 서버가 시작할 때 스크립트를 찾지 못해 실행이 멈춥니다. 그래서 지금은 주석 한 줄만 있는 파일을 만들어 둡니다.

예제 1-2 – 공통 설정과 dev 설정

이어서: 예제 1-1

파일명: `src/main/resources/application.properties`

PROPERTIES

```
spring.application.name=blog
# 프로필 설정: application-dev.properties 를 함께 읽는다
spring.profiles.active=dev
```

파일명: `src/main/resources/application-dev.properties`

PROPERTIES

```
# ===== Server =====
server.port=8080
server.servlet.encoding.charset=UTF-8
server.servlet.encoding.force=true

# ===== H2 Datasource (메모리 모드: 서버를 끄면 데이터가 사라짐) =====
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:mem:test
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true

# ===== JPA / Hibernate =====
# create: 실행할 때마다 테이블을 새로 만든다 (개발 단계 전용, 운영에서 사용 금지)
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false

# ===== SQL Init (data.sql) =====
spring.sql.init.data-locations=classpath:db/data.sql
# 테이블 생성(JPA) 이후에 data.sql 을 실행한다
spring.jpa.defer-datasource-initialization=true

# ===== Logging Level =====
logging.level.com.newtenco.blog=DEBUG

# ===== CORS Allow (콤마로 여러 개 지정 가능) =====
app.cors.allowed-origins=http://localhost:3000,http://localhost:5173
```

파일명: `src/main/resources/db/data.sql`

SQL

```
-- 더미 데이터는 2장에서 작성합니다
```

실행 결과

서버를 다시 실행하면 콘솔 앞부분에 활성 프로파일과 H2 콘솔 주소가 출력됩니다.

출력

```
The following 1 profile is active: "dev"
...
H2 console available at '/h2-console'. Database available at 'jdbc:h2:mem:test'
```

첫 줄이 보이면 `application-dev.properties`가 적용된 것입니다. 이 줄 대신 `No active profile set`이 나오면 `application.properties`의 `spring.profiles.active` 값을 확인합니다.

SSR 게시판에서 쓰던 설정과 뜻이 다르거나 처음 보는 H2 · JPA 설정은 다음과 같습니다.

설정	의미
`spring.datasource.url=jdbc:h2:mem:test`	`test` 라는 이름의 메모리 DB를 씁니다. 서버를 끄면 데이터가 모두 사라집니다
`spring.h2.console.enabled=true`	브라우저에서 DB를 볼 수 있는 `/h2-console` 을 엽니다
`spring.jpa.hibernate.ddl-auto=create`	실행할 때마다 기존 테이블을 지우고 엔티티를 보고 새로 만듭니다. 개발 단계 전용입니다
`spring.jpa.open-in-view=false`	트랜잭션이 끝나면 영속성 컨텍스트를 닫고 DB 연결을 반납합니다
`spring.sql.init.data-locations`	서버가 시작할 때 실행할 SQL 파일의 위치입니다
`spring.jpa.defer-datasource-initialization=true`	JPA가 테이블을 만든 다음에 data.sql을 실행합니다

`open-in-view`의 기본값은 `true`입니다. 이 값이 `true`면 응답이 나갈 때까지 DB 연결을 붙잡고 있어서, 요청이 몰리면 연결이 모자랄 수 있습니다. `false`로 두면 서비스의 트랜잭션 안에서 필요한 데이터를 모두 꺼내 DTO에 담아야 합니다. 7장에서 응답 DTO를 만드는 이유 중 하나도 `open-in-view`를 `false`로 두었기 때문입니다.

`defer-datasource-initialization`을 빼면 실행 순서가 뒤집힙니다. data.sql이 먼저 실행되어 아직 만들어지지 않은 테이블에 INSERT하다가 오류가 납니다.

지금 `/h2-console`에 접속하면 H2 콘솔이 아니라 3절의 로그인 화면이 나옵니다. 시큐리티가 모든 요청을 막고 있기 때문입니다. 콘솔을 여는 설정은 6장 5절에서 합니다.

4.3 CORS 허용 출처 값 미리 적어 두기

`application-dev.properties`의 마지막 줄은 스프링이 미리 정해 둔 키가 아닙니다. 이 프로젝트에서 직접 만든 키입니다.

코드 조각

PROPERTIES

```
app.cors.allowed-origins=http://localhost:3000,http://localhost:5173
```

브라우저는 기본적으로 다른 주소에서 불러온 화면이 이 서버로 보내는 요청을 막습니다. 이 값에는 요청을 허용할 화면의 주소를 적습니다. `3000`과 `5173`은 React 개발 서버가 주로 쓰는 포트입니다.

허용할 주소는 환경마다 다릅니다. 개발 중에는 `localhost`를, 운영에서는 실제 도메인을 허용해야 합니다. 그래서 이 값을 코드에 넣지 않고 프로파일 파일에 둡니다. IntelliJ가 이 키에 "Cannot resolve configuration property" 경고를 띄울 수 있습니다. 직접 만든 키라서 나오는 경고이므로 실행에는 문제가 없습니다.

CORS가 무엇이고 브라우저가 왜 요청을 막는지는 5장에서 설명합니다. 이 값을 읽어 허용 규칙을 만드는 코드는 6장 CORS 설정에서 작성합니다.

2장. 엔티티와 더미 데이터 만들기

02

1. User 엔티티

1.1 UserDetails 구현하기

UserDetails는 스프링 시큐리티가 사용자 한 명의 정보를 읽을 때 쓰는 인터페이스입니다. 시큐리티는 개발자가 만든 회원 클래스의 구조를 모릅니다. 그래서 아이디, 비밀번호, 권한을 `UserDetails`에 정해진 메서드 이름으로 꺼내 갑니다. User 엔티티가 이 인터페이스를 구현하면 DB에서 조회한 회원 객체를 그대로 로그인한 사용자로 쓸 수 있습니다.

반드시 구현해야 하는 메서드는 세 개입니다.

메서드	돌려주는 값	User 엔티티에서 채우는 방법
`getUsername()`	아이디	Lombok `@Getter`가 만든 메서드
`getPassword()`	암호화된 비밀번호	Lombok `@Getter`가 만든 메서드
`getAuthorities()`	권한 목록	직접 작성(1.3에서 설명)

`isAccountNonExpired()`, `isAccountNonLocked()`, `isCredentialsNonExpired()`, `isEnabled()`는 계정 상태를 묻는 메서드입니다. 스프링 시큐리티 6.3부터는 이 네 메서드가 `true`를 돌려주는 기본 메서드로 인터페이스 안에 들어 있습니다. 계정 정지나 휴면 정책이 없으면 따로 작성하지 않습니다.

예제 2-1 — User 엔티티 만들기

이어서: 예제 1-2

파일명: `src/main/java/com/newtenco/blog/user/User.java`

Java

```java
package com.newtenco.blog.user;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

@NoArgsConstructor
@AllArgsConstructor
@Builder
@Getter
@Entity
@Table(name = "user_tb")
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(unique = true, length = 20, nullable = false)
    private String username;

    @Column(length = 60, nullable = false)
    private String password;

    @Column(length = 30, nullable = false)
    private String email;

    // 권한 목록은 user_role_tb 테이블에 한 줄씩 저장된다
    // EAGER: 회원을 조회할 때 권한도 함께 가져온다
    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "user_role_tb", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    @Builder.Default
    private List<String> roles = new ArrayList<>(List.of("USER"));

    @CreationTimestamp
    private Timestamp createdAt;
```

```
    public void update(String email, String password) {
        this.email = email;
        this.password = password;
    }

    // hasRole("USER")로 검사하려면 권한 이름 앞에 ROLE_ 이 있어야 한다
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .toList();
    }
}
```

`getUsername()`과 `getPassword()`를 직접 작성하지 않아도 컴파일됩니다. `@Getter`가 필드 이름에 맞춰 같은 이름의 메서드를 만들기 때문입니다. 필드 이름을 `userId`나 `pwd`로 바꾸면 만들어지는 메서드 이름도 달라집니다. 그러면 인터페이스의 메서드를 구현하지 못해 컴파일 오류가 납니다. 필드 이름은 `username`, `password`로 둡니다.

`password` 열의 길이가 60인 것은 BCrypt로 암호화한 값이 60자이기 때문입니다. DB에는 평문 비밀번호를 저장하지 않습니다.

1.2 권한 목록과 @ElementCollection

회원 한 명이 권한을 여러 개 가질 수 있습니다. 일반 회원은 USER 하나만 갖고, 관리자는 USER와 ADMIN을 함께 갖습니다. 권한을 `"USER,ADMIN"`처럼 한 칸에 쉼표로 이어 넣으면 1차 정규화 규칙에 어긋납니다. 한 칸에는 값을 하나만 두어야 하기 때문입니다. 권한 하나만 지우거나 특정 권한을 가진 회원을 찾을 때마다 문자열을 잘라야 하는 불편도 생깁니다.

@ElementCollection은 엔티티가 아닌 값의 모음(여기서는 `List<String>`)을 별도 테이블에 저장하는 JPA 어노테이션입니다. 권한용 엔티티 클래스를 따로 만들지 않아도 JPA가 테이블을 만들고 회원 id로 연결합니다.

어노테이션	하는 일
`@ElementCollection(fetch = FetchType.EAGER)`	`roles`를 별도 테이블에 저장하고, 회원을 조회할 때 함께 읽어 옵니다
`@CollectionTable(name = "user_role_tb", ...)`	테이블 이름을 `user_role_tb`로, 회원을 가리키는 외래 키 열을 `user_id`로 정합니다
`@Column(name = "role")`	권한 값을 담는 열 이름을 `role`로 정합니다

어노테이션	하는 일
`@Builder.Default`	빌더로 회원을 만들 때도 기본값 `USER` 가 들어가게 합니다

두 테이블에는 다음처럼 저장됩니다. cos는 권한이 두 개라서 `user_role_tb` 에 두 줄이 생깁니다.

user_tb

id	username
1	ssar
2	cos

user_role_tb

user_id	role
1	USER
2	USER
2	ADMIN

`EAGER` 를 직접 적은 것은 지연 로딩 때문입니다. `@ElementCollection` 은 기본값이 지연 로딩(LAZY)이라서 값을 실제로 쓰는 순간에야 DB에서 읽어 옵니다. 그런데 1장에서 `spring.jpa.open-in-view=false` 로 설정했으므로 서비스의 트랜잭션이 끝나면 DB 연결이 반납됩니다. 그 뒤에 시큐리티가 `getAuthorities()` 를 부르면 아직 읽지 않은 권한을 가져오지 못해 `LazyInitializationException` 이 발생합니다. 권한은 회원을 쓸 때마다 필요한 값이므로 처음부터 함께 가져옵니다.

`@Builder.Default` 도 빼면 안 됩니다. 이 어노테이션이 없으면 빌더는 필드에 적은 초기값을 무시합니다. 그래서 `User.builder()...build()` 로 만든 객체의 `roles` 가 `null` 이 됩니다. 7장의 회원가입은 빌더로 회원을 만들기 때문에, 새 회원이 USER 권한을 자동으로 받으려면 이 어노테이션이 필요합니다.

1.3 getAuthorities()와 ROLE_ 접두사

`getAuthorities()` 는 회원의 권한 목록을 시큐리티가 이해하는 `GrantedAuthority` 객체 목록으로 바꿔 돌려줍니다. GrantedAuthority는 사용자에게 허락된 권한 하나를 나타내는 인터페이스입니다. `SimpleGrantedAuthority` 는 권한 이름 문자열 하나를 담는 기본 구현체입니다.

DB에는 `USER`, `ADMIN`만 저장하고, 돌려줄 때 앞에 `ROLE_`을 붙입니다. 6장에서는 접근 규칙을 `hasRole("ADMIN")`으로 씁니다. 이때 시큐리티는 `ROLE_ADMIN`이라는 권한이 있는지 검사합니다. `getAuthorities()`에서 `ROLE_`을 붙이지 않으면 관리자 계정으로도 관리자 API에 들어가지 못합니다.

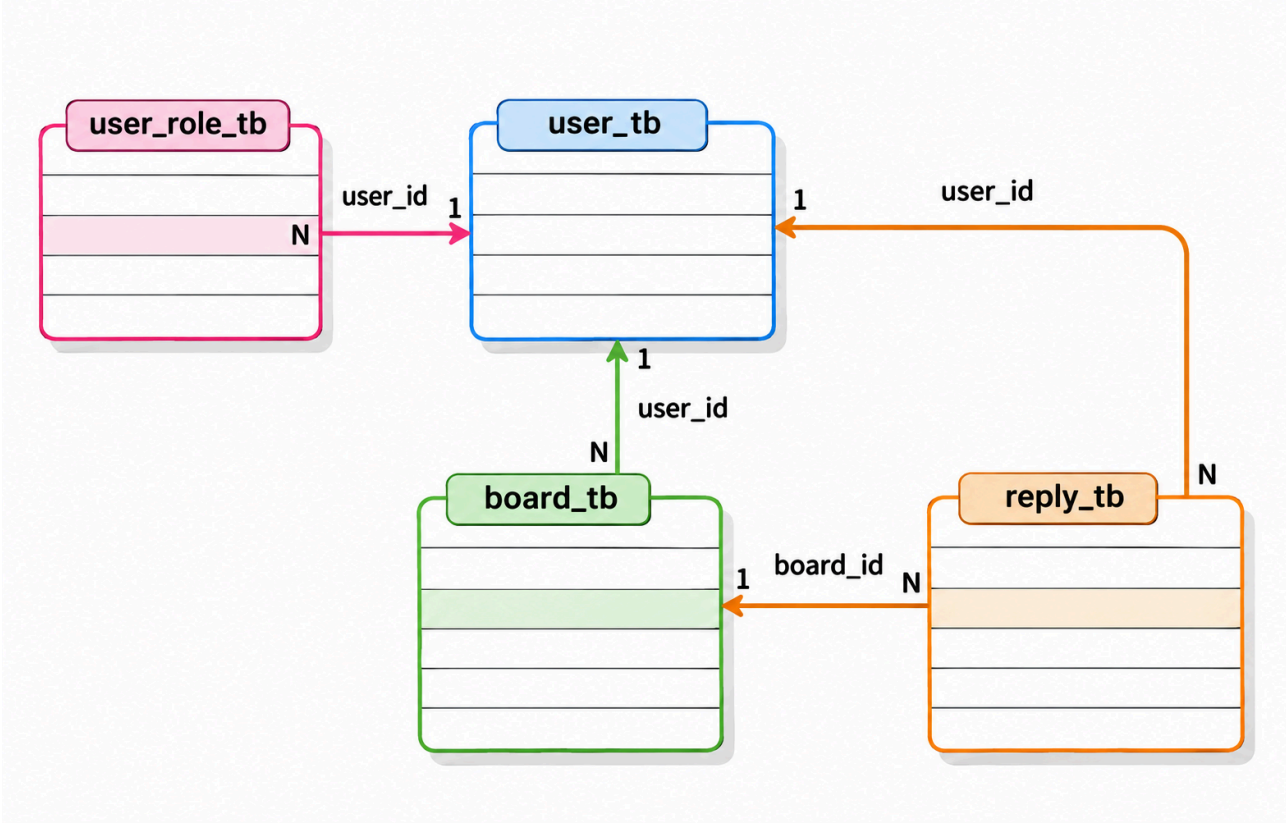
DB의 role 값	getAuthorities()가 돌려주는 권한	6장에서 검사하는 방법
USER	ROLE_USER	`hasRole("USER")`
ADMIN	ROLE_ADMIN	`hasRole("ADMIN")`

접두사를 붙이는 코드는 `getAuthorities()` 한 곳에만 있습니다. 그래서 DB 값과 접근 규칙 코드에는 `ROLE_` 없이 읽기 쉬운 이름만 씁니다.

---

2. Board · Reply 엔티티

이 책에서는 게시글과 댓글 API를 만들지 않습니다. 회원과 다른 데이터가 연결된 실제 프로젝트 구조를 갖추려고 테이블만 준비합니다. 나중에 게시판 API로 확장할 때 이 엔티티를 그대로 쓸 수 있습니다.



네 테이블의 관계

그림 2-1처럼 게시글은 작성자 한 명을 가리키고, 댓글은 작성자 한 명과 게시글 하나를 가리킵니다. 외래 키는 모두 N 쪽 테이블에 생깁니다.

예제 2-2 – 게시글과 댓글 엔티티 만들기

이어서: 예제 2-1

파일명: `src/main/java/com/newtenco/blog/board/Board.java`

Java

```java
package com.newtenco.blog.board;

import com.newtenco.blog.reply.Reply;
import com.newtenco.blog.user.User;
import jakarta.persistence.*;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;

import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;

@NoArgsConstructor
@Getter
@Entity
@Table(name = "board_tb")
public class Board {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 100, nullable = false)
    private String title;

    @Column(length = 1000, nullable = false)
    private String content;

    // 게시글(N) : 회원(1) -> 외래 키 user_id 는 board_tb 에 생긴다
    @ManyToOne(fetch = FetchType.LAZY)
    private User user;

    // 연관관계의 주인이 아님(mappedBy) -> 조회 전용
    @OneToMany(mappedBy = "board", fetch = FetchType.LAZY, cascade = CascadeType.REMOVE)
    private List<Reply> replies = new ArrayList<>();

    @CreationTimestamp
    private Timestamp createdAt;

    @Builder
    public Board(Integer id, String title, String content, User user, Timestamp createdAt) {
        this.id = id;
        this.title = title;
        this.content = content;
```

```
        this.user = user;
        this.createdAt = createdAt;
    }

    public void update(String title, String content) {
        this.title = title;
        this.content = content;
    }
}
```

파일명: `src/main/java/com/newtenco/blog/reply/Reply.java`

Java

```java
package com.newtenco.blog.reply;

import com.newtenco.blog.board.Board;
import com.newtenco.blog.user.User;
import jakarta.persistence.*;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import org.hibernate.annotations.CreationTimestamp;

import java.sql.Timestamp;

@NoArgsConstructor
@Getter
@Entity
@Table(name = "reply_tb")
public class Reply {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(length = 100, nullable = false)
    private String comment;

    @ManyToOne(fetch = FetchType.LAZY)
    private User user;

    // 연관관계의 주인: 외래 키 board_id 를 관리한다
    @ManyToOne(fetch = FetchType.LAZY)
    private Board board;

    @CreationTimestamp
    private Timestamp createdAt;

    @Builder
    public Reply(Integer id, String comment, User user, Board board, Timestamp createdAt) {
        this.id = id;
        this.comment = comment;
        this.user = user;
        this.board = board;
        this.createdAt = createdAt;
    }
}
```

연관관계 매핑은 SSR 게시판에서 써 본 방식과 같습니다. 여기서는 3절의 더미 데이터와 이어지는 부분만 짚습니다.

어노테이션	위치	역할
`@ManyToOne`	`Board.user`, `Reply.user`, `Reply.board`	N:1 관계입니다. 외래 키가 이 엔티티의 테이블에 생기며, 이쪽이 **연관관계의 주인**입니다
`@OneToMany(mappedBy = "board")`	`Board.replies`	주인이 아니므로 외래 키를 바꾸지 못하고 조회만 합니다

외래 키 열 이름은 `필드 이름_참조 테이블의 기본 키 이름` 규칙으로 정해집니다. 그래서 `Board.user`는 `user_id`, `Reply.board`는 `board_id` 열이 됩니다. 3절의 data.sql은 이 열 이름으로 값을 넣습니다. `mappedBy`의 `"board"`는 열 이름이 아니라 `Reply` 클래스의 필드 이름입니다.

`cascade = CascadeType.REMOVE`를 적으면 게시글을 지울 때 딸린 댓글도 함께 지워집니다. 이 설정이 없으면 댓글이 달린 게시글을 지울 때 외래 키 제약 때문에 삭제가 실패합니다.

---

3. 더미 데이터 넣기

3.1 data.sql 작성

더미 데이터는 개발 중에 API 동작을 확인하려고 미리 넣어 두는 시험용 데이터입니다. 1장 4.2에서 `spring.sql.init.data-locations=classpath:db/data.sql`로 파일 위치를 정하고 빈 파일을 만들어 두었습니다. `spring.jpa.defer-datasource-initialization=true` 설정 덕분에 이 파일은 JPA가 테이블을 만든 다음에 실행됩니다. H2 메모리 DB는 서버를 켤 때마다 빈 상태로 시작하므로, 실행할 때마다 같은 데이터가 다시 들어갑니다.

예제 2-3 — 더미 데이터 넣기

이어서: 예제 2-2

파일명: `src/main/resources/db/data.sql`

SQL

```sql
-- 비밀번호는 모두 1234 를 BCrypt 로 암호화한 값이다
insert into user_tb (username, password, email, created_at) values ('ssar',
'$2a$10$eiDus5jn3LsNyfjng4/HxukQh.7EZmn1ZNL1g4s3kkqPYpwY5DiuO', 'ssar@metacoding.com', now());
insert into user_tb (username, password, email, created_at) values ('cos',
'$2a$10$eiDus5jn3LsNyfjng4/HxukQh.7EZmn1ZNL1g4s3kkqPYpwY5DiuO', 'cos@metacoding.com', now());

-- 권한은 한 줄에 하나씩 (ssar: USER / cos: USER, ADMIN)
insert into user_role_tb (user_id, role) values (1, 'USER');
insert into user_role_tb (user_id, role) values (2, 'USER');
insert into user_role_tb (user_id, role) values (2, 'ADMIN');

insert into board_tb (title, content, user_id, created_at) values ('제목1', '내용1', 1, now());
insert into board_tb (title, content, user_id, created_at) values ('제목2', '내용2', 1, now());
insert into board_tb (title, content, user_id, created_at) values ('제목3', '내용3', 1, now());
insert into board_tb (title, content, user_id, created_at) values ('제목4', '내용4', 2, now());
insert into board_tb (title, content, user_id, created_at) values ('제목5', '내용5', 2, now());

insert into reply_tb (comment, board_id, user_id, created_at) values ('댓글1', 4, 2, now());
insert into reply_tb (comment, board_id, user_id, created_at) values ('댓글2', 4, 2, now());
insert into reply_tb (comment, board_id, user_id, created_at) values ('댓글3', 4, 2, now());
insert into reply_tb (comment, board_id, user_id, created_at) values ('댓글4', 5, 1, now());
insert into reply_tb (comment, board_id, user_id, created_at) values ('댓글5', 5, 1, now());
```

데이터는 참조되는 테이블부터 넣습니다. `user_role_tb`, `board_tb`, `reply_tb`는 모두 `user_tb`의 id를 외래 키로 가리킵니다. 회원보다 권한이나 게시글을 먼저 넣으면 외래 키 제약 위반이 일어나 서버가 시작하지 못합니다. 댓글을 게시글 다음에 넣는 것도 같은 이유입니다.

`user_id`에 1과 2를 바로 적을 수 있는 것은 `IDENTITY` 전략이 id를 1부터 차례로 붙이기 때문입니다. 먼저 넣은 ssar가 1번, cos가 2번이 됩니다.

권한을 `user_role_tb`에 직접 넣는 것은 엔티티의 기본값이 SQL에는 적용되지 않기 때문입니다. 엔티티에 적은 기본값 `USER`는 자바에서 객체를 만들 때만 들어갑니다. SQL로 `user_tb`에 바로 넣은 행에는 이 기본값이 들어가지 않습니다.

3.2 테스트 계정(ssar · cos, 비밀번호 1234)

로그인 실습에서는 다음 두 계정을 씁니다.

아이디	비밀번호	권한	쓰임
ssar	1234	USER	일반 회원으로 요청할 때

아이디	비밀번호	권한	쓰임
cos	1234	USER, ADMIN	관리자 API를 요청할 때

`password` 열에 들어 있는 값은 `1234`가 아니라 `$2a$10$`로 시작하는 60자 문자열입니다. `1234`를 BCrypt로 암호화한 값입니다. 로그인할 때는 평문 `1234`를 입력하고, 서버가 입력값과 저장된 암호문을 비교합니다. 암호화와 비교 원리는 7장에서 설명합니다.

---

4. 지금 확인할 수 있는 것

4.1 서버 실행 로그에서 테이블 생성 확인

아직 컨트롤러와 시큐리티 설정이 없어서 API로는 확인할 수 없습니다. 대신 서버를 실행하고, 엔티티가 테이블로 만들어졌는지 로그에서 봅니다. 1장에서 `spring.jpa.show-sql=true`와 `format_sql=true`를 켜 두었으므로 Hibernate가 실행한 DDL이 줄을 맞춰 콘솔에 출력됩니다.

IntelliJ에서 `BlogApplication`의 `main()`을 실행합니다.

실행 결과

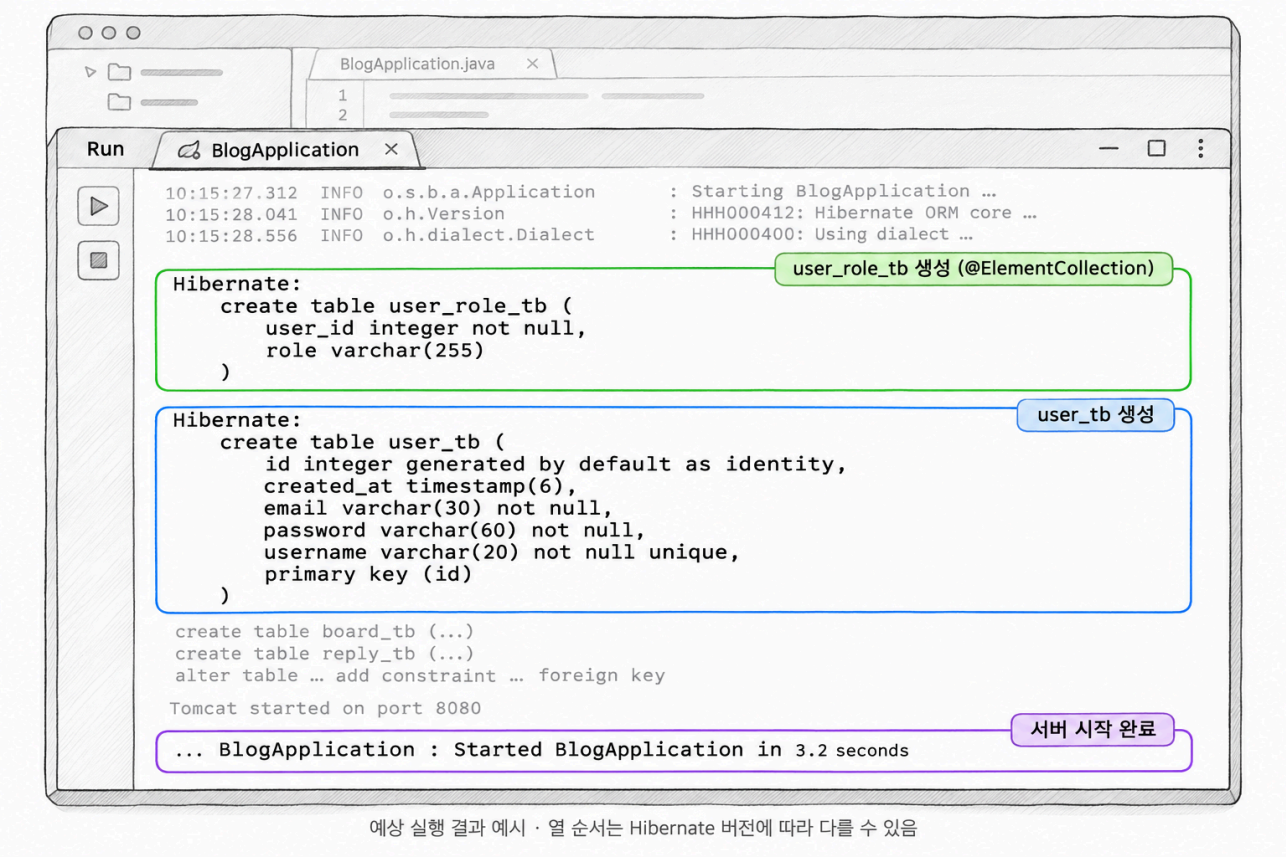
콘솔에 `create table` 문이 네 개 출력됩니다. 다음은 그중 `user_tb`와 `user_role_tb` 부분입니다.

출력

```
Hibernate:
    create table user_role_tb (
        user_id integer not null,
        role varchar(255)
    )
Hibernate:
    create table user_tb (
        id integer generated by default as identity,
        created_at timestamp(6),
        email varchar(30) not null,
        password varchar(60) not null,
        username varchar(20) not null unique,
        primary key (id)
    )
```

`board_tb`와 `reply_tb`도 같은 형식으로 출력됩니다. 열 순서와 unique 제약의 표기는 Hibernate 버전에 따라 다를 수 있습니다 [확인 필요]. 이어서 `alter table ... add constraint ... foreign key` 문

이 나오면서 외래 키가 연결됩니다.



예상 실행 결과 예시 · 열 순서는 Hibernate 버전에 따라 다를 수 있음

IntelliJ에서 서버를 실행한 Run 창

생성형 예시 · 실제 실행 안 함

로그에서는 `@Table`에 적은 이름대로 테이블이 생겼는지, `@ElementCollection` 때문에 `user_role_tb`가 따로 생겼는지 확인합니다. 서버가 `Started BlogApplication`까지 오류 없이 떴는지도 봅니다.

data.sql의 `insert` 문은 로그에 보이지 않습니다. `insert` 문은 Hibernate가 아니라 스프링이 JDBC로 직접 실행하기 때문에 `show-sql` 로그에 나오지 않습니다. 서버가 정상으로 떴다면 data.sql도 오류 없이 실행된 것입니다. 열 이름이 틀렸거나 넣는 순서가 잘못되었다면 서버가 시작하지 못하고 SQL 오류를 출력합니다. 들어간 데이터는 6장에서 H2 콘솔을 연 뒤 직접 조회합니다.

3장. JWT로 토큰 만들고 검증하기

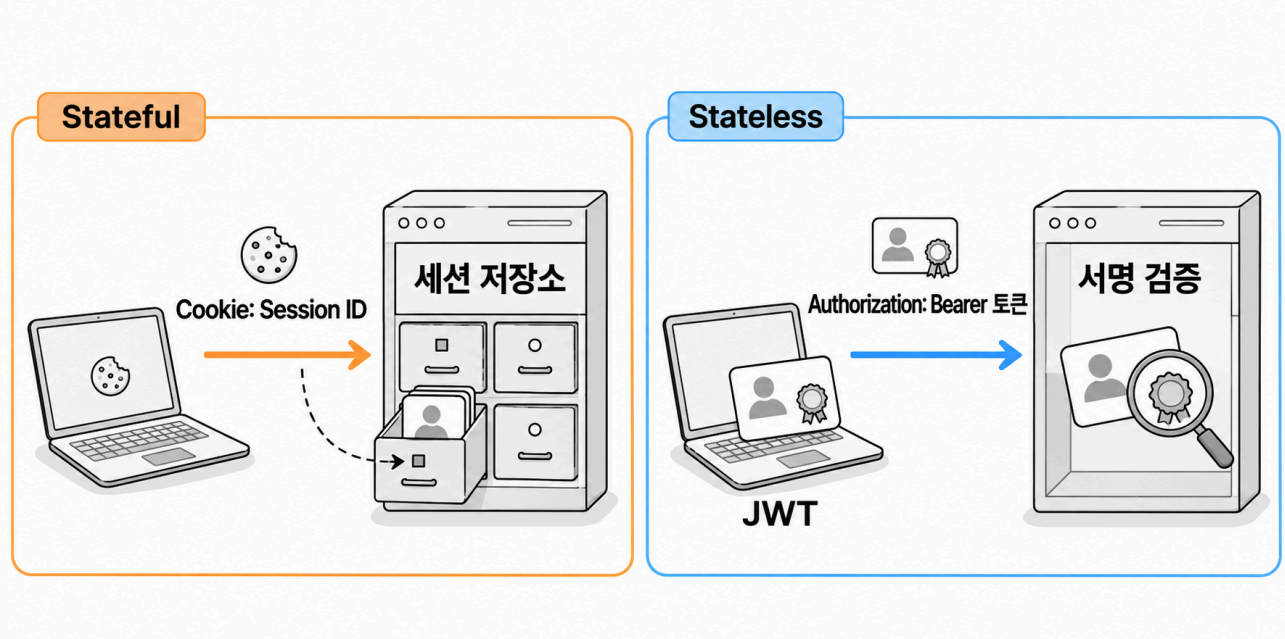
03

1. 세션 방식과 JWT 방식

1.1 Stateful과 Stateless

SSR 게시판에서는 로그인에 성공하면 서버가 **세션(session)**에 사용자 정보를 저장했습니다. 브라우저는 쿠키에 세션 ID 하나만 담아 보냈습니다. 서버는 요청이 올 때마다 그 세션 ID로 세션 저장소를 찾아 요청한 사람을 알아냈습니다. 이처럼 서버가 로그인 상태를 기억하는 방식을 **Stateful(상태 유지)**이라고 합니다.

JWT(JSON Web Token)는 로그인 정보를 JSON으로 담고 서버의 비밀키로 서명한 문자열입니다. 서버는 로그인에 성공한 사용자에게 토큰을 발급하지만 발급한 토큰을 따로 저장하지는 않습니다. 클라이언트가 요청마다 토큰을 보내면, 서버는 서명을 검증해 보낸 사람을 판단합니다. 서버가 로그인 상태를 기억하지 않으므로 이 방식을 **Stateless(무상태)**라고 합니다.



세션 방식과 JWT 방식

그림 3-1처럼 두 방식은 로그인 정보를 보관하는 곳이 다릅니다. 아래 표에 정리한 나머지 차이도 모두 이 차이에서 나옵니다.

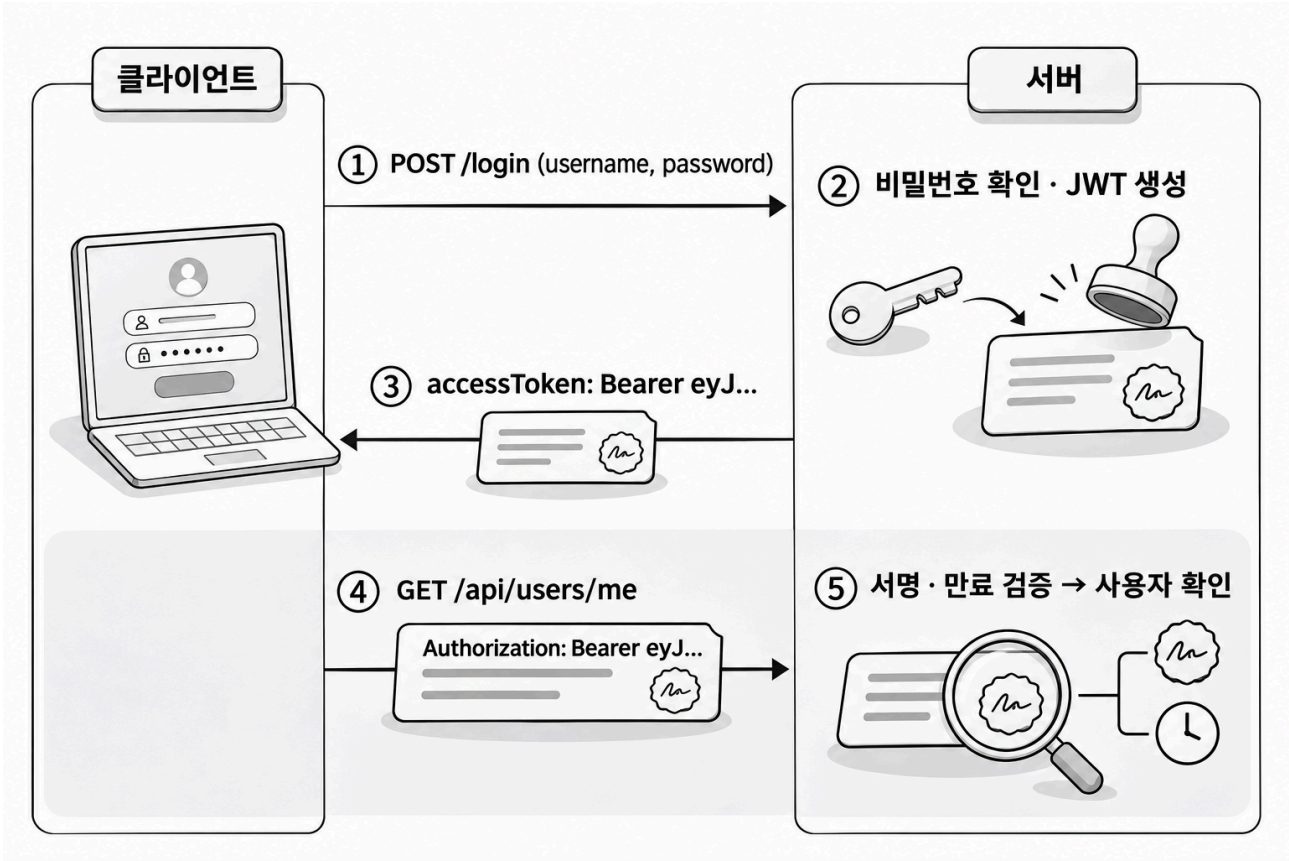
구분	세션	JWT
로그인 정보 저장 위치	서버 메모리	클라이언트(토큰)
서버 상태	Stateful	Stateless
요청마다 보내는 값	쿠키의 세션 ID	`Authorization` 헤더의 토큰
서버를 여러 대로 늘릴 때	서버끼리 세션을 공유하도록 설정해야 함	같은 비밀키로 서명만 검증하면 됨
강제 로그아웃	쉬움(세션 삭제)	어려움(만료 전까지 유효)
주 사용처	SSR 웹	REST API, SPA, 모바일 앱

JWT 방식에는 약점이 하나 있습니다. 서버가 토큰을 저장하지 않으므로, 이미 발급한 토큰을 서버에서 지워 무효로 만들 수 없습니다. 그래서 토큰에 만료 시간을 넣어 쓸 수 있는 기간을 제한합니다.

1장에서 본 것처럼 REST API는 브라우저 말고도 모바일 앱이나 다른 서버가 호출합니다. JWT는 쿠키와 세션에 묶이지 않으므로 이런 환경에 잘 맞습니다.

1.2 JWT 로그인 흐름

JWT 로그인은 토큰을 받는 단계와 받은 토큰을 쓰는 단계로 나뉩니다.



JWT 로그인 흐름

그림 3-2의 다섯 단계를 이 책의 API로 적으면 다음과 같습니다.

1. 클라이언트가 `POST /login`으로 `{"username":"ssar","password":"1234"}`를 보냅니다.
2. 서버는 DB의 비밀번호와 비교하고, 일치하면 JWT를 만듭니다.
3. 서버는 응답 본문에 `"accessToken":"Bearer eyJ..."`를 담아 돌려줍니다.
4. 클라이언트는 받은 토큰을 저장해 두고, 요청할 때마다 `Authorization: Bearer eyJ...` 헤더에 넣어 보냅니다.
5. 서버는 토큰의 서명과 만료 시간을 검증한 뒤 토큰에 담긴 정보로 사용자를 알아냅니다.

토큰 앞의 **Bearer**는 "이 토큰을 가진 사람에게 권한을 준다"는 뜻의 인증 방식 이름입니다. `Authorization` 헤더로 토큰을 보낼 때는 이 접두사를 붙이는 것이 관례입니다. 5단계에서 서버는 세션 저장소를 찾지 않고 토큰만 검사합니다.

2단계는 7장의 로그인 API에서, 5단계는 5장의 JWT 필터에서 구현합니다. 이 장에서는 두 곳에서 함께 쓸 JwtUtil을 만듭니다.

---

2. JWT의 구조

2.1 헤더 · 페이로드 · 서명

JWT는 점(`.`) 두 개로 구분된 세 부분짜리 문자열입니다.

기본 문법

출력

```
헤더.페이로드.서명
```

부분	담는 내용	예
헤더(Header)	서명 알고리즘, 토큰 종류	`{"alg":"HS512","typ":"JWT"}`
페이로드(Payload)	전달할 데이터(클레임)	`{"sub":"ssar","id":1,"roles":["USER"],"exp":1767000000}`
서명(Signature)	헤더와 페이로드를 비밀키로 계산한 값	위조 여부 판단

헤더와 페이로드는 JSON을 **Base64Url**로 인코딩한 문자열입니다. Base64Url은 데이터를 URL이나 HTTP 헤더에 그대로 넣을 수 있는 글자(영문, 숫자, `-`, `_`)로 바꾸는 방식입니다. JWT가 늘 `eyJ`로 시작하는 것도 이 인코딩 때문입니다. 헤더 JSON은 항상 `{"` 다음에 영문자가 오는 모양으로 시작하고, 이 앞부분을 Base64Url로 바꾸면 `eyJ`가 됩니다.

서명은 다음과 같이 계산합니다.

기본 문법

출력

```
서명 = HMACSHA512( Base64Url(헤더) + "." + Base64Url(페이로드), 비밀키 )
```

이 책은 **HS512**(HMAC-SHA512) 알고리즘을 씁니다. HMAC은 서명을 만들 때와 검증할 때 같은 비밀키를 쓰는 방식입니다. 서버는 받은 토큰의 헤더와 페이로드로 서명을 다시 계산해, 토큰에 붙어 온 서명과 비교합니다. 두 값이 같으면 이 서버가 발급한 뒤로 아무도 고치지 않은 토큰입니다.

2.2 표준 클레임

페이로드에 담는 이름과 값의 쌍 하나하나를 **클레임(claim)**이라고 합니다. JWT 표준(RFC 7519)은 자주 쓰는 클레임의 이름을 세 글자로 정해 두었습니다. 이 책에서 쓰는 표준 클레임은 다음과 같습니다.

표준 클레임	이름	의미
`sub`	Subject	토큰의 주인. 이 책에서는 username
`exp`	Expiration Time	만료 시간
`iat`	Issued At	발급 시간

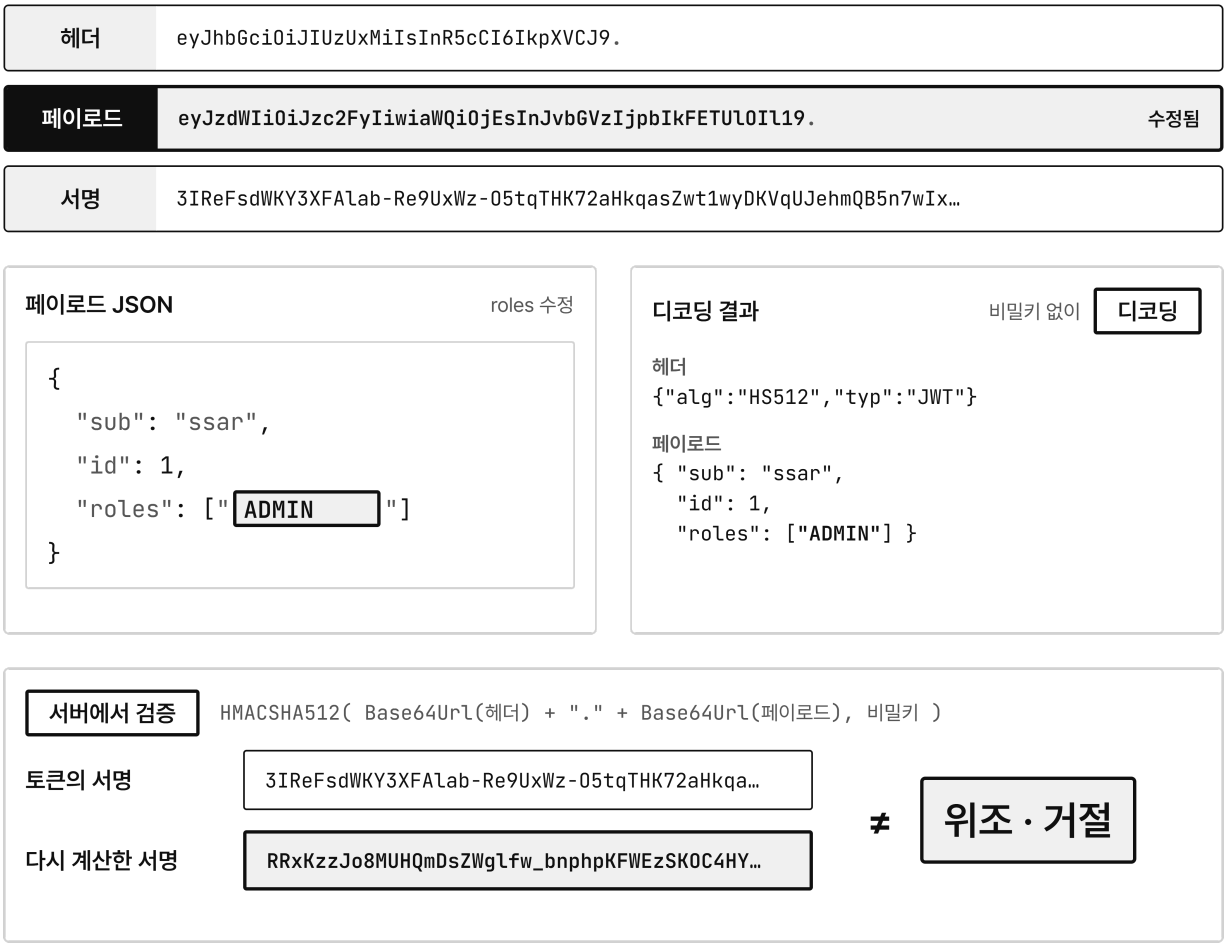
발급자 `iss`, 대상 `aud` 같은 표준 클레임도 있지만 이 책에서는 쓰지 않습니다. `exp`와 `iat`의 값은 1970년 1월 1일부터 흐른 시간을 **초** 단위로 센 숫자입니다. JwtUtil은 만료 시간을 자바 `Date`로 넘기고, 초 단위로 바꾸는 일은 라이브러리가 맡습니다.

표준에 없는 클레임은 이름을 직접 정해서 넣습니다. 이 책에서는 사용자 번호 `id`와 권한 목록 `roles`를 넣습니다. 토큰만 보고 사용자 번호와 권한을 알 수 있으니, 요청마다 DB를 조회하지 않고도 권한을 검사할 수 있습니다.

2.3 페이로드는 암호화가 아니다

Base64Url은 인코딩이지 암호화가 아닙니다. 비밀키가 없어도 누구나 원래 JSON으로 되돌릴 수 있습니다. jwt.io 같은 사이트에 토큰을 붙여 넣으면 페이로드 내용이 그대로 보입니다. 그래서 페이로드에는 비밀번호나 주민등록번호 같은 민감한 정보를 넣지 않습니다.

JWT가 지켜 주는 것은 내용의 비밀이 아니라 **위조 방지**입니다. 누군가 페이로드의 `"roles":["USER"]`를 `["ADMIN"]`으로 고치면 페이로드 문자열이 달라집니다. 그러면 서버가 다시 계산한 서명이 토큰에 붙은 서명과 맞지 않아 토큰이 거절됩니다. 서명까지 새로 맞추려면 비밀키가 필요한데, 비밀키는 서버만 갖고 있습니다.



페이로드를 고치면 서명이 맞지 않는다

그림 3-3의 시뮬레이터에서 직접 고쳐 보면 차이가 분명히 보입니다. 디코딩에는 비밀키가 필요 없고, 서명을 맞추는 데는 비밀키가 필요합니다. 실제 토큰을 jwt.io에서 열어 보는 일은 7장에서 로그인 API로 토큰을 받은 뒤에 합니다.

---

3. JwtUtil 만들기

JwtUtil은 토큰을 만들고 검증하는 일만 하는 클래스입니다. HTTP 요청이나 스프링 시큐리티를 전혀 쓰지 않는 순수 자바 코드입니다. `User`를 받으면 토큰을 만들고(`create()`), 토큰을 받으면 검증한 뒤 `User`를 돌려줍니다(`verify()`). 역할을 이렇게 좁혀 두었기 때문에 7장의 로그인 서비스와 5장의 JWT 필터가 같은 클래스를 그대로 가져다 씁니다.

메서드가 모두 `static`이라 객체를 만들거나 빈으로 등록할 필요가 없습니다. `JwtUtil.create(user)`처럼 바로 호출합니다. 토큰 생성과 검증은 1장에서 추가한 java-jwt 라이브러리(`com.auth0:java-jwt:4.5.0`)가 맡습니다.

예제 3-1 — JwtUtil 만들기

이어서: 예제 2-3

파일명: `src/main/java/com/newtenco/blog/core/util/JwtUtil.java`

Java

```java
package com.newtenco.blog.core.util;

import com.auth0.jwt.JWT;
import com.auth0.jwt.algorithms.Algorithm;
import com.auth0.jwt.interfaces.DecodedJWT;
import com.newtenco.blog.user.User;

import java.util.Date;
import java.util.List;

/**
 * JWT 를 만들고(create) 검증하는(verify) 순수 자바 도구.
 * HTTP 나 시큐리티를 전혀 모른다.
 */
public class JwtUtil {

    public static final String HEADER = "Authorization";   // 토큰을 담는 HTTP 헤더 이름
    public static final String TOKEN_PREFIX = "Bearer ";   // 토큰 앞에 붙는 접두사 (공백 포함 7글자)
    public static final long EXPIRATION_TIME = 1000L * 60 * 60 * 24; // 유효기간 1일

    // 서명용 비밀키. HMAC512 는 64바이트(영문 64자) 이상을 권장한다.
    // 실무에서는 코드에 두지 않고 환경 변수로 관리한다.
    private static final String SECRET = "newtenco-blog-jwt-secret-key-must-be-at-least-64-
bytes-long-0123456789";
    private static final Algorithm ALGORITHM = Algorithm.HMAC512(SECRET);

    // 토큰 생성: "Bearer eyJ..." 형태로 반환
    public static String create(User user) {
        String jwt = JWT.create()
                .withSubject(user.getUsername())                    // sub: 누구의 토큰인가
                .withExpiresAt(new Date(System.currentTimeMillis() + EXPIRATION_TIME)) // exp:
만료 시간
                .withClaim("id", user.getId())                      // 사용자 정의 클레임
                .withClaim("roles", user.getRoles())                // ["USER", "ADMIN"]
                .sign(ALGORITHM);                                   // 비밀키로 서명
        return TOKEN_PREFIX + jwt;
    }

    // 토큰 검증: 서명이 틀리거나 만료되면 JWTVerificationException 이 발생한다
    public static User verify(String jwt) {
        DecodedJWT decoded = JWT.require(ALGORITHM).build().verify(jwt);

        Integer id = decoded.getClaim("id").asInt();
        String username = decoded.getSubject();
        List<String> roles = decoded.getClaim("roles").asList(String.class);
```

```
        // DB 조회 없이 토큰 안의 정보만으로 User 객체를 복원한다 (비밀번호는 없음)
        return User.builder().id(id).username(username).roles(roles).build();
    }
}
```

`HEADER`와 `TOKEN_PREFIX`는 5장에서 요청 헤더의 토큰을 꺼낼 때 씁니다. 아직 이 클래스를 호출하는 코드가 없으니, 4절에서는 서버가 정상으로 뜨는지만 확인합니다.

3.1 토큰 생성 create()

`create()`는 `JWT.create()`로 토큰 빌더를 열고 클레임을 하나씩 넣은 다음, `sign()`으로 서명해 토큰 문자열을 완성합니다. ssar 계정으로 토큰을 만들면 각 줄에 다음 값이 들어갑니다.

코드	들어가는 클레임	값
`withSubject(user.getUsername())`	`sub`	`"ssar"`
`withExpiresAt(...)`	`exp`	지금부터 1일 뒤
`withClaim("id", user.getId())`	`id`	`1`
`withClaim("roles", user.getRoles())`	`roles`	`["USER"]`
`sign(ALGORITHM)`	서명	HS512로 계산한 값

만료 시간은 현재 시각 `System.currentTimeMillis()`(밀리초)에 `EXPIRATION_TIME`을 더한 값입니다. `1000L * 60 * 60 * 24`는 1000밀리초 × 60초 × 60분 × 24시간, 즉 하루입니다. 8장 연습문제에서는 이 값을 줄여서 만료된 토큰이 거절되는 모습을 확인합니다.

`roles`는 2장에서 `List<String>`으로 만든 권한 목록입니다. `withClaim()`에 리스트를 넘기면 토큰 안에는 JSON 배열로 들어갑니다.

예제 3-1에는 발급 시간 `iat`이 없습니다. 필요하면 `.withIssuedAt(new Date())`를 한 줄 추가합니다.

마지막 줄은 완성된 토큰 앞에 `TOKEN_PREFIX`(`"Bearer "`)를 붙여서 돌려줍니다. 그래서 7장 로그인 응답의 `accessToken`은 `Bearer eyJ...` 모양이 됩니다. 클라이언트는 이 문자열을 고치지 않고 그대로 `Authorization` 헤더 값으로 넣으면 됩니다.

3.2 토큰 검증 verify()

`verify()`는 `Bearer `를 뗀 토큰 문자열만 받습니다. 접두사를 떼는 일은 5장의 JwtProvider가 합니다.

코드 조각

Java

```
DecodedJWT decoded = JWT.require(ALGORITHM).build().verify(jwt);
```

`JWT.require(ALGORITHM).build()`는 토큰을 만들 때와 같은 알고리즘·비밀키로 검증기를 만듭니다. 검증기의 `verify()`는 아래 항목을 확인하고, 하나라도 어긋나면 예외를 던집니다.

검사	실패할 때 던지는 예외
점으로 나뉜 세 부분이고 Base64Url로 읽히는지	`JWTDecodeException`
헤더의 알고리즘이 HS512인지	`AlgorithmMismatchException`
다시 계산한 서명이 토큰의 서명과 같은지	`SignatureVerificationException`
`exp`가 지나지 않았는지	`TokenExpiredException`

네 예외는 모두 `JWTVerificationException`의 자식 클래스입니다. 그래서 토큰을 쓰는 쪽에서 `JWTVerificationException` 하나만 잡으면 형식 오류, 위조, 만료를 한꺼번에 처리할 수 있습니다. 5장의 JWT 필터는 이 예외를 잡아 401 응답으로 바꿉니다.

검증을 통과하면 `DecodedJWT`에서 클레임을 꺼냅니다. `sub`는 `getSubject()`로, `id` 같은 숫자 클레임은 `getClaim("id").asInt()`로 꺼냅니다. 리스트로 넣은 `roles`는 `asList(String.class)`로 꺼내야 다시 `List<String>`이 됩니다.

꺼낸 값으로는 `User.builder()`를 써서 `User` 객체를 새로 만듭니다. DB를 조회하지 않고 토큰만으로 만든 객체라서 `password`와 `email`은 `null`입니다. 그래도 사용자가 누구인지(id, username), 무엇을 할 수 있는지(roles)는 이 정보로 판단할 수 있습니다. 서명 검증을 통과했다면 이 값들은 서버가 직접 넣은 것이므로 믿고 써도 됩니다. 이메일처럼 토큰에 없는 정보가 필요하면 8장에서처럼 id로 DB를 다시 조회합니다.

3.3 비밀키 관리

토큰에 서명할 때도, 서명을 검증할 때도 쓰는 열쇠는 비밀키 하나뿐입니다. 비밀키가 새어 나가면 누구나 `"roles":["ADMIN"]`이 든 토큰을 만들어 서명할 수 있고, 서버는 그 토큰을 진짜로 받아들입니다. 그래서 비밀키는 길이와 보관 위치를 함께 신경 써야 합니다.

먼저 길이입니다. JWT 알고리즘 표준(RFC 7518)은 HMAC 키가 해시 결과보다 짧으면 안 된다고 정합니다. HS512의 해시 결과는 512비트(64바이트)이므로 키도 64바이트 이상이어야 합니다. 짧은 키는 가능한 조합을 모두 대입해 보는 공격에 약합니다. 예제 3-1의 `SECRET`은 영문 · 숫자 · 하이픈으로 된 70자(70바이트)입니다.

다음은 보관 위치입니다. 예제 3-1은 실습하기 편하도록 키를 코드에 상수로 넣었습니다. 이 코드를 GitHub 같은 공개 저장소에 올리면 키도 함께 공개됩니다. 실무에서는 키를 서버의 환경 변수에 두고, 코드에서는 읽기만 합니다.

코드 조각

Java

```
private static final String SECRET = System.getenv("JWT_SECRET");
```

이렇게 바꾸면 서버를 실행하는 컴퓨터에 `JWT_SECRET` 환경 변수가 있어야 합니다. 주의할 점은 환경 변수가 없어도 서버가 정상으로 뜬다는 것입니다. `JwtUtil`은 빈이 아니어서 클래스가 처음 쓰이는 순간에 초기화됩니다. 첫 로그인이나 첫 토큰 검증 때 `SECRET`이 `null`인 채로 `Algorithm.HMAC512()`가 호출되면 `IllegalArgumentException`이 납니다. 정적 필드를 초기화하다가 난 예외라서 `ExceptionInInitializerError`로 감싸지고, 그 요청은 실패합니다. 이 오류는 `Exception`이 아니라 `Error`이므로 5장 `JwtProvider`의 `catch`에도 잡히지 않습니다. 이 책은 실습 흐름을 단순하게 하려고 예제 3-1의 상수를 그대로 씁니다.

비밀키를 바꾸면 그전에 발급한 토큰은 모두 서명 검증에 실패합니다. 따라서 키가 유출되었을 때 키를 바꾸면, 그동안 발급한 토큰을 한꺼번에 무효로 만들 수 있습니다.

---

4. 지금 확인할 수 있는 것

4.1 서버 정상 실행 확인

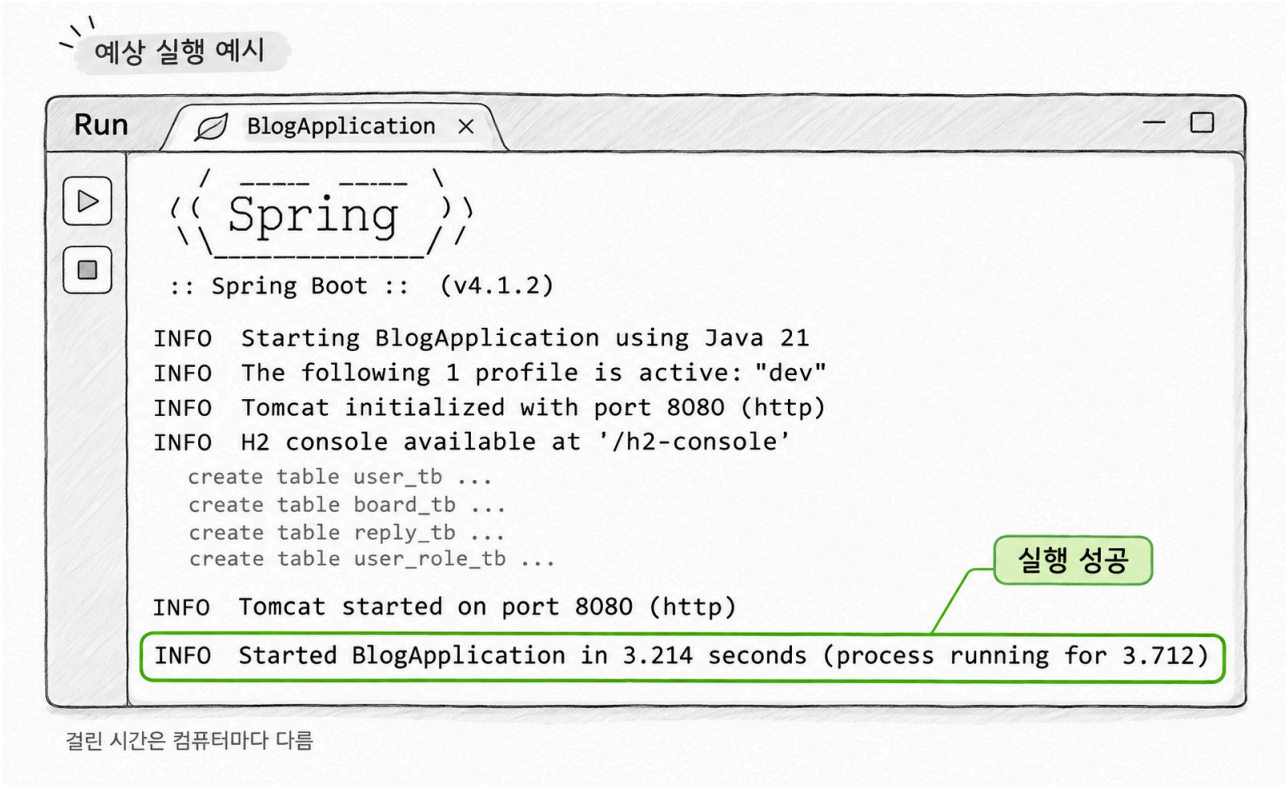
JwtUtil을 호출하는 코드가 아직 없으므로, 지금은 코드가 컴파일되고 서버가 정상으로 뜨는지만 확인합니다. IntelliJ에서 `BlogApplication`의 실행 버튼을 누릅니다.

실행 결과

콘솔 마지막 부분에 다음과 같은 줄이 나오면 성공입니다. 걸린 시간은 컴퓨터마다 다릅니다.

출력

```
Started BlogApplication in 3.214 seconds (process running for 3.712)
```



IntelliJ에서 BlogApplication을 실행한 Run 창

생성형 예시 · 실제 실행 안 함

`com.auth0.jwt`로 시작하는 import가 빨간색으로 표시되고 컴파일 오류가 나면 java-jwt 의존성이 아직 반영되지 않은 것입니다. 먼저 1장의 `build.gradle`에 `implementation 'com.auth0:java-jwt:4.5.0'`이 있는지 확인합니다. 그다음 Gradle 창에서 Reload All Gradle Projects를 누르고 다시 실행합니다.

토큰은 7장의 로그인 API가 `JwtUtil.create()`를 호출할 때 처음 발급됩니다. 그때 받은 토큰을 jwt.io에 붙여 넣으면 2절에서 본 헤더와 페이로드를 직접 확인할 수 있습니다.

4장. 공통 응답과 예외 처리 만들기

04

1. 응답 모양을 통일하는 이유

SSR 게시판은 HTML 화면을 돌려주는 서버였습니다. 오류가 나면 에러 페이지 하나를 보여 주면 끝났습니다. REST API는 JSON을 돌려주고, 프런트엔드 코드가 그 JSON을 읽어서 화면을 그립니다. 그런데 상황마다 JSON 모양이 다르면 클라이언트는 응답을 받을 때마다 구조부터 따져 봐야 합니다.

같은 API를 규칙 없이 만든 경우와 응답 모양을 하나로 맞춘 경우는 다음과 같이 다릅니다.

상황	통일하지 않은 응답	통일한 응답
조회 성공	`{"id":1,"username":"ssar"}`	`{"status":200,"msg":"성공","body":{"id":1,"username":"ssar"}}`
삭제 성공	(빈 본문)	`{"status":200,"msg":"성공","body":null}`
오류	`{"timestamp":"...","status":400,"error":"Bad Request","path":"/join"}`	`{"status":400,"msg":"이미 존재하는 유저네임입니다","body":null}`

왼쪽 열의 성공 응답은 데이터만 있거나 본문이 아예 비어 있습니다. 오류 응답은 스프링부트가 기본으로 만든 JSON이라서 필드 이름이 또 다릅니다. 오른쪽 열은 성공과 실패 모두 `status` · `msg` · `body` 세 필드로 이루어집니다. 클라이언트는 먼저 `status` 를 봅니다. 성공이면 `body` 를 꺼내고, 실패면 `msg` 를 꺼내 보여 줍니다. 이 한 가지 규칙으로 모든 응답을 다룰 수 있습니다.

이 장에서 만드는 것은 셋입니다. 이 모양의 응답을 만드는 상자 `Resp` 를 먼저 만듭니다. 그다음 실패 상황을 나타내는 예외 클래스를 만들고, 마지막으로 던진 예외를 한곳에서 받아 `Resp` 모양으로 바꾸는 처리기를 만듭니다.

2. Resp 공통 응답

2.1 record로 만든 응답 상자

`Resp`는 `status` · `msg` · `body` 세 값을 담아 전달하기만 하는 객체입니다. 이런 객체는 클래스보다 **record**로 만드는 편이 간단합니다. record는 Java 16에서 정식 문법이 되었습니다. 선언부에 필드 목록만 적으면 생성자, 접근자, `equals()` · `hashCode()` · `toString()`을 컴파일러가 만들어 줍니다.

기본 문법

Java

```
public record 레코드명(타입 필드명, 타입 필드명) {
}
```

record의 필드는 모두 `final`이므로 객체를 만든 뒤에는 값을 바꿀 수 없습니다. 접근자 이름은 `getStatus()`가 아니고 필드 이름 그대로 `status()`입니다. JSON 변환을 맡는 Jackson이 record 접근자를 알아보기 때문에 `Resp` 객체는 별도 설정 없이 JSON으로 바뀝니다.

`body`에 들어갈 값은 API마다 다릅니다. 회원 정보일 때도 있고 게시글 목록일 때도 있습니다. 그래서 `body`의 타입은 고정하지 않고 제네릭 `T`로 둡니다.

2.2 ok()와 fail()

컨트롤러마다 `new Resp<>(200, "성공", body)`를 직접 쓰면 숫자나 문구를 틀리게 적기 쉽습니다. 그래서 성공 응답은 `ok()`, 실패 응답은 `fail()`이라는 정적 메서드로 만듭니다.

두 메서드는 `Resp`를 `ResponseEntity`에 담아서 돌려줍니다. **ResponseEntity**는 응답 본문과 HTTP 상태 코드, 헤더를 함께 정하는 스프링의 응답 객체입니다. JSON 안의 `status`만 400이고 실제 HTTP 상태 코드가 200이면, HTTP 상태 코드를 보는 클라이언트는 이 응답을 성공으로 처리합니다. `ok()`와 `fail()`은 같은 상태 코드를 `ResponseEntity`와 `Resp` 양쪽에 넣습니다. 그래서 두 값이 어긋날 일이 없습니다.

예제 4-1 – 공통 응답 상자 만들기

이어서: 예제 3-1

파일명: `src/main/java/com/newtenco/blog/core/util/Resp.java`

Java

```java
package com.newtenco.blog.core.util;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

/**
 * 모든 API 응답을 같은 모양으로 맞추는 공통 상자.
 * { "status": 200, "msg": "성공", "body": { ... } }
 */
public record Resp<T>(Integer status, String msg, T body) {

    public static <B> ResponseEntity<Resp<B>> ok(B body) {
        return ResponseEntity.ok(new Resp<>(200, "성공", body));
    }

    public static ResponseEntity<Resp<Void>> fail(HttpStatus status, String msg) {
        return ResponseEntity.status(status).body(new Resp<>(status.value(), msg, null));
    }
}
```

실행 결과

`Resp`는 다른 코드에서 불러야 동작하므로 이 파일만으로는 출력되는 것이 없습니다. 서버가 컴파일 오류 없이 기동되면 성공입니다.

`ok()`는 HTTP 상태를 200으로, `msg`를 `"성공"`으로 정하고 넘겨받은 데이터를 `body`에 넣습니다. `fail()`은 넘겨받은 `HttpStatus`를 HTTP 상태 코드와 `status` 필드에 똑같이 씁니다. 실패 응답에는 돌려줄 데이터가 없으므로 `body`는 `null`이고, 반환 타입의 제네릭도 `Void`입니다.

두 메서드는 부르는 곳이 다릅니다.

메서드	부르는 곳	응답
`Resp.ok(body)`	컨트롤러	HTTP 200 + `{"status":200,"msg":"성공","body":...}`
`Resp.fail(status, msg)`	전역 예외 처리기	HTTP status + `{"status":...,"msg":...,"body":null}`

컨트롤러는 성공했을 때만 `ok()`를 부릅니다. 실패하면 응답을 직접 만들지 않고 예외를 던집니다. 그 예외를 응답으로 바꾸는 일은 4절의 처리기가 맡습니다.

---

3. 상태 코드별 사용자 정의 예외

서비스 코드가 "이미 존재하는 유저네임" 같은 실패 상황을 만나면 그 자리에서 예외를 던집니다. 이때 `RuntimeException`을 그대로 던지면 처리기는 이 예외를 400으로 보낼지 404로 보낼지 정할 근거가 없습니다. 그래서 HTTP 상태 코드마다 예외 클래스를 하나씩 만듭니다. 예외 이름만 보고도 어떤 상태 코드로 응답할지 알 수 있게 하려는 것입니다.

예외	상태 코드	쓰는 상황(예)
`BadRequestException`	400 Bad Request	이미 존재하는 유저네임
`UnauthorizedException`	401 Unauthorized	로그인 실패(비밀번호 불일치)
`ForbiddenException`	403 Forbidden	남의 글 수정
`NotFoundException`	404 Not Found	없는 게시글 조회

401은 요청한 사람이 누구인지 확인되지 않았다는 뜻입니다. 403은 누구인지는 알지만 그 일을 할 권한이 없다는 뜻입니다. 두 코드의 차이는 5장에서 인증과 인가를 배울 때 자세히 다룹니다.

예제 4-2 – 상태 코드별 예외 만들기

이어서: 예제 4-1

파일명: `src/main/java/com/newtenco/blog/core/handler/errors/BadRequestException.java`

Java

```java
package com.newtenco.blog.core.handler.errors;

// 400: 잘못된 요청 (예: 이미 존재하는 유저네임)
public class BadRequestException extends RuntimeException {
    public BadRequestException(String msg) {
        super(msg);
    }
}
```

파일명: `src/main/java/com/newtenco/blog/core/handler/errors/UnauthorizedException.java`

Java

```
package com.newtenco.blog.core.handler.errors;

// 401: 인증 실패 (예: 비밀번호 불일치)
public class UnauthorizedException extends RuntimeException {
    public UnauthorizedException(String msg) {
        super(msg);
    }
}
```

파일명: `src/main/java/com/newtenco/blog/core/handler/errors/ForbiddenException.java`

Java

```
package com.newtenco.blog.core.handler.errors;

// 403: 권한 없음 (예: 남의 글 수정)
public class ForbiddenException extends RuntimeException {
    public ForbiddenException(String msg) {
        super(msg);
    }
}
```

파일명: `src/main/java/com/newtenco/blog/core/handler/errors/NotFoundException.java`

Java

```
package com.newtenco.blog.core.handler.errors;

// 404: 자원 없음 (예: 없는 게시글)
public class NotFoundException extends RuntimeException {
    public NotFoundException(String msg) {
        super(msg);
    }
}
```

실행 결과

아직 네 예외를 던지는 코드가 없으므로 출력은 없습니다. 서버가 컴파일 오류 없이 기동되면 성공입니다.

네 클래스는 이름과 주석만 다릅니다. 생성자로 받은 `msg`를 `super(msg)`로 부모에게 넘기므로, 나중에 `getMessage()`로 같은 문구를 꺼낼 수 있습니다. 이 문구가 응답 JSON의 `msg`가 됩니다.

부모 클래스로 `RuntimeException`을 고른 이유는 두 가지입니다. 첫째, `RuntimeException` 계열은 언체크 예외라서 메서드 선언에 `throws`를 붙이지 않고도 던질 수 있습니다. 서비스에서 던진 예외는 컨트롤러를 지나 처리기까지 올라가는데, 그 사이의 메서드마다 `throws`를 적지 않아도 됩니다. 둘째, 스프링의 `@Transactional`은 기본 설정에서 언체크 예외가 발생했을 때 롤백합니다.

코드 조각

Java

```
if (userRepository.findByUsername(username).isPresent()) {
    throw new BadRequestException("이미 존재하는 유저네임입니다");
}
```

서비스는 실패 조건을 만나면 예외를 던지는 데서 끝납니다. 응답을 어떤 모양으로 만들지는 서비스가 신경 쓰지 않습니다. `findByUsername()`은 7장에서 만드는 `UserRepository`의 메서드입니다.

---

4. 전역 예외 처리기

4.1 @RestControllerAdvice와 @ExceptionHandler

던진 예외를 받아 응답으로 바꾸는 클래스가 **전역 예외 처리기**입니다. 어노테이션 두 개로 만듭니다.

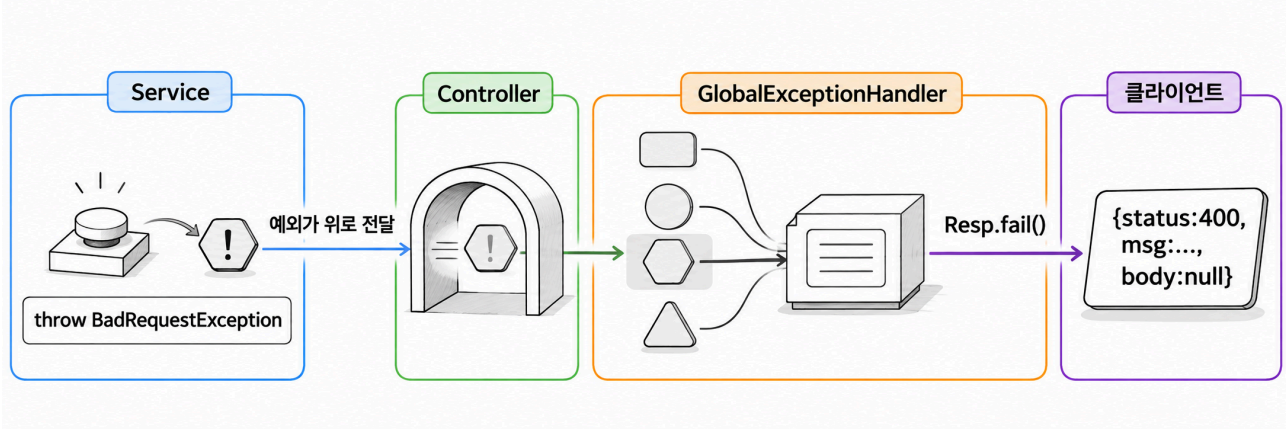
`@ExceptionHandler(예외클래스.class)`는 메서드에 붙입니다. 괄호 안의 예외가 발생하면 이 메서드가 실행되고, 메서드의 반환값이 그대로 응답이 됩니다. `@RestControllerAdvice`는 클래스에 붙입니다. 이 어노테이션이 있으면 클래스 안의 `@ExceptionHandler`가 컨트롤러 하나가 아니라 모든 컨트롤러에 적용됩니다. 이름에 `Rest`가 들어 있는 만큼 반환값을 뷰 이름이 아닌 응답 본문(JSON)으로 씁니다.

기본 문법

Java

```
@RestControllerAdvice
public class 처리기클래스명 {

    @ExceptionHandler(예외클래스.class)
    public ResponseEntity<?> 메서드명(예외클래스 e) {
        return 응답;
    }
}
```

예외가 처리기까지 가는 길은 그림 4-1과 같습니다.



던진 예외가 Resp 응답이 되기까지

컨트롤러와 서비스는 예외를 `try-catch`로 잡지 않습니다. 예외는 호출한 쪽으로 계속 올라가 스프링 MVC에 닿습니다. 스프링 MVC는 예외 타입에 맞는 `@ExceptionHandler` 메서드를 골라 실행하고, 그 메서드가 `Resp.fail()`로 응답을 만듭니다. 실패 응답을 만드는 코드가 처리기 클래스 하나에 모이므로, 나중에 응답 모양을 바꿀 때도 이 클래스만 고치면 됩니다.

3절에서 만든 예외 네 개를 받는 처리기부터 만듭니다.

예제 4-3 – 상태 코드별 예외 처리기 만들기

이어서: 예제 4-2

파일명: `src/main/java/com/newtenco/blog/core/handler/GlobalExceptionHandler.java`

Java

```java
package com.newtenco.blog.core.handler;

import com.newtenco.blog.core.handler.errors.BadRequestException;
import com.newtenco.blog.core.handler.errors.ForbiddenException;
import com.newtenco.blog.core.handler.errors.NotFoundException;
import com.newtenco.blog.core.handler.errors.UnauthorizedException;
import com.newtenco.blog.core.util.Resp;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/**
 * 컨트롤러/서비스에서 던진 예외를 한곳에서 잡아 Resp 모양으로 응답한다.
 */
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<?> ex400(BadRequestException e) {
        log.warn("[400] {}", e.getMessage());
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getMessage());
    }

    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<?> ex401(UnauthorizedException e) {
        log.warn("[401] {}", e.getMessage());
        return Resp.fail(HttpStatus.UNAUTHORIZED, e.getMessage());
    }

    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<?> ex403(ForbiddenException e) {
        log.warn("[403] {}", e.getMessage());
        return Resp.fail(HttpStatus.FORBIDDEN, e.getMessage());
    }

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<?> ex404(NotFoundException e) {
        log.warn("[404] {}", e.getMessage());
        return Resp.fail(HttpStatus.NOT_FOUND, e.getMessage());
    }
}
```

실행 결과

예외를 던지는 컨트롤러가 아직 없으므로 서버가 정상 기동되는지만 확인합니다.

네 메서드는 받는 예외 클래스와 `HttpStatus` 값만 다릅니다. 서비스가 `new BadRequestException("이미 존재하는 유저네임입니다")`를 던졌다면 `ex400()`이 실행됩니다. 이때 HTTP 400과 함께 `{"status":400,"msg":"이미 존재하는 유저네임입니다","body":null}`이 응답으로 나갑니다.

`@Slf4j`는 Lombok이 클래스에 `log` 필드를 만들어 주는 어노테이션입니다. 클라이언트가 잘못된 값을 보내서 생긴 실패는 서버 고장이 아니므로 `log.warn()`으로 남깁니다. 실습 중 로그에 `[400]`이 보이면 서버 코드보다 요청 내용을 먼저 확인합니다.

4.2 @Valid 검증 실패와 쿼리스트링 누락 처리

`@Valid`는 SSR 게시판에서 이미 써 본 검증 어노테이션입니다. 요청 DTO 필드에 `@NotBlank` · `@Size` 같은 조건을 달고 컨트롤러 파라미터에 `@Valid`를 붙이면, 스프링이 컨트롤러 메서드를 실행하기 전에 값을 검사합니다.

SSR에서는 `Errors`(또는 `BindingResult`)를 파라미터로 함께 받고 `if (errors.hasErrors())`로 실패를 직접 확인했습니다. `Errors` 파라미터가 없으면 스프링은 검증 실패 시 컨트롤러 메서드를 실행하지 않습니다. 대신 `MethodArgumentNotValidException`을 던집니다. 이 예외를 전역 예외 처리기에서 받으면 컨트롤러마다 같은 `if` 문을 반복할 필요가 없어집니다.

`@RequestBody`로 JSON을 받는 요청은 다음 순서로 검증됩니다.

1. 클라이언트가 JSON을 보냅니다.
2. 스프링이 JSON을 요청 DTO 객체로 바꿉니다.
3. `@Valid`가 DTO 필드의 `@NotBlank` · `@Size` · `@Email` 조건을 검사합니다.
4. 조건이 하나라도 어긋나면 컨트롤러 메서드를 실행하지 않고 `MethodArgumentNotValidException`을 던집니다.
5. 전역 예외 처리기가 이 예외를 받아 첫 번째 오류를 `"필드명:메시지"` 모양으로 응답합니다.

코드 조각

Java

```
// @Valid 검증 실패 -> 400
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<?> validation(MethodArgumentNotValidException e) {
    FieldError fe = e.getBindingResult().getFieldErrors().getFirst();
    String msg = fe.getField() + ":" + fe.getDefaultMessage();
    log.warn("[400] 유효성 검사 실패 - {}", msg);
    return Resp.fail(HttpStatus.BAD_REQUEST, msg);
}
```

`getBindingResult()`로 꺼낸 검증 결과에는 실패한 필드가 여러 개 있을 수 있습니다. 응답의 `msg`는 문자열 하나뿐이므로 `getFirst()`로 첫 번째 오류만 씁니다. `getFirst()`는 Java 21에서 `List`에 추가된 메서드입니다. `getField()`는 실패한 필드 이름을, `getDefaultMessage()`는 어노테이션의 `message` 속성에 적은 문구를 돌려줍니다. 7장의 회원가입 DTO에 `username`을 두 글자로 보내면 `msg`는 `"username:유저네임은 4~20자여야 합니다"`가 됩니다. 클라이언트는 콜론 앞의 필드 이름을 보고 어느 입력칸에 오류를 표시할지 정합니다.

요청 값이 잘못되는 경우는 하나 더 있습니다. `?username=ssar`처럼 쿼리스트링으로 받아야 하는 값이 빠지면 스프링은 `MissingServletRequestParameterException`을 던집니다. 이것도 클라이언트가 값을 빠뜨린 것이므로 400으로 응답합니다.

코드 조각

Java

```
// 필수 쿼리스트링(?username=) 누락 -> 400
@ExceptionHandler(MissingServletRequestParameterException.class)
public ResponseEntity<?> missingParam(MissingServletRequestParameterException e) {
    return Resp.fail(HttpStatus.BAD_REQUEST, e.getParameterName() + ":값을 입력해주세요");
}
```

`getParameterName()`은 빠진 파라미터의 이름을 돌려줍니다. 덕분에 이 응답도 `@Valid` 실패와 같은 `"필드명:메시지"` 모양이 됩니다. 두 메서드는 다음 항의 예제 4-4에서 처리기에 넣습니다.

4.3 예상하지 못한 예외와 500 응답

지금까지 만든 처리기는 미리 정해 둔 예외만 받습니다. `NullPointerException`이나 DB 오류처럼 예상하지 못한 예외가 나면 스프링부트의 기본 오류 응답이 나가서 응답 모양이 다시 흐트러집니다. 그래서 마지막 안전망으로 `Exception.class`를 받는 메서드를 둡니다.

모든 예외가 `Exception`의 자식이니 `BadRequestException`도 이 메서드로 가지 않을까 걱정될 수 있습니다. 스프링은 발생한 예외와 상속 관계가 가장 가까운 `@ExceptionHandler`를 고릅니다. 그래서 `BadRequestException`은 그대로 `ex400()`으로 가고, 맞는 메서드가 따로 없는 예외만 `Exception.class` 메서드에 도착합니다.

이 메서드는 응답을 만들 때 두 가지를 지킵니다.

첫째, 내부 오류 메시지를 클라이언트에 보내지 않습니다. 예외 메시지에는 테이블 이름이나 SQL 같은 서버 내부 정보가 섞여 있을 수 있습니다. 클라이언트에는 `"관리자에게 문의하세요"`만 보내고, 원인은 `log.error()`로 스택 트레이스까지 서버 로그에 남깁니다.

둘째, 스프링 MVC가 원래 상태 코드를 정해 둔 예외는 500으로 바꾸지 않습니다. 없는 주소를 요청하면 404(`NoResourceFoundException`)가, 허용하지 않는 HTTP 메서드로 요청하면 405가 나가야 맞습니다. 이런 예외는 스프링의 `ErrorResponse` 인터페이스를 구현하므로 `getStatusCode()`로 원래 상태 코드를 꺼낼 수 있습니다.

예제 4-4 – 검증 실패와 예상하지 못한 예외 처리 추가하기

이어서: 예제 4-3

파일명: `src/main/java/com/newtenco/blog/core/handler/GlobalExceptionHandler.java`

Java

```java
package com.newtenco.blog.core.handler;

import com.newtenco.blog.core.handler.errors.BadRequestException;
import com.newtenco.blog.core.handler.errors.ForbiddenException;
import com.newtenco.blog.core.handler.errors.NotFoundException;
import com.newtenco.blog.core.handler.errors.UnauthorizedException;
import com.newtenco.blog.core.util.Resp;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.ErrorResponse;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.MissingServletRequestParameterException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/**
 * 컨트롤러/서비스에서 던진 예외를 한곳에서 잡아 Resp 모양으로 응답한다.
 */
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    // @Valid 검증 실패 -> 400
    // 컨트롤러마다 if (errors.hasErrors()) 를 쓰지 않아도 여기서 한 번에 처리된다.
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<?> validation(MethodArgumentNotValidException e) {
        FieldError fe = e.getBindingResult().getFieldErrors().getFirst();
        String msg = fe.getField() + ":" + fe.getDefaultMessage();
        log.warn("[400] 유효성 검사 실패 - {}", msg);
        return Resp.fail(HttpStatus.BAD_REQUEST, msg);
    }

    // 필수 쿼리스트링(?username=) 누락 -> 400
    @ExceptionHandler(MissingServletRequestParameterException.class)
    public ResponseEntity<?> missingParam(MissingServletRequestParameterException e) {
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getParameterName() + ":값을 입력해주세요");
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<?> ex400(BadRequestException e) {
        log.warn("[400] {}", e.getMessage());
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getMessage());
    }
```

```
    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<?> ex401(UnauthorizedException e) {
        log.warn("[401] {}", e.getMessage());
        return Resp.fail(HttpStatus.UNAUTHORIZED, e.getMessage());
    }

    @ExceptionHandler(ForbiddenException.class)
    public ResponseEntity<?> ex403(ForbiddenException e) {
        log.warn("[403] {}", e.getMessage());
        return Resp.fail(HttpStatus.FORBIDDEN, e.getMessage());
    }

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<?> ex404(NotFoundException e) {
        log.warn("[404] {}", e.getMessage());
        return Resp.fail(HttpStatus.NOT_FOUND, e.getMessage());
    }

    // 예상하지 못한 모든 예외 -> 500 (내부 메시지는 숨긴다)
    @ExceptionHandler(Exception.class)
    public ResponseEntity<?> exUnknown(Exception e) {
        // 없는 주소(404), 허용하지 않는 메서드(405) 같은 스프링 MVC 예외는 원래 상태 코드를 유지한다
        if (e instanceof ErrorResponse er) {
            HttpStatus status = HttpStatus.valueOf(er.getStatusCode().value());
            return Resp.fail(status, status.getReasonPhrase());
        }
        log.error("[500] 예상하지 못한 오류", e);
        return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, "관리자에게 문의하세요");
    }
}
```

실행 결과

서버가 정상 기동되면 성공입니다. 실제 응답은 컨트롤러와 시큐리티 설정을 만든 뒤에 확인합니다 (5절 참고).

`e instanceof ErrorResponse er`는 Java 16부터 쓸 수 있는 패턴 매칭입니다. 타입 검사를 통과하면 형변환된 값이 `er` 변수에 바로 들어갑니다. `getReasonPhrase()`는 상태 코드의 표준 문구(`"Not Found"`, `"Method Not Allowed"`)를 돌려줍니다.

완성된 처리기는 예외를 다음과 같이 응답으로 바꿉니다.

발생 상황	예외	응답
`@Valid` 검증 실패	`MethodArgumentNotValidException`	400, `"필드명:메시지"`
필수 쿼리스트링 누락	`MissingServletRequestParameterException`	400, `"필드명:값을 입력해주세요"`
이미 존재하는 유저네임	`BadRequestException`	400, 던질 때 넣은 메시지
비밀번호 불일치	`UnauthorizedException`	401, 던질 때 넣은 메시지
남의 글 수정	`ForbiddenException`	403, 던질 때 넣은 메시지
없는 자원	`NotFoundException`	404, 던질 때 넣은 메시지
없는 주소 요청	`NoResourceFoundException` (`ErrorResponse` 구현)	404, `"Not Found"`
그 밖의 예외	`Exception`	500, `"관리자에게 문의하세요"`

---

5. 지금 확인할 수 있는 것

5.1 서버 정상 실행 확인과 필터 단계 예외의 한계

IntelliJ에서 `BlogApplication` 을 실행합니다. 이 장에서 추가한 파일은 여섯 개입니다.

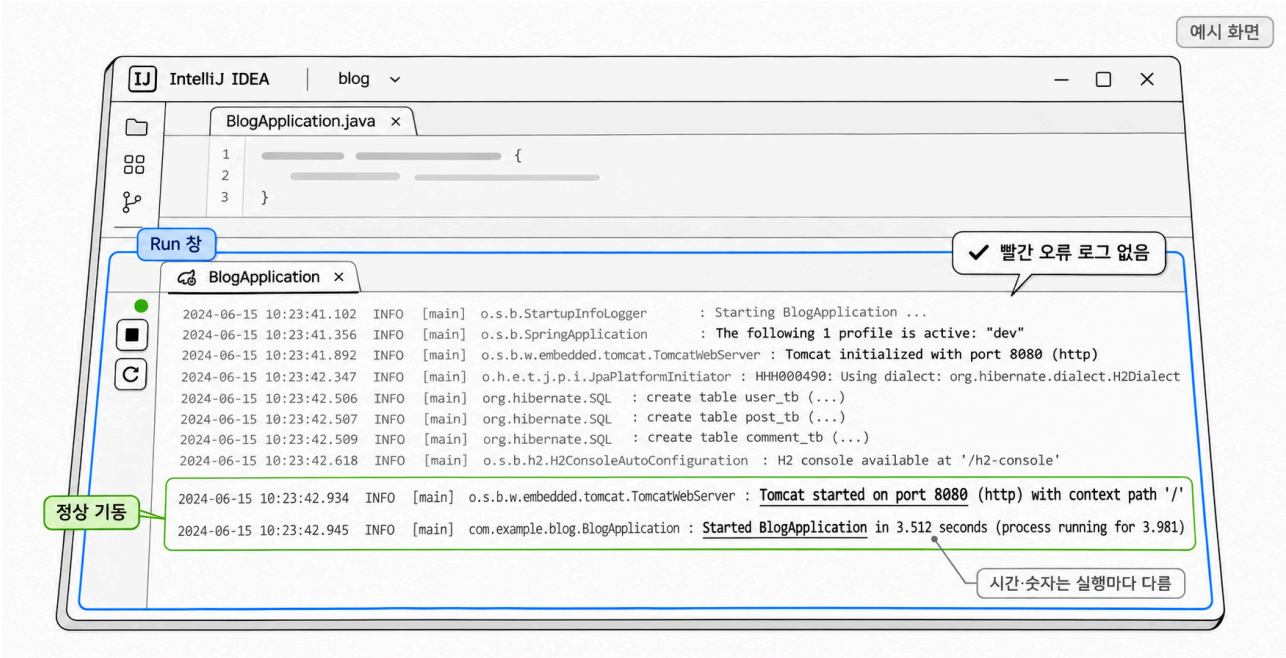
패키지	파일
`core.util`	`Resp.java`
`core.handler.errors`	`BadRequestException.java` · `UnauthorizedException.java` · `ForbiddenException.java` · `NotFoundException.java`
`core.handler`	`GlobalExceptionHandler.java`

실행 결과

콘솔 끝부분에 다음과 같은 로그가 나오면 정상입니다. 시간과 숫자는 실행할 때마다 달라집니다.

출력

```
Tomcat started on port 8080 (http) with context path '/'
Started BlogApplication in 3.512 seconds (process running for 3.981)
```



IntelliJ에서 BlogApplication을 실행한 Run 창

생성형 예시 · 실제 실행 안 함

지금 Talend API Tester로 요청을 보내도 `Resp` 모양의 응답은 볼 수 없습니다. 예외를 던질 컨트롤러가 아직 없기 때문입니다. 게다가 1장에서 본 것처럼 시큐리티 기본 설정이 모든 요청을 로그인 화면으로 보냅니다.

전역 예외 처리기에는 알아 둘 한계가 하나 있습니다. `@RestControllerAdvice`가 받는 예외는 컨트롤러와 그 아래(서비스, 리포지토리)에서 던진 것뿐입니다. 시큐리티 필터는 요청이 컨트롤러에 닿기 전에 실행됩니다. 그래서 필터에서 인증에 실패하면 이 처리기를 거치지 않습니다. 필터 단계에서 쓸 실패 응답은 6장에서 따로 만듭니다. `Resp` 모양의 JSON은 6장에서 시큐리티 설정을 바꾼 뒤에 처음 확인할 수 있습니다. 필터 단계의 `{"status":401,...}` 응답은 6장 끝에서 토큰 없이 `/api/users/me`를 요청해 확인합니다.

5장. 스프링 시큐리티와 JWT 필터 만들기 05

1. 스프링 시큐리티의 동작 원리

1.1 인증과 인가(401과 403)

스프링 시큐리티(Spring Security)는 요청이 컨트롤러에 도착하기 전에 여러 필터를 거치게 해서 보안 검사를 맡는 프레임워크입니다. 검사는 요청의 입구에서 한 번만 합니다. 그래서 SSR 게시판처럼 컨트롤러 메서드마다 로그인 여부를 확인하는 코드를 넣을 필요가 없습니다.

보안 검사는 두 가지입니다. **인증(Authentication)**은 요청을 보낸 사람이 누구인지 확인합니다. **인가(Authorization)**는 확인된 사람에게 이 기능을 쓸 권한이 있는지 판단합니다. 누구인지 알아야 권한을 따질 수 있으므로 인가는 항상 인증 뒤에 옵니다.

구분	확인하는 것	실패할 때 상태 코드	이 프로젝트의 예
인증	누구인가?	401 Unauthorized	토큰 없이 `/api/users/me` 요청
인가	이 기능을 써도 되는가?	403 Forbidden	USER 토큰으로 `/api/admin/hello` 요청

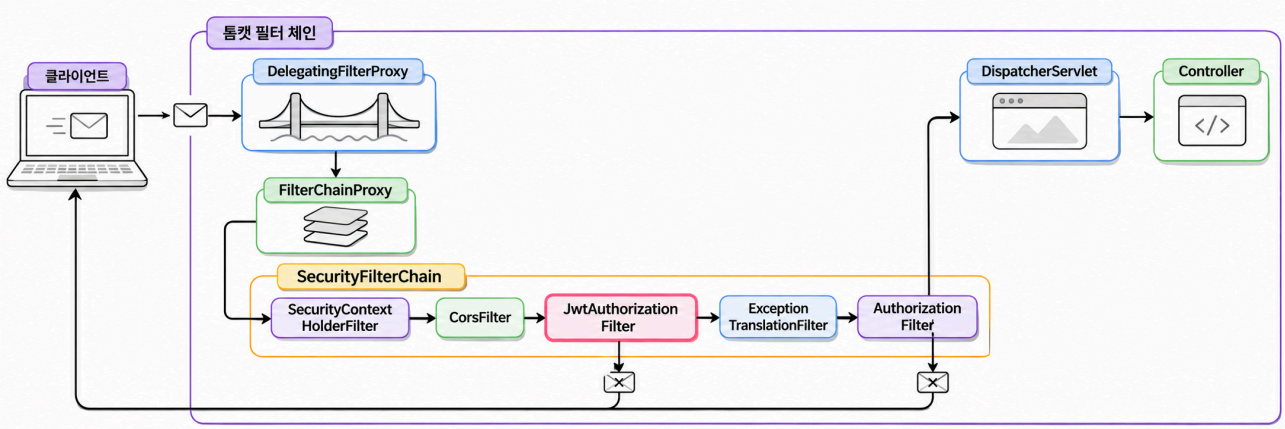
401의 이름은 Unauthorized지만 실제 뜻은 "인증되지 않음"입니다. 이름 때문에 403과 헷갈린다면 기준 하나만 기억합니다. 서버가 요청한 사람을 모르면 401, 누구인지는 알지만 허락하지 않으면 403입니다.

1.2 필터 체인과 실행 순서

필터(Filter)는 요청이 서블릿에 닿기 전에 요청을 검사하거나 바꾸는 객체입니다. 스프링 MVC에서 이 서블릿은 `DispatcherServlet`입니다. 필터는 여러 개가 한 줄로 이어지는데, 이 줄을 **필터 체인(filter chain)**이라고 합니다. 각 필터는 자기 일을 마치면 다음 필터에 요청을 넘기고, 넘기지 않으면 요청은 그 자리에서 끝납니다.

톰캣의 필터 체인에 스프링 시큐리티가 등록하는 필터는 `DelegatingFilterProxy` 하나뿐입니다. 톰캣은 스프링 빈을 모르기 때문에 이 필터가 톰캣과 스프링 컨테이너 사이를 잇습니다.

`DelegatingFilterProxy`는 요청을 스프링 빈인 `FilterChainProxy`에 넘깁니다. `FilterChainProxy`는 개발자가 정의한 **SecurityFilterChain** 안의 보안 필터들을 순서대로 실행합니다. 6장에서 만들 `SecurityConfig`가 이 `SecurityFilterChain`을 정의합니다.



요청이 컨트롤러에 닿기까지

그림 5-1에서 보듯 보안 필터는 모두 `DispatcherServlet` 앞에 있습니다. 그래서 인증이나 인가에 실패한 요청은 컨트롤러 코드를 한 줄도 실행하지 않고 돌아갑니다.

`SecurityFilterChain` 안의 필터는 순서가 정해져 있습니다. 실제로는 필터가 더 많지만, 이 프로젝트에서 알아야 할 필터만 순서대로 추리면 다음과 같습니다.

순서	필터	하는 일
1	SecurityContextHolderFilter	이번 요청의 인증 정보 보관소를 준비하고, 요청이 끝나면 비운다
2	HeaderWriterFilter	보안 관련 응답 헤더(X-Frame-Options 등)를 붙인다
3	CorsFilter	CORS 요청을 검사한다
4	CsrfFilter	CSRF 토큰을 검사한다(이 프로젝트는 끈다)
5	LogoutFilter	로그아웃 요청을 처리한다
6	**JwtAuthorizationFilter**	직접 만드는 필터. 토큰을 검증해 인증 정보를 넣는다
7	UsernamePasswordAuthenticationFilter	폼 로그인을 처리한다(이 프로젝트는 끄고, 끼워 넣을 위치의 기준으로만 쓴다)

순서	필터	하는 일
8	AnonymousAuthenticationFilter	인증 정보가 비어 있으면 익명 사용자로 채운다
9	ExceptionTranslationFilter	뒤에서 발생한 인증 · 인가 예외를 401 · 403 응답으로 바꾼다
10	AuthorizationFilter	URL 규칙에 따라 통과와 거부를 최종 결정한다

이 표에서 두 가지를 기억해 둡니다. 첫째, `JwtAuthorizationFilter`는 반드시 `AuthorizationFilter`보다 앞에 와야 합니다. 토큰으로 사용자를 먼저 표시해 둬야 마지막 필터가 들여보낼지 판단할 수 있습니다. 6장에서 이 필터를 `UsernamePasswordAuthenticationFilter` 앞에 넣는 것도 이 때문입니다. 둘째, 401 · 403 응답은 `ExceptionTranslationFilter`가 만듭니다. 바로 뒤에 있는 `AuthorizationFilter`가 거부 예외를 던지면 이 필터가 받아서 응답으로 바꿉니다.

1.3 Filter와 AOP 비교

필터와 AOP는 여러 곳에 흩어질 공통 처리를 한곳에 모은다는 점은 같습니다. 차이는 실행 위치와 관리 주체에 있습니다.

구분	Filter	AOP
실행 위치	`DispatcherServlet` 앞	스프링 빈(컨트롤러 · 서비스) 메서드의 앞뒤
관리 주체	서블릿 컨테이너(톰캣)	스프링 컨테이너
다루는 대상	HTTP 요청 · 응답 전체	메서드 호출, 인자, 반환값
주 용도	인증, CORS, 인코딩, 요청 로깅	트랜잭션, 메서드 단위 로깅
이 책의 예	`JwtAuthorizationFilter`	`@Transactional`

인증을 필터에 두는 이유는 실행 위치 때문입니다. 필터는 요청이 스프링 MVC에 들어가기 전에 실행되므로 컨트롤러뿐 아니라 정적 파일이나 H2 콘솔 같은 경로도 같은 규칙으로 막습니다. AOP는 스프링 빈의 메서드가 호출될 때만 끼어듭니다. 이미 컨트롤러까지 들어온 요청만 다룰 수 있어서 입구 검사에는 맞지 않습니다.

---

2. JwtProvider 만들기

3장의 `JwtUtil`은 HTTP도 시큐리티도 모르는 순수 자바 클래스입니다. 토큰 문자열을 받아 `User`를 돌려주는 일만 합니다. 시큐리티 필터가 이 클래스를 쓰려면 두 가지 일이 더 필요합니다. HTTP 요청에서 토큰을 꺼내는 일, 그리고 복원한 사용자를 시큐리티가 읽는 객체로 바꾸는 일입니다. **JwtProvider**는 이 두 일을 맡아 HTTP 요청, 시큐리티, `JwtUtil`을 연결합니다.

메서드	받는 것	돌려주는 것
`resolveToken()`	HTTP 요청	"Bearer "를 뗀 토큰 문자열(없으면 `null`)
`getAuthentication()`	토큰 문자열	인증 객체 `Authentication` (검증 실패면 `null`)

2.1 Authentication 객체

Authentication은 이번 요청을 보낸 사용자가 누구이고 어떤 권한이 있는지 담는 스프링 시큐리티의 인터페이스입니다. 시큐리티의 필터와 접근 규칙은 개발자가 만든 `User` 클래스를 직접 읽지 않습니다. 이 `Authentication` 객체만 읽습니다.

구성 요소	담는 것	이 프로젝트에서 넣는 값
principal	사용자 자신	토큰에서 복원한 `User` 객체
credentials	비밀번호 같은 증명 수단	`null` (토큰 검증으로 이미 증명함)
authorities	권한 목록	`user.getAuthorities()` (ROLE_USER, ROLE_ADMIN 등)

인터페이스이므로 실제 객체는 구현 클래스로 만듭니다. 아이디 · 비밀번호 로그인에서 쓰는 `UsernamePasswordAuthenticationToken`을 그대로 씁니다. 이름에 Password가 들어 있지만 사용자와 권한을 담는 클래스이므로 JWT 방식에서도 문제없이 쓸 수 있습니다.

기본 문법

Java

```
Authentication 변수명 = new UsernamePasswordAuthenticationToken(사용자객체, 증명수단, 권한목록);
```

세 인자 생성자는 인증이 끝난 객체를 만듭니다. 권한 목록이 없는 두 인자 생성자는 아직 인증되지 않은 객체를 만들기 때문에 접근 규칙이 인증된 사용자로 인정하지 않습니다. 이 객체는 토큰 검증을 통과한 다음에 만들므로 세 인자 생성자를 씁니다.

2장에서 `User`가 `UserDetails`를 구현하고 `getAuthorities()`가 "ROLE_" 접두사를 붙이게 만든 것이 이 자리에서 쓰입니다. `User`를 그대로 principal에, 그 권한 목록을 그대로 authorities에 넣으면 됩니다.

2.2 헤더에서 토큰 꺼내기 resolveToken()

클라이언트는 토큰을 `Authorization` 헤더에 담아 `Bearer eyJ...` 모양으로 보냅니다. `resolveToken()`은 이 헤더를 읽고 "Bearer "로 시작할 때만 접두사 뒤의 토큰 문자열을 잘라 돌려줍니다. 헤더가 없거나 Basic 같은 다른 방식이면 `null`을 돌려주고, 필터는 이를 토큰이 없는 요청으로 처리합니다.

헤더 이름과 접두사는 3장 `JwtUtil`의 상수 `HEADER`("Authorization")와 `TOKEN_PREFIX`("Bearer ")를 씁니다. 토큰을 만드는 `create()`와 꺼내는 `resolveToken()`이 같은 상수를 써야 접두사 끝의 공백 하나가 어긋나는 실수가 생기지 않습니다.

2.3 인증 객체 만들기 getAuthentication()

`getAuthentication()`은 토큰을 `JwtUtil.verify()`로 검증하고, 통과하면 복원한 `User`를 `Authentication`에 담아 돌려줍니다. 형식이 틀리거나 서명이 맞지 않거나 만료된 토큰이면 `verify()`가 `JWTVerificationException`을 던집니다. `getAuthentication()`은 이 예외를 잡아 `null`을 돌려줍니다. 예외를 밖으로 던지지 않는 이유는 3.3에서 설명합니다.

예제 5-1 — JwtProvider 만들기

이어서: 예제 4-4

파일명: `src/main/java/com/newtenco/blog/core/util/JwtProvider.java`

Java

```java
package com.newtenco.blog.core.util;

import com.auth0.jwt.exceptions.JWTVerificationException;
import com.newtenco.blog.user.User;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;

/**
 * HTTP 요청 · 시큐리티 · JwtUtil 을 연결한다.
 * 요청 헤더에서 토큰을 꺼내고, 검증한 뒤, 시큐리티가 읽는 인증 객체로 바꾼다.
 */
@Component
public class JwtProvider {

    // "Authorization: Bearer eyJ..." 에서 "Bearer " 뒤의 토큰만 꺼낸다
    public String resolveToken(HttpServletRequest request) {
        String bearerToken = request.getHeader(JwtUtil.HEADER);
        if (bearerToken != null && bearerToken.startsWith(JwtUtil.TOKEN_PREFIX)) {
            return bearerToken.substring(JwtUtil.TOKEN_PREFIX.length());
        }
        return null;
    }

    // 검증에 성공하면 Authentication, 형식 오류 · 위조 · 만료면 null
    public Authentication getAuthentication(String token) {
        try {
            User user = JwtUtil.verify(token);
            return new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
        } catch (JWTVerificationException e) {
            return null;
        }
    }
}
```

`@Component`를 붙였으므로 `JwtProvider`는 스프링 빈이 되고, 다음 절의 필터가 이 빈을 받아 씁니다. 여기서 principal로 들어가는 `User`는 DB에서 조회한 객체가 아닙니다. 토큰 안의 id · username · roles로만 복원했기 때문에 비밀번호가 비어 있습니다. 이 차이는 8장에서 내 정보를 조회할 때 다시 다룹니다.

---

3. JwtAuthorizationFilter 만들기

3.1 OncePerRequestFilter

OncePerRequestFilter는 한 요청에서 필터가 한 번만 실행되도록 보장하는 스프링의 추상 클래스입니다. 서블릿에서는 요청 하나가 내부에서 다른 경로로 전달(forward)되거나 오류 처리 경로로 다시 들어갈 수 있습니다. 일반 필터는 이때마다 다시 실행됩니다. 같은 요청의 토큰을 두 번 검증할 이유가 없으므로 JWT 필터는 이 클래스를 상속합니다.

기본 문법

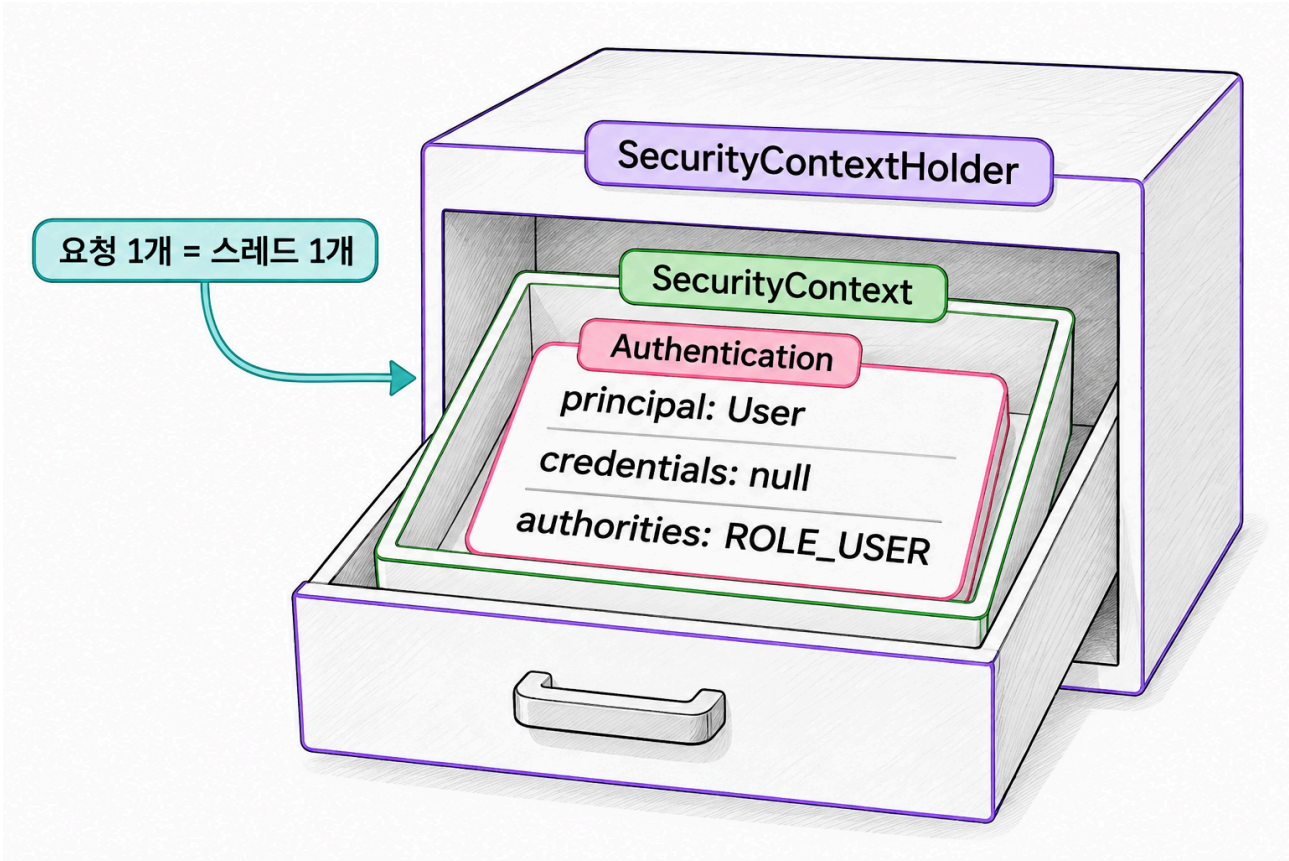
Java

```
public class 필터이름 extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain)
            throws ServletException, IOException {
        // 요청 검사
        filterChain.doFilter(request, response); // 다음 필터로 넘긴다
    }
}
```

검사 코드는 `doFilterInternal()` 안에 씁니다. 마지막 줄의 `filterChain.doFilter()`가 요청을 다음 필터로 넘깁니다. 이 줄을 빠뜨리면 요청이 필터에서 멈춰 컨트롤러에 가지 못하고, 클라이언트는 빈 응답을 받습니다.

3.2 SecurityContextHolder에 인증 정보 넣기

SecurityContextHolder는 현재 요청의 인증 정보를 보관하는 곳입니다. 안에 `SecurityContext`가 있고, 그 안에 2절의 `Authentication`이 들어갑니다. 기본 설정에서는 스레드마다 값을 따로 두는 `ThreadLocal`에 보관합니다. 요청 하나는 스레드 하나가 처리하므로 동시에 들어온 다른 사용자의 인증 정보와 섞이지 않습니다.



SecurityContextHolder 안의 인증 정보

그림 5-2의 구조에 인증 정보를 넣을 때는 세 단계를 거칩니다. 빈 `SecurityContext`를 새로 만들고, 그 안에 `Authentication`을 넣고, 이 컨텍스트를 `SecurityContextHolder`에 등록합니다. `SecurityContextHolder.getContext().setAuthentication()`으로 기존 컨텍스트를 고쳐도 동작은 합니다. 그래도 스프링 시큐리티 공식 문서는 새 컨텍스트를 만드는 방식을 권합니다. 같은 컨텍스트를 여러 스레드가 함께 고치면서 생기는 경쟁 상태를 피할 수 있기 때문입니다.

넣은 인증 정보는 이번 요청이 끝날 때까지만 유지됩니다. 요청이 끝나면 `SecurityContextHolderFilter`가 보관소를 비웁니다. 세션에도 저장하지 않으므로 다음 요청이 오면 토큰을 다시 검증해서 다시 넣습니다. 3장에서 본 Stateless(무상태)가 코드에서는 이렇게 나타납니다.

예제 5-2 — JwtAuthorizationFilter 만들기

이어서: 예제 5-1

파일명: `src/main/java/com/newtenco/blog/core/filter/JwtAuthorizationFilter.java`

Java

```java
package com.newtenco.blog.core.filter;

import com.newtenco.blog.core.util.JwtProvider;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

/**
 * 요청마다 한 번 실행되는 JWT 필터.
 * 토큰이 유효하면 이번 요청 동안만 SecurityContextHolder 에 인증 정보를 넣는다.
 * 토큰이 없거나 틀려도 여기서 막지 않는다. 막을지는 SecurityConfig 의 URL 규칙이 정한다.
 */
@RequiredArgsConstructor
public class JwtAuthorizationFilter extends OncePerRequestFilter {

    private final JwtProvider jwtProvider;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain)
            throws ServletException, IOException {

        String token = jwtProvider.resolveToken(request);

        if (token != null) {
            Authentication authentication = jwtProvider.getAuthentication(token);
            if (authentication != null) {
                SecurityContext context = SecurityContextHolder.createEmptyContext();
                context.setAuthentication(authentication);
                SecurityContextHolder.setContext(context);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

토큰이 있고 검증까지 통과했을 때만 인증 정보를 넣습니다. 어느 경우든 마지막 줄에서 다음 필터로 요청을 넘깁니다.

`JwtProvider`와 달리 이 필터에는 `@Component`를 붙이지 않았습니다. 스프링부트는 빈으로 등록된 필터를 톰캣 필터 체인에도 자동으로 등록합니다. `@Component`를 붙이면 이 필터가 시큐리티 필터 체인 밖에서 한 번 더 실행됩니다. 6장의 `SecurityConfig`에서 `new JwtAuthorizationFilter(jwtProvider)`로 직접 만들어 시큐리티 필터 체인의 정해진 자리에만 넣습니다.

3.3 토큰이 없거나 만료 · 위조되었을 때

`JwtAuthorizationFilter`는 토큰이 없거나 틀려도 요청을 막지 않습니다. 인증 정보를 비워 둔 채 다음 필터로 넘깁니다. 이 필터가 맡는 일은 요청한 사람이 누구인지 표시하는 데서 끝납니다.

이렇게 나눈 이유는 공개 주소에 있습니다. `/join`, `/login`은 토큰이 없는 상태에서 요청하는 주소입니다. 필터가 토큰 없는 요청을 모두 막으면 회원가입부터 할 수 없습니다. 위조 토큰에 필터가 바로 예외를 던져도 같은 문제가 생깁니다. 브라우저에 만료된 토큰이 남아 있는 사용자는 공개 주소조차 쓰지 못합니다. 2.3에서 `getAuthentication()`이 예외 대신 `null`을 돌려준 것도 같은 이유입니다.

요청을 막을지는 뒤쪽 필터가 결정합니다. 1.2의 필터 순서를 따라가면 토큰 상태별 결과는 다음과 같습니다.

토큰 상태	JwtAuthorizationFilter	뒤쪽 필터	결과
없음	아무것도 넣지 않고 넘긴다	`AnonymousAuthenticationFilter`가 익명 사용자로 채운다	공개 주소는 통과, 보호 주소는 401
만료 · 위조	`getAuthentication()`이 `null`이라 아무것도 넣지 않는다	토큰이 없을 때와 같다	공개 주소는 통과, 보호 주소는 401
유효	`SecurityContextHolder`에 `Authentication`을 넣는다	`AuthorizationFilter`가 역할 규칙과 비교한다	권한이 맞으면 통과, 아니면 403

익명 사용자가 보호 주소를 요청하면 `AuthorizationFilter`가 접근 거부 예외를 던집니다. `ExceptionTranslationFilter`는 이 예외를 받고 현재 사용자를 확인합니다. 익명 사용자이면 누구인지 모르는 상태이므로 인증 실패(401)로, 인증된 사용자이면 권한 부족(403)으로 처리합니다. 1.1에서 본 401과 403의 구분이 이 필터에서 이루어집니다. 401 · 403 응답을 4장의 공통 응답 JSON 모양으로 만드는 설정은 6장에서 합니다.

4. CORS와 CSRF

4.1 출처와 CORS

출처(Origin)는 프로토콜, 도메인, 포트를 합친 값입니다. 셋 중 하나만 달라도 다른 출처입니다.

화면을 연 주소	요청을 보낸 주소	같은 출처인가
http://localhost:8080	http://localhost:8080/login	같음
http://localhost:5173	http://localhost:8080/login	다름(포트)
http://localhost:5173	https://localhost:5173/login	다름(프로토콜)

브라우저는 화면과 다른 출처로 보낸 요청의 응답을 자바스크립트가 읽지 못하게 막습니다. 이 규칙이 **동일 출처 정책(Same-Origin Policy)**입니다. **CORS(Cross-Origin Resource Sharing)**는 서버가 특정 출처에 응답을 공개해도 된다고 허락하는 방법입니다. 서버가 응답에 `Access-Control-Allow-Origin` 헤더를 붙이면, 브라우저는 그 헤더에 적힌 출처의 코드에만 응답을 넘깁니다.

REST API 서버는 화면을 다른 서버가 맡는 경우가 많습니다. 예를 들어 React 개발 서버(http://localhost:5173)에서 이 API 서버(http://localhost:8080)로 요청하면 포트가 달라 CORS 허용이 필요합니다. 1장 프로파일 설정에 적어 둔 허용 출처 값은 6장에서 CORS 설정과 연결합니다.

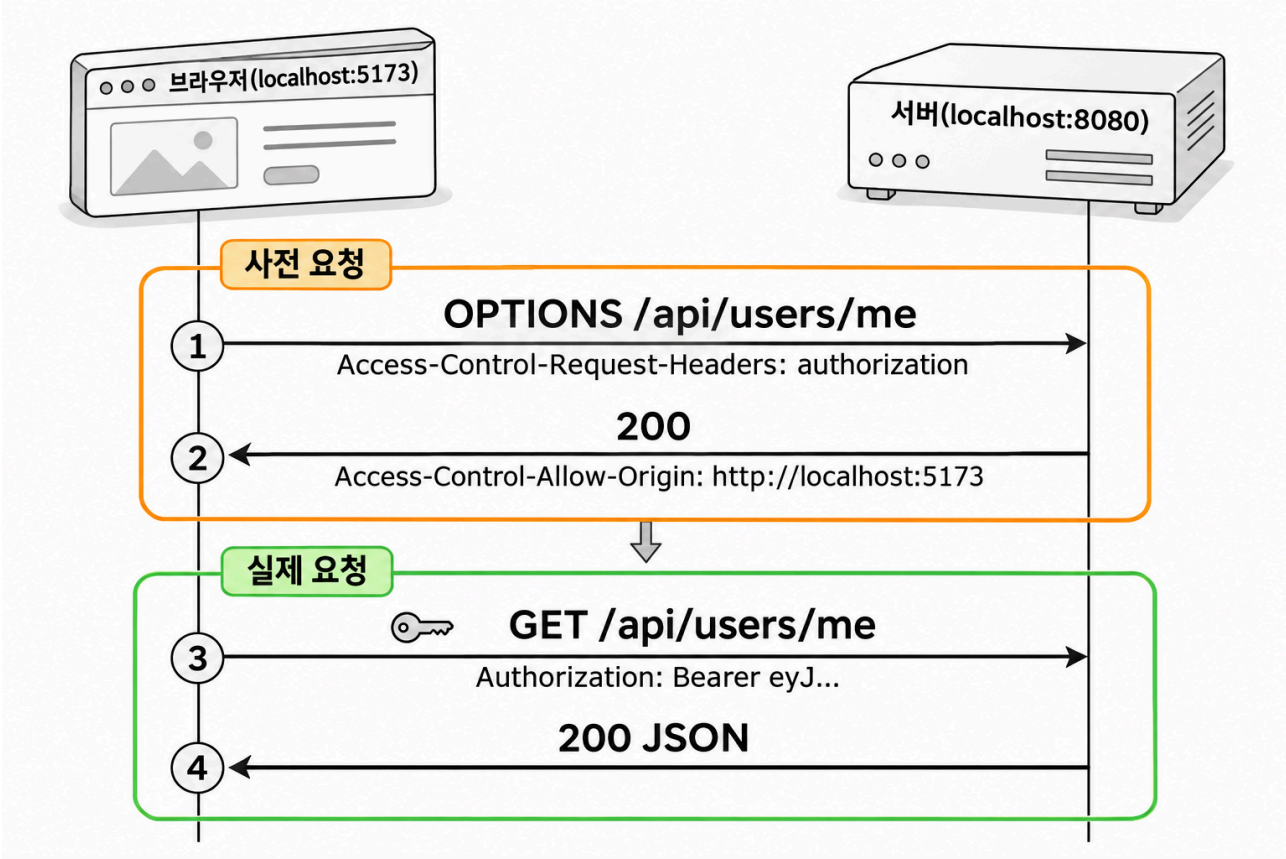
CORS 검사는 브라우저가 하므로 브라우저 밖에서 보내는 요청에는 동일 출처 정책이 적용되지 않습니다. 다만 서버 쪽 `CorsFilter`는 요청에 `Origin` 헤더가 있고 그 출처가 허용 목록에 없으면 요청을 거부합니다. 이 책에서 쓰는 Talend API Tester는 크롬 확장 프로그램이라 요청에 자기 출처를 담아 보냅니다. 그래서 6장에서 이 출처를 따로 허용합니다.

4.2 사전 요청(Preflight)

브라우저는 다른 출처로 보내는 요청 가운데 일부를 곧바로 보내지 않습니다. 실제 요청 전에 `OPTIONS` 메서드로 이런 요청을 보내도 되는지 먼저 묻습니다. 이것이 **사전 요청(Preflight)**입니다. 다음 가운데 하나라도 해당하면 사전 요청이 먼저 나갑니다.

- `Authorization`처럼 브라우저가 기본으로 허용하지 않는 헤더를 붙인다.
- `Content-Type`이 `application/json`이다.
- 메서드가 `GET` · `HEAD` · `POST`가 아니다(`PUT`, `DELETE` 등).

이 프로젝트의 요청은 토큰을 헤더에 싣거나 JSON을 보내므로 대부분 사전 요청을 거칩니다.



사전 요청과 실제 요청

그림 5-3에서 사전 요청에는 `Authorization` 헤더가 없습니다. 브라우저는 authorization 헤더를 보내도 되는지만 묻고 토큰은 싣지 않습니다. 그래서 CORS 처리는 인증 검사보다 앞에 있어야 합니다. 인증 검사가 먼저 실행되면 토큰이 없는 사전 요청은 401로 거절되고, 실제 요청은 아예 나가지 못합니다. 1.2의 필터 순서에서 `CorsFilter`가 `AuthorizationFilter`보다 앞에 있는 것도 이 때문입니다. `CorsFilter`는 허용된 출처의 사전 요청에 직접 응답하고, 요청을 뒤쪽 필터로 넘기지 않습니다. 이 필터를 등록하는 설정은 6장 2절에서 합니다.

4.3 CSRF와 JWT 방식

CSRF(Cross-Site Request Forgery, 사이트 간 요청 위조)는 사용자의 로그인 상태를 이용해 사용자가 의도하지 않은 요청을 서버로 보내게 하는 공격입니다. 브라우저가 요청을 보낼 때 그 사이트의 쿠키를 자동으로 붙인다는 점을 노립니다. 세션 방식이라면 공격자 사이트에 숨긴 폼 하나로 충분합니다. 사용자가 그 페이지를 열기만 해도 세션 쿠키가 붙은 요청이 사용자가 로그인해 둔 서버로 전송됩니다.

인증 정보를 보내는 방법	브라우저가 자동으로 붙이는가	CSRF 위험
쿠키(세션 방식)	붙인다	있음

인증 정보를 보내는 방법	브라우저가 자동으로 붙이는가	CSRF 위험
`Authorization` 헤더(JWT 방식)	붙이지 않는다. 프런트엔드 코드가 직접 넣는다	없음

스프링 시큐리티는 CSRF 보호가 기본으로 켜져 있어서 `POST` · `PUT` · `DELETE` 요청마다 CSRF 토큰을 요구합니다. 이 프로젝트는 인증 정보를 쿠키가 아니라 `Authorization` 헤더로 보냅니다. 다른 사이트가 이 헤더를 대신 채워 보낼 수 없으므로 CSRF 보호를 끕니다. 켜 두면 CSRF 토큰이 없는 회원가입 요청까지 403으로 막힙니다. 나중에 JWT를 쿠키에 담아 보내도록 바꾼다면 CSRF 위험이 다시 생기므로, 그때는 보호를 다시 켜야 합니다.

---

5. 지금 확인할 수 있는 것

5.1 서버 정상 실행 확인

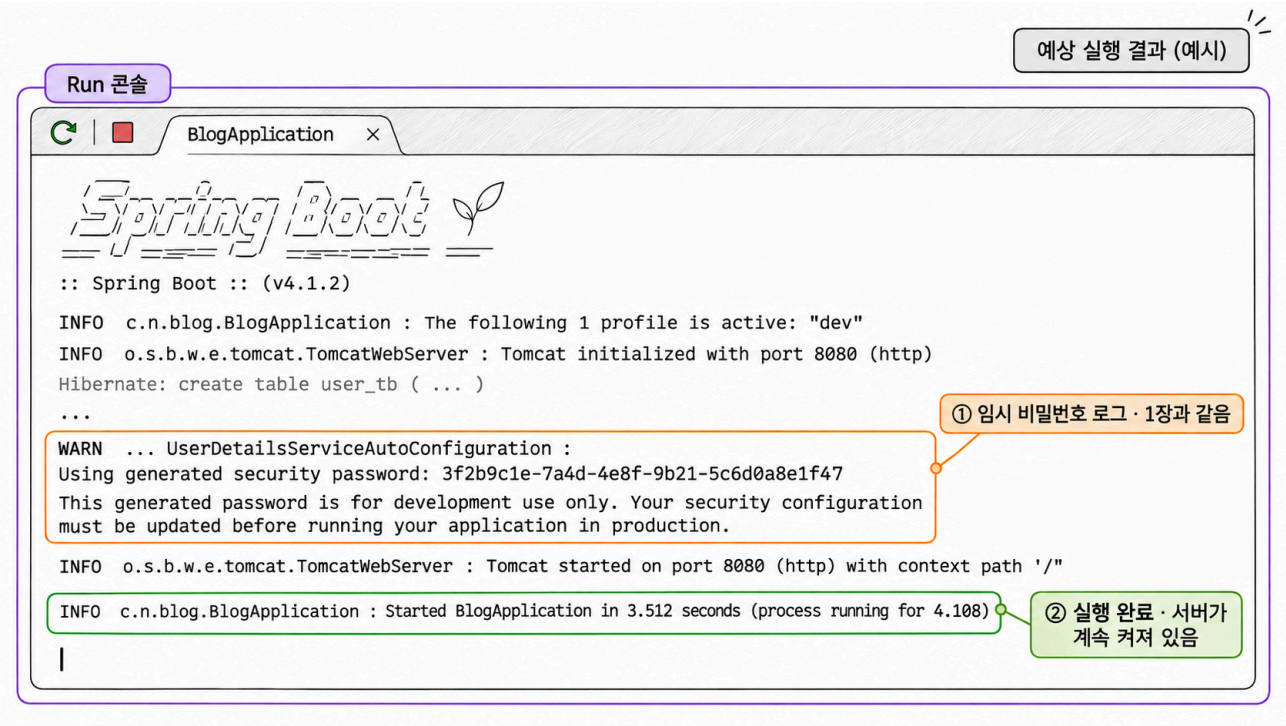
`JwtProvider`는 빈으로 등록되었지만 `JwtAuthorizationFilter`는 아직 시큐리티 필터 체인에 연결되지 않았습니다. 시큐리티 설정도 없으므로 1장에서 본 기본 동작, 즉 모든 요청이 로그인 화면으로 이동하는 동작이 그대로 남아 있습니다. 지금은 새 코드가 컴파일되고 서버가 오류 없이 실행되는지만 확인합니다.

IntelliJ에서 `BlogApplication`을 실행하고 Run 콘솔을 봅니다.

실행 결과

다음 두 로그가 보이면 정상입니다.

- 1장에서 본 임시 비밀번호 로그 `Using generated security password: ...`가 여전히 출력된다.
- 마지막에 `Started BlogApplication in ... seconds`가 출력되고 서버가 종료되지 않는다.



IntelliJ에서 BlogApplication을 실행한 Run 콘솔

생성형 예시 · 실제 실행 안 함

import 경로나 `JwtUtil`의 상수 이름이 틀리면 서버가 뜨기 전에 컴파일 오류가 납니다. 오류 메시지가 가리키는 줄을 예제 5-1, 예제 5-2와 비교해 고칩니다. 필터를 시큐리티 필터 체인에 넣고 토큰 없는 요청이 401로 돌아오는 것은 6장에서 확인합니다.

6장. SecurityFilterChain 설정하기

06

1. SecurityConfig 기본 틀

SecurityFilterChain은 보안 설정을 한데 묶은 것입니다. 어떤 필터를 어떤 순서로 실행할지, 어떤 주소를 누구에게 열지를 여기서 정합니다. `SecurityFilterChain` 빈을 직접 등록하면 스프링 시큐리티는 1장 3절의 기본 설정 대신 이 빈의 설정을 씁니다. 이 장에서는 `SecurityConfig` 클래스 하나에 이 빈을 만들고, 5장에서 만든 `JwtProvider`와 `JwtAuthorizationFilter`를 연결합니다.

설정은 `HttpSecurity` 객체의 기능별 메서드를 호출해서 만듭니다. 메서드마다 람다를 넘겨 세부 내용을 정하고, 마지막에 `http.build()`를 호출해 필터 체인을 완성합니다.

기본 문법

Java

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.기능(설정 -> 설정.세부설정());
    return http.build();
}
```

1.1 쓰지 않는 기능 끄기(CSRF · 폼 로그인 · HTTP Basic)

스프링 시큐리티는 SSR 웹사이트에 맞춰 몇 가지 기능을 기본으로 켜 둡니다. JWT로 인증하는 REST API에는 필요 없는 기능이므로 먼저 끕니다.

기능	기본 동작	끄는 이유
CSRF 보호	POST 같은 요청에 CSRF 토큰이 없으면 거부	토큰을 쿠키가 아니라 Authorization 헤더로 보내므로 CSRF 위험이 없습니다(5장 4.3)
폼 로그인	1장 3.1의 자동 로그인 화면 제공	로그인은 7장에서 JSON을 받는 API로 직접 만듭니다

기능	기본 동작	끄는 이유
HTTP Basic	브라우저 팝업 창으로 아이디와 비밀번호를 받음	요청마다 아이디와 비밀번호를 보내는 방식이라 JWT 방식과 맞지 않습니다

CSRF 보호를 켜 두면 7장의 회원가입 POST 요청이 403으로 막힙니다. 세 기능은 모두 `disable()` 로 끕니다.

1.2 세션 정책 STATELESS

세션 정책(session creation policy)은 시큐리티가 세션을 만들고 쓰는 방식을 정하는 값입니다. `SessionCreationPolicy.STATELESS` 로 두면 시큐리티는 세션을 만들지 않고, 세션에서 인증 정보를 찾지도 않습니다.

3장에서 본 Stateless(무상태) 방식이 이 설정 한 줄로 정해집니다. `JwtAuthorizationFilter` 가 `SecurityContextHolder` 에 넣은 인증 정보는 요청 하나가 끝나면 사라집니다. 다음 요청이 오면 필터가 토큰을 처음부터 다시 검증합니다.

1.3 JWT 필터 끼워 넣기

`JwtAuthorizationFilter` 는 5장에서 만들었지만 아직 필터 체인에 연결하지 않았습니다. `addFilterBefore()` 를 쓰면 새 필터가 기준 필터 바로 앞 자리에 들어갑니다.

기본 문법

Java

```
http.addFilterBefore(새필터, 기준필터.class);
```

기준 필터로 쓰는 `UsernamePasswordAuthenticationFilter` 는 폼 로그인을 처리하는 필터입니다. 1.1에서 폼 로그인을 껐으므로 이 필터는 실행되지 않습니다. 하지만 시큐리티가 필터마다 매겨 둔 순서 값은 남아 있어서 자리를 가리키는 기준으로는 쓸 수 있습니다. 이 자리는 권한을 검사하는 `AuthorizationFilter` 보다 앞입니다. 그래서 권한 검사가 시작되기 전에 `SecurityContextHolder` 에 인증 정보가 들어갑니다.

5장 3.2에서 보았듯이 `JwtAuthorizationFilter` 에는 `@Component` 가 없어서 스프링이 객체를 만들어 주지 않습니다. `SecurityConfig` 가 주입받은 `JwtProvider` 를 생성자에 넘겨 `new` 로 직접 만듭니다.

1.4 PasswordEncoder 빈 등록

`PasswordEncoder`는 비밀번호를 암호화하고 비교하는 도구입니다. 실제로는 7장의 회원가입과 로그인에서 쓰지만, 보안 설정에 속하는 빈이라 `SecurityConfig`에 함께 등록합니다. 이 장에서는 등록만 해 둡니다. BCrypt가 비밀번호를 어떻게 암호화하는지는 7장 3절에서 다룹니다.

1.1부터 1.4까지의 설정을 모으면 `SecurityConfig`의 기본 틀이 됩니다.

예제 6-1 — SecurityConfig 기본 틀 만들기

이어서: 예제 5-2

파일명: `src/main/java/com/newtenco/blog/core/SecurityConfig.java`

Java

```java
package com.newtenco.blog.core;

import com.newtenco.blog.core.filter.JwtAuthorizationFilter;
import com.newtenco.blog.core.util.JwtProvider;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@RequiredArgsConstructor
@Configuration
public class SecurityConfig {

    private final JwtProvider jwtProvider;

    // 비밀번호 암호화 도구 (회원가입: encode / 로그인: matches) - 7장에서 사용
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // 1. 쓰지 않는 기본 기능 끄기
        http.csrf(csrf -> csrf.disable());           // 쿠키를 쓰지 않으므로 CSRF 위험 없음
        http.formLogin(form -> form.disable());      // 스프링 기본 로그인 화면 끄기
        http.httpBasic(basic -> basic.disable());    // 브라우저 팝업 로그인 끄기

        // 2. 세션을 만들지 않는다 (JWT = Stateless)
        http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // 3. JWT 필터를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣는다
        http.addFilterBefore(new JwtAuthorizationFilter(jwtProvider),
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

실행 결과

서버가 오류 없이 실행됩니다. 폼 로그인을 껐으므로 브라우저로 `http://localhost:8080` 을 열어도 1장의 자동 로그인 화면은 더 이상 나오지 않습니다. 아직 컨트롤러가 없으므로, 4장의 `GlobalExceptionHandler` 가 404를 `Resp` 모양으로 바꾼 JSON이 보입니다.

JSON

```
{"status":404,"msg":"Not Found","body":null}
```

접근 규칙을 하나도 정하지 않았으므로 지금은 모든 요청이 그대로 통과합니다. 접근 규칙은 3절에서 붙입니다.

---

2. CORS 설정

2.1 CorsConfigurationSource

5장 4.1에서 보았듯이, 브라우저는 다른 출처(Origin)로 보낸 요청의 응답을 서버가 허락할 때만 넘겨줍니다. React 같은 프런트엔드 개발 서버는 보통 `http://localhost:5173` 에서 실행되고, API 서버는 `http://localhost:8080` 에서 실행됩니다. 포트가 다르므로 둘은 서로 다른 출처입니다. 서버가 CORS 응답 헤더를 붙이지 않으면 브라우저가 응답을 막습니다.

`CorsConfigurationSource` 는 요청 주소마다 어떤 CORS 규칙을 적용할지 알려 주는 객체입니다. 규칙 하나는 `CorsConfiguration` 에 담고, `UrlBasedCorsConfigurationSource` 에 주소 패턴과 함께 등록합니다.

메서드	정하는 것
`setAllowedOriginPatterns()`	허용할 출처(패턴에 `*` 사용 가능)
`setAllowedMethods()`	허용할 HTTP 메서드
`addAllowedHeader("*")`	요청에 붙여도 되는 헤더(`*` 는 모두)
`setAllowCredentials()`	쿠키 같은 인증 정보를 함께 보내도 되는지

이 프로젝트는 토큰을 쿠키가 아니라 Authorization 헤더에 실어 보내므로 `setAllowCredentials(false)` 로 둡니다. 허용 메서드에 `OPTIONS` 를 넣는 것은 5장 4.2의 사전 요청(Preflight) 때문입니다.

이 빈을 만든 뒤 `http.cors()`에 넘겨야 시큐리티 필터 체인이 CORS를 처리합니다. 시큐리티 필터는 컨트롤러보다 먼저 실행되므로 CORS도 필터 단계에서 처리해야 합니다. 그래야 사전 요청이 인증 검사에 걸리지 않습니다.

2.2 허용 출처를 설정 파일에서 읽기

허용 출처는 개발 환경과 운영 환경에서 다릅니다. 코드에 직접 적으면 환경이 바뀔 때마다 코드를 고쳐야 합니다. 그래서 1장 4.3에서 `application-dev.properties`에 적어 둔 `app.cors.allowed-origins` 값을 읽어 씁니다.

`@Value("${설정 키}")`는 설정 파일의 값을 필드에 넣어 줍니다. 이 값은 `http://localhost:3000,http://localhost:5173`처럼 콤마로 이어진 문자열 하나입니다. `split(",")`으로 나눠서 목록으로 바꿉니다.

예제 6-2 — CORS 설정 추가하기

이어서: 예제 6-1

파일명: `src/main/java/com/newtenco/blog/core/SecurityConfig.java`

Java

```java
package com.newtenco.blog.core;

import com.newtenco.blog.core.filter.JwtAuthorizationFilter;
import com.newtenco.blog.core.util.JwtProvider;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@RequiredArgsConstructor
@Configuration
public class SecurityConfig {

    private final JwtProvider jwtProvider;

    // application-dev.properties 의 app.cors.allowed-origins 값을 읽는다
    @Value("${app.cors.allowed-origins}")
    private String allowedOrigins;

    // 비밀번호 암호화 도구 (회원가입: encode / 로그인: matches) - 7장에서 사용
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // CORS: 다른 출처(예: React localhost:5173)에서 오는 요청을 허용한다
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of(allowedOrigins.split(",")));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.addAllowedHeader("*");
        // JWT 는 쿠키가 아니라 Authorization 헤더로 보내므로 쿠키 동봉은 필요 없다
        config.setAllowCredentials(false);
```

```
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // 1. CORS 적용
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()));

        // 2. 쓰지 않는 기본 기능 끄기
        http.csrf(csrf -> csrf.disable());           // 쿠키를 쓰지 않으므로 CSRF 위험 없음
        http.formLogin(form -> form.disable());      // 스프링 기본 로그인 화면 끄기
        http.httpBasic(basic -> basic.disable());    // 브라우저 팝업 로그인 끄기

        // 3. 세션을 만들지 않는다 (JWT = Stateless)
        http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // 4. JWT 필터를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣는다
        http.addFilterBefore(new JwtAuthorizationFilter(jwtProvider),
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

실행 결과

서버가 오류 없이 실행됩니다. 설정 파일에서 `app.cors.allowed-origins` 키를 지우거나 이름을 잘못 쓰면 `@Value`가 읽을 값을 찾지 못해 서버가 시작되지 않습니다.

`registerCorsConfiguration("/**", config)`는 서버의 모든 주소에 같은 CORS 규칙을 적용합니다. 설정 파일에 출처를 여러 개 적을 때는 콤마 뒤에 공백을 넣지 않습니다. `split(",")`은 공백을 지우지 않으므로, 공백이 붙은 출처는 실제 요청의 출처와 맞지 않을 수 있습니다.

2.3 Talend API Tester를 위한 chrome-extension://* 허용

이 책은 크롬 확장 프로그램인 Talend API Tester로 API를 요청합니다. 확장 프로그램이 보내는 요청에는 `Origin: chrome-extension://확장프로그램ID` 헤더가 붙습니다. 이 출처는 허용 목록에 없습니다. 그래서 스프링의 CORS 처리기는 요청을 컨트롤러로 넘기지 않고 403으로 거부합니다. 이때 본문은 Resp 모양이 아니라 `Invalid CORS request`라는 글자뿐입니다. 이대로 두면 7장의 첫 회원가입 요청부터 막힙니다.

`setAllowedOriginPatterns()`는 출처에 `*` 패턴을 받습니다. 허용 출처에 `chrome-extension://*`를 더하면 모든 크롬 확장 프로그램의 요청을 받아 줍니다. 코드는 그대로 두고 설정 파일의 값만 바꿉니다.

예제 6-3 – 허용 출처에 크롬 확장 프로그램 더하기

이어서: 예제 6-2

파일명: `src/main/resources/application-dev.properties`

PROPERTIES

```
# ===== Server =====
server.port=8080
server.servlet.encoding.charset=UTF-8
server.servlet.encoding.force=true

# ===== H2 Datasource (메모리 모드: 서버를 끄면 데이터가 사라짐) =====
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.url=jdbc:h2:mem:test
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true

# ===== JPA / Hibernate =====
# create: 실행할 때마다 테이블을 새로 만든다 (개발 단계 전용, 운영에서 사용 금지)
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false

# ===== SQL Init (data.sql) =====
spring.sql.init.data-locations=classpath:db/data.sql
# 테이블 생성(JPA) 이후에 data.sql 을 실행한다
spring.jpa.defer-datasource-initialization=true

# ===== Logging Level =====
logging.level.com.newtenco.blog=DEBUG

# ===== CORS Allow (콤마로 여러 개 지정 가능, 콤마 뒤에 공백 금지) =====
# chrome-extension://* : Talend API Tester 같은 크롬 확장 프로그램 허용 (개발용)
app.cors.allowed-origins=http://localhost:3000,http://localhost:5173,chrome-extension://*
```

실행 결과

서버가 오류 없이 실행됩니다. 확장 프로그램의 요청이 실제로 통과하는지는 6절에서 Talend API Tester로 확인합니다.

`chrome-extension://*`는 개발할 때만 쓰는 값이라 `application-dev.properties`에만 둡니다. 운영 환경의 설정 파일에는 실제 프런트엔드 주소만 적습니다.

---

3. URL · 역할별 접근 규칙

`authorizeHttpRequests()`는 요청 주소와 메서드마다 누가 들어올 수 있는지 정합니다. 이 규칙은 필터 체인 끝 쪽의 `AuthorizationFilter`가 검사합니다. JWT 필터는 `SecurityContextHolder`에 인증 정보를 넣을 뿐 요청을 막지 않습니다(5장 3.3). 요청을 막을지는 이 규칙이 정합니다.

기본 문법

Java

```
http.authorizeHttpRequests(auth -> auth
        .requestMatchers(HTTP메서드, "주소패턴").규칙()
        .requestMatchers("주소패턴").규칙()
        .anyRequest().규칙()
);
```

주소 패턴의 `/**`는 그 아래의 모든 주소를 뜻합니다. 예를 들어 `/api/**`는 `/api/users/me`에도 맞고 `/api/admin/hello`에도 맞습니다.

3.1 OPTIONS 사전 요청 허용

브라우저는 Authorization 헤더를 붙인 요청을 보내기 전에 OPTIONS 메서드로 사전 요청을 먼저 보냅니다(5장 4.2). 이 사전 요청은 2절의 CORS 설정이 필터 단계에서 처리합니다. `CorsFilter`는 허용된 출처의 사전 요청에 직접 응답하고 필터 체인을 거기서 끝내므로, 사전 요청은 `AuthorizationFilter`까지 오지 않습니다.

그래도 첫 규칙으로 모든 주소의 OPTIONS 요청을 허용해 둡니다. CORS 설정이 빠지거나 일부 주소에 적용되지 않으면 사전 요청이 이 규칙까지 내려옵니다. 사전 요청에는 토큰이 없으므로 로그인 필요 규칙에 걸려 401을 받고, 브라우저는 실제 요청을 보내지 않습니다. OPTIONS 허용 규칙은 이런 경우를 대비한 안전장치입니다.

코드 조각

Java

```
.requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
```

3.2 permitAll · hasRole · hasAnyRole · denyAll

규칙	뜻
`permitAll()`	토큰이 없어도 누구나 허용
`hasRole("ADMIN")`	`ROLE_ADMIN` 권한이 있을 때만 허용
`hasAnyRole("USER", "ADMIN")`	`ROLE_USER` 나 `ROLE_ADMIN` 가운데 하나만 있어도 허용
`denyAll()`	누구도 허용하지 않음

`hasRole()` 에는 `ROLE_` 을 빼고 씁니다. 시큐리티가 앞에 `ROLE_` 을 붙인 뒤 권한 목록과 비교하기 때문입니다. 2장 1.3에서 `getAuthorities()` 가 `ROLE_` 을 붙인 권한을 돌려주게 만든 이유가 여기에 있습니다. `hasRole("ROLE_ADMIN")` 처럼 접두사까지 쓰면 서버가 시작될 때 오류가 납니다.

이 프로젝트의 규칙은 다음과 같습니다. 게시글 규칙은 2장의 Board 엔티티로 게시판 API를 붙일 때를 대비해 미리 넣어 둡니다.

순서	주소	규칙	예
1	OPTIONS `/**`	모두 허용	CORS 사전 요청
2	`/join` , `/login` , `/check-username`	모두 허용	회원가입, 로그인, 중복체크(7장)
3	`/h2-console/**`	모두 허용	DB 확인(5절)
4	`/api/admin/**`	ADMIN만	`/api/admin/hello` (8장)
5	GET `/api/boards/**`	모두 허용	글 목록, 상세
6	`/api/**`	USER 또는 ADMIN	`/api/users/me` (8장), 글쓰기
7	그 외	모두 거부	정하지 않은 주소

마지막 규칙을 `anyRequest().permitAll()` 로 두면 편해 보입니다. 하지만 새 API를 만들고 규칙 추가를 잊으면 그 API가 누구에게나 열립니다. 허용할 주소만 하나씩 열고 나머지를 `denyAll()` 로 막아

두면 이런 실수가 보안 사고로 번지지 않습니다.

예제 6-4 — URL · 역할별 접근 규칙 추가하기

이어서: 예제 6-3

파일명: `src/main/java/com/newtenco/blog/core/SecurityConfig.java`

Java

```java
package com.newtenco.blog.core;

import com.newtenco.blog.core.filter.JwtAuthorizationFilter;
import com.newtenco.blog.core.util.JwtProvider;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@RequiredArgsConstructor
@Configuration
public class SecurityConfig {

    private final JwtProvider jwtProvider;

    // application-dev.properties 의 app.cors.allowed-origins 값을 읽는다
    @Value("${app.cors.allowed-origins}")
    private String allowedOrigins;

    // 비밀번호 암호화 도구 (회원가입: encode / 로그인: matches) - 7장에서 사용
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // CORS: 다른 출처(예: React localhost:5173)에서 오는 요청을 허용한다
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of(allowedOrigins.split(",")));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.addAllowedHeader("*");
        // JWT 는 쿠키가 아니라 Authorization 헤더로 보내므로 쿠키 동봉은 필요 없다
        config.setAllowCredentials(false);
```

```
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // 1. CORS 적용
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()));

        // 2. 쓰지 않는 기본 기능 끄기
        http.csrf(csrf -> csrf.disable());           // 쿠키를 쓰지 않으므로 CSRF 위험 없음
        http.formLogin(form -> form.disable());      // 스프링 기본 로그인 화면 끄기
        http.httpBasic(basic -> basic.disable());    // 브라우저 팝업 로그인 끄기

        // 3. 세션을 만들지 않는다 (JWT = Stateless)
        http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // 4. JWT 필터를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣는다
        http.addFilterBefore(new JwtAuthorizationFilter(jwtProvider),
UsernamePasswordAuthenticationFilter.class);

        // 5. URL 별 접근 규칙 (위에서부터 먼저 맞는 규칙이 적용된다)
        http.authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()                // CORS
사전 요청
                .requestMatchers("/join", "/login", "/check-username").permitAll()     // 회원가
입, 로그인, 중복체크
                .requestMatchers("/h2-console/**").permitAll()                         // H2 콘
솔
                .requestMatchers("/api/admin/**").hasRole("ADMIN")                     // 관리자
전용
                .requestMatchers(HttpMethod.GET, "/api/boards/**").permitAll()         // 글 목
록/상세는 누구나
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")               // 나머지
API 는 로그인 필요
                .anyRequest().denyAll()                                                // 그 외
주소는 모두 차단
        );

        return http.build();
    }
}
```

실행 결과

서버가 오류 없이 실행됩니다. 이제 요청은 정해 둔 규칙에 따라 허용되거나 막힙니다. 막힌 요청의 응답 모양은 4절에서 정합니다.

3.3 규칙 순서가 중요한 이유

`AuthorizationFilter`는 규칙을 위에서부터 차례로 비교합니다. 요청에 처음 맞는 규칙 하나만 적용하고, 그 아래 규칙은 보지 않습니다.

5번과 6번 규칙의 순서를 바꾸면 다음과 같습니다.

코드 조각

Java

```
.requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
.requestMatchers(HttpMethod.GET, "/api/boards/**").permitAll()   // 이 규칙까지 오지 못한다
```

`GET /api/boards` 요청은 윗줄의 `/api/**`에 먼저 맞습니다. 누구나 볼 수 있어야 할 글 목록에 로그인이 필요해집니다. 4번 관리자 규칙도 같습니다. 이 규칙을 `/api/**` 아래로 내리면 USER 토큰으로도 `/api/admin/hello`에 들어갈 수 있습니다.

규칙은 좁은 주소를 위에, 넓은 주소를 아래에 씁니다. 가장 넓은 `anyRequest()`는 늘 마지막에 둡니다. `anyRequest()` 뒤에 다른 규칙을 쓰면 서버가 시작될 때 오류가 납니다.

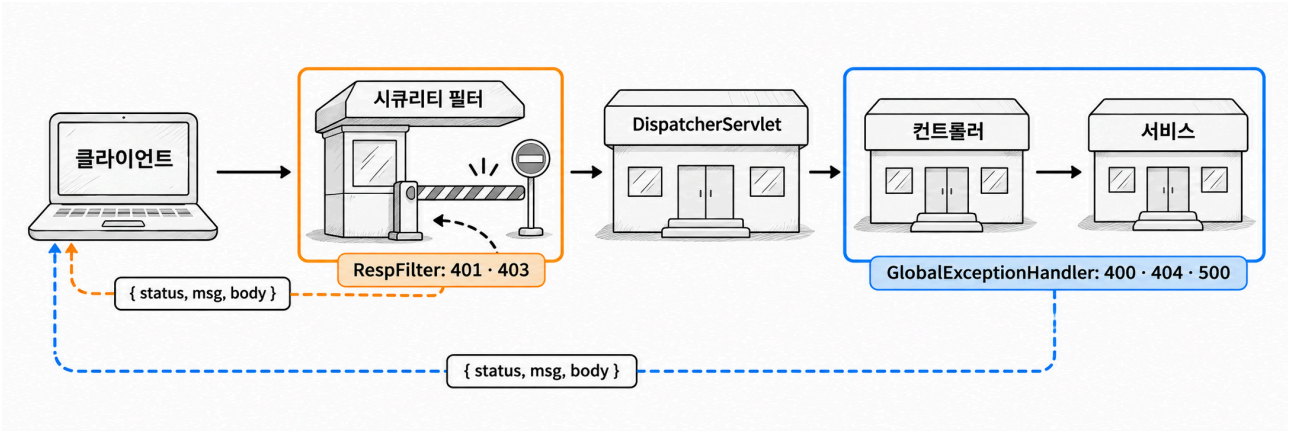
---

4. 401 · 403 실패 응답

4.1 필터 단계의 응답을 따로 만드는 이유

예제 6-4 상태에서 토큰 없이 보호 주소를 요청하면 401이 아니라 403이 옵니다. 본문도 4장에서 정한 Resp 모양의 JSON이 아닙니다. 폼 로그인과 HTTP Basic을 끄면서 인증 실패를 처리하던 기능도 함께 사라졌기 때문입니다. 이때 시큐리티는 기본 처리기로 403만 보냅니다.

4장의 `GlobalExceptionHandler`로는 이 문제를 고칠 수 없습니다. `@RestControllerAdvice`는 컨트롤러와 서비스에서 던진 예외를 잡습니다. 시큐리티 필터에서 막힌 요청은 DispatcherServlet과 컨트롤러에 닿지 못하므로 그 처리기까지 가지 않습니다. 컨트롤러가 돌려준 객체를 JSON으로 바꿔 주는 스프링 MVC의 자동 변환도 일어나지 않습니다.



실패 응답을 만드는 곳

그림 6-1처럼 실패 응답을 만드는 곳은 두 군데입니다. 컨트롤러 안쪽은 `GlobalExceptionHandler`가 맡습니다. 필터 단계에서는 응답 객체에 JSON을 직접 쓰는 도구가 필요합니다. 두 곳이 같은 Resp 모양으로 응답해야 클라이언트가 모든 실패 응답을 한 가지 방법으로 읽을 수 있습니다.

4.2 RespFilter와 Jackson 3

RespFilter는 필터 단계에서 `HttpServletResponse`에 상태 코드와 Resp 모양의 JSON을 직접 써 주는 도구입니다. 객체를 JSON 문자열로 바꾸는 일은 JSON 라이브러리인 **Jackson**이 맡습니다.

스프링부트 4는 Jackson 3을 씁니다. Jackson 3은 패키지 이름이 `com.fasterxml.jackson`에서 `tools.jackson`으로 바뀌었습니다. 객체를 JSON으로 바꿀 때도 `ObjectMapper` 대신 `JsonMapper`를 씁니다. 인터넷에 있는 예전 코드처럼 `com.fasterxml.jackson.databind.ObjectMapper`를 import하면 컴파일 오류가 납니다.

구분	Jackson 2(스프링부트 3)	Jackson 3(스프링부트 4)
패키지	`com.fasterxml.jackson.databind`	`tools.jackson.databind`
변환 도구	`ObjectMapper`	`JsonMapper`
변환 예외	체크 예외 (`JsonProcessingException`)	언체크 예외(`RuntimeException` 계열)

Jackson 3의 예외는 `RuntimeException` 계열입니다. 그래서 `writeValueAsString()`을 try-catch로 감싸지 않아도 됩니다.

예제 6-5 – 필터용 응답 도구 RespFilter 만들기

이어서: 예제 6-4

파일명: `src/main/java/com/newtenco/blog/core/util/RespFilter.java`

Java

```java
package com.newtenco.blog.core.util;

import jakarta.servlet.http.HttpServletResponse;
import tools.jackson.databind.json.JsonMapper;

import java.io.IOException;

/**
 * 컨트롤러에 도착하기 전(시큐리티 필터 단계)에서 실패했을 때
 * Resp 모양의 JSON 으로 직접 응답하는 도구.
 * 필터 단계에서는 스프링 MVC 의 자동 JSON 변환이 동작하지 않으므로 직접 변환한다.
 */
public class RespFilter {

    // Spring Boot 4 는 Jackson 3 를 사용한다 (패키지: tools.jackson)
    private static final JsonMapper mapper = JsonMapper.builder().build();

    public static void fail(HttpServletResponse response, int status, String msg) throws
IOException {
        response.setStatus(status);
        response.setContentType("application/json;charset=utf-8");

        String body = mapper.writeValueAsString(new Resp<>(status, msg, null));
        response.getWriter().write(body);
    }
}
```

실행 결과

서버가 오류 없이 실행됩니다. `RespFilter`는 4.3에서 시큐리티에 연결해야 동작합니다.

`RespFilter`는 4장의 `Resp` record와 같은 `core.util` 패키지에 있어서 import 없이 `new Resp<>(...)`를 씁니다. 컨트롤러와 같은 record를 JSON으로 바꾸므로 응답 모양도 똑같습니다.
`setContentType()`에 `charset=utf-8`을 빼면 한글 메시지가 깨져 보일 수 있습니다.

4.3 authenticationEntryPoint와 accessDeniedHandler

`exceptionHandling()`에는 실패 이유에 따라 처리기 두 개를 등록합니다.

처리기	실행되는 때	응답
`authenticationEntryPoint`	인증 정보 없이 로그인이 필요한 주소를 요청했을 때	401
`accessDeniedHandler`	로그인은 했지만 권한이 모자랄 때	403

처리기가 둘로 나뉘는 것은 5장 1.1의 인증(Authentication)과 인가(Authorization) 구분 때문입니다. 토큰이 없거나 만료 · 위조되었으면 `JwtAuthorizationFilter`는 `SecurityContextHolder`를 비워 둔 채 요청을 넘깁니다. 시큐리티는 이 요청을 로그인하지 않은 요청으로 보고 `authenticationEntryPoint`를 호출합니다. USER 토큰으로 `/api/admin/**`를 요청한 경우는 다릅니다. 인증은 되었지만 `ROLE_ADMIN`이 없으므로 `accessDeniedHandler`가 실행됩니다.

두 처리기는 메서드가 하나뿐인 인터페이스라서 람다로 씁니다. 람다의 세 인자는 차례로 요청, 응답, 발생한 예외입니다.

예제 6-6 — 401 · 403 응답을 JSON으로 바꾸기

이어서: 예제 6-5

파일명: `src/main/java/com/newtenco/blog/core/SecurityConfig.java`

Java

```java
package com.newtenco.blog.core;

import com.newtenco.blog.core.filter.JwtAuthorizationFilter;
import com.newtenco.blog.core.util.JwtProvider;
import com.newtenco.blog.core.util.RespFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@RequiredArgsConstructor
@Configuration
public class SecurityConfig {

    private final JwtProvider jwtProvider;

    // application-dev.properties 의 app.cors.allowed-origins 값을 읽는다
    @Value("${app.cors.allowed-origins}")
    private String allowedOrigins;

    // 비밀번호 암호화 도구 (회원가입: encode / 로그인: matches) - 7장에서 사용
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // CORS: 다른 출처(예: React localhost:5173)에서 오는 요청을 허용한다
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of(allowedOrigins.split(",")));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.addAllowedHeader("*");
        // JWT 는 쿠키가 아니라 Authorization 헤더로 보내므로 쿠키 동봉은 필요 없다
```

```
        config.setAllowCredentials(false);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // 1. CORS 적용
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()));

        // 2. 쓰지 않는 기본 기능 끄기
        http.csrf(csrf -> csrf.disable());           // 쿠키를 쓰지 않으므로 CSRF 위험 없음
        http.formLogin(form -> form.disable());      // 스프링 기본 로그인 화면 끄기
        http.httpBasic(basic -> basic.disable());    // 브라우저 팝업 로그인 끄기

        // 3. 세션을 만들지 않는다 (JWT = Stateless)
        http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // 4. JWT 필터를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣는다
        http.addFilterBefore(new JwtAuthorizationFilter(jwtProvider),
UsernamePasswordAuthenticationFilter.class);

        // 5. 인증(401) / 권한(403) 실패 시 JSON 으로 응답
        http.exceptionHandling(ex -> ex
                .authenticationEntryPoint((req, res, e) -> RespFilter.fail(res, 401, "로그인 후
이용해주세요"))
                .accessDeniedHandler((req, res, e) -> RespFilter.fail(res, 403, "권한이 없습니
다")));

        // 6. URL 별 접근 규칙 (위에서부터 먼저 맞는 규칙이 적용된다)
        http.authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()                // CORS
사전 요청
                .requestMatchers("/join", "/login", "/check-username").permitAll()     // 회원가
입, 로그인, 중복체크
                .requestMatchers("/h2-console/**").permitAll()                         // H2 콘
솔
                .requestMatchers("/api/admin/**").hasRole("ADMIN")                     // 관리자
전용
                .requestMatchers(HttpMethod.GET, "/api/boards/**").permitAll()         // 글 목
록/상세는 누구나
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")                // 나머지
API 는 로그인 필요
                .anyRequest().denyAll()                                                // 그 외
```

```
주소는 모두 차단
        );

        return http.build();
    }
}
```

실행 결과

서버가 오류 없이 실행됩니다. 401 응답은 6절에서 확인합니다. 403 응답은 8장에서 관리자 API를 만든 뒤 확인합니다.

`AuthenticationEntryPoint`와 `AccessDeniedHandler`의 메서드는 `IOException`을 던질 수 있게 선언되어 있습니다. 그래서 `IOException`을 던지는 `RespFilter.fail()`을 람다 안에서 바로 호출해도 됩니다. 처리기가 응답을 직접 쓰고 끝내므로 요청은 컨트롤러까지 가지 않습니다.

---

5. H2 콘솔 열기

5.1 frameOptions와 콘솔 접속 허용

H2 콘솔 화면은 HTML의 `<iframe>`(페이지 안에 다른 페이지를 넣는 틀)으로 만들어져 있습니다. **클릭재킹(clickjacking)**은 내 사이트를 다른 사이트의 iframe 안에 숨겨 두고 사용자가 모르게 버튼을 누르게 만드는 공격입니다. 스프링 시큐리티는 이 공격을 막으려고 모든 응답에 `X-Frame-Options: DENY` 헤더를 붙입니다. 브라우저는 이 헤더가 붙은 페이지를 iframe에 띄우지 않으므로 H2 콘솔이 빈 화면으로 나옵니다.

`frameOptions(frame -> frame.sameOrigin())`은 같은 출처의 iframe만 허용합니다. H2 콘솔은 서버와 같은 `localhost:8080`에서 열리므로 화면이 제대로 보입니다. 다른 사이트가 이 서버를 iframe에 넣는 것은 계속 막힙니다.

콘솔 주소 `/h2-console/**`는 3절에서 이미 `permitAll()`로 열어 두었습니다. 콘솔이 보내는 폼 요청에는 CSRF 토큰이 없지만 1.1에서 CSRF 보호를 껐으므로 막히지 않습니다. 이 예제의 `SecurityConfig`가 이 장의 최종 코드입니다.

예제 6-7 — H2 콘솔 화면 허용하기

이어서: 예제 6-6

파일명: `src/main/java/com/newtenco/blog/core/SecurityConfig.java`

Java

```java
package com.newtenco.blog.core;

import com.newtenco.blog.core.filter.JwtAuthorizationFilter;
import com.newtenco.blog.core.util.JwtProvider;
import com.newtenco.blog.core.util.RespFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@RequiredArgsConstructor
@Configuration
public class SecurityConfig {

    private final JwtProvider jwtProvider;

    // application-dev.properties 의 app.cors.allowed-origins 값을 읽는다
    @Value("${app.cors.allowed-origins}")
    private String allowedOrigins;

    // 비밀번호 암호화 도구 (회원가입: encode / 로그인: matches) - 7장에서 사용
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    // CORS: 다른 출처(예: React localhost:5173)에서 오는 요청을 허용한다
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of(allowedOrigins.split(",")));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.addAllowedHeader("*");
        // JWT 는 쿠키가 아니라 Authorization 헤더로 보내므로 쿠키 동봉은 필요 없다
```

```
        config.setAllowCredentials(false);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // 1. H2 콘솔은 iframe 으로 동작하므로 같은 출처의 iframe 은 허용
        http.headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()));

        // 2. CORS 적용
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()));

        // 3. 쓰지 않는 기본 기능 끄기
        http.csrf(csrf -> csrf.disable());           // 쿠키를 쓰지 않으므로 CSRF 위험 없음
        http.formLogin(form -> form.disable());      // 스프링 기본 로그인 화면 끄기
        http.httpBasic(basic -> basic.disable());    // 브라우저 팝업 로그인 끄기

        // 4. 세션을 만들지 않는다 (JWT = Stateless)
        http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

        // 5. JWT 필터를 UsernamePasswordAuthenticationFilter 앞에 끼워 넣는다
        http.addFilterBefore(new JwtAuthorizationFilter(jwtProvider),
UsernamePasswordAuthenticationFilter.class);

        // 6. 인증(401) / 권한(403) 실패 시 JSON 으로 응답
        http.exceptionHandling(ex -> ex
                .authenticationEntryPoint((req, res, e) -> RespFilter.fail(res, 401, "로그인 후
이용해주세요"))
                .accessDeniedHandler((req, res, e) -> RespFilter.fail(res, 403, "권한이 없습니
다")));

        // 7. URL 별 접근 규칙 (위에서부터 먼저 맞는 규칙이 적용된다)
        http.authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()                // CORS
사전 요청
                .requestMatchers("/join", "/login", "/check-username").permitAll()     // 회원가
입, 로그인, 중복체크
                .requestMatchers("/h2-console/**").permitAll()                         // H2 콘
솔
                .requestMatchers("/api/admin/**").hasRole("ADMIN")                     // 관리자
전용
                .requestMatchers(HttpMethod.GET, "/api/boards/**").permitAll()         // 글 목
록/상세는 누구나
```

```
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")                // 나머지
API 는 로그인 필요
                .anyRequest().denyAll()                                                 // 그 외
주소는 모두 차단
        );

        return http.build();
    }
}
```

실행 결과

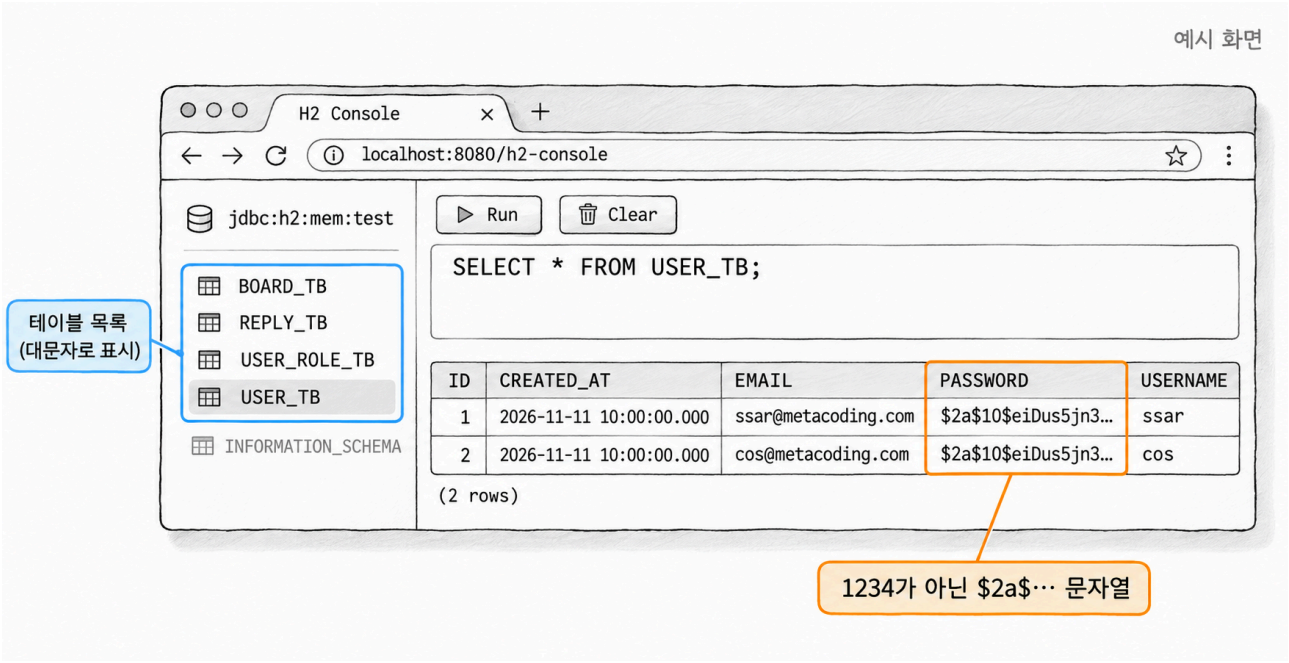
서버가 오류 없이 실행됩니다. 브라우저로 `http://localhost:8080/h2-console`을 열면 H2 콘솔의 접속 화면이 나옵니다.

5.2 테이블과 더미 데이터 확인

접속 화면에는 1장 4.2의 `application-dev.properties`와 같은 값을 넣습니다.

칸	입력값
JDBC URL	`jdbc:h2:mem:test`
User Name	`sa`
Password	비워 둠

Connect를 누르면 왼쪽 목록에 `BOARD_TB`, `REPLY_TB`, `USER_ROLE_TB`, `USER_TB` 테이블이 보입니다. H2 콘솔은 테이블 이름을 대문자로 보여 줍니다. 테이블 이름을 누르면 오른쪽 입력 창에 `SELECT` 문이 채워집니다. **Run**을 누르면 2장에서 `data.sql`로 넣은 더미 데이터가 나옵니다. `USER_TB`의 PASSWORD 칸에는 `1234`가 아니라 `$2a$`로 시작하는 긴 문자열이 들어 있습니다. 이 문자열의 정체는 7장 3절에서 설명합니다.



브라우저로 연 H2 콘솔에서 USER_TB를 조회한 화면

생성형 예시 · 실제 실행 안 함

메모리 DB이므로 서버를 다시 시작하면 테이블과 데이터가 새로 만들어집니다. 콘솔에서 데이터를 고쳐도 재시작하면 `data.sql`의 처음 상태로 돌아갑니다. JDBC URL 칸에 `jdbc:h2:~/test` 같은 다른 값이 채워져 있으면 빈 DB에 접속됩니다. 이때는 반드시 `jdbc:h2:mem:test`로 고친 뒤 접속합니다.

---

6. 지금 확인할 수 있는 것

6.1 토큰 없이 /api/users/me 요청하면 401 JSON

필터 체인이 모두 연결되었으므로, 필터 단계의 실패 응답이 4장의 공통 응답 모양으로 오는지 처음 확인할 수 있습니다. 크롬에서 Talend API Tester를 열고 다음과 같이 요청합니다. Authorization 헤더는 넣지 않습니다.

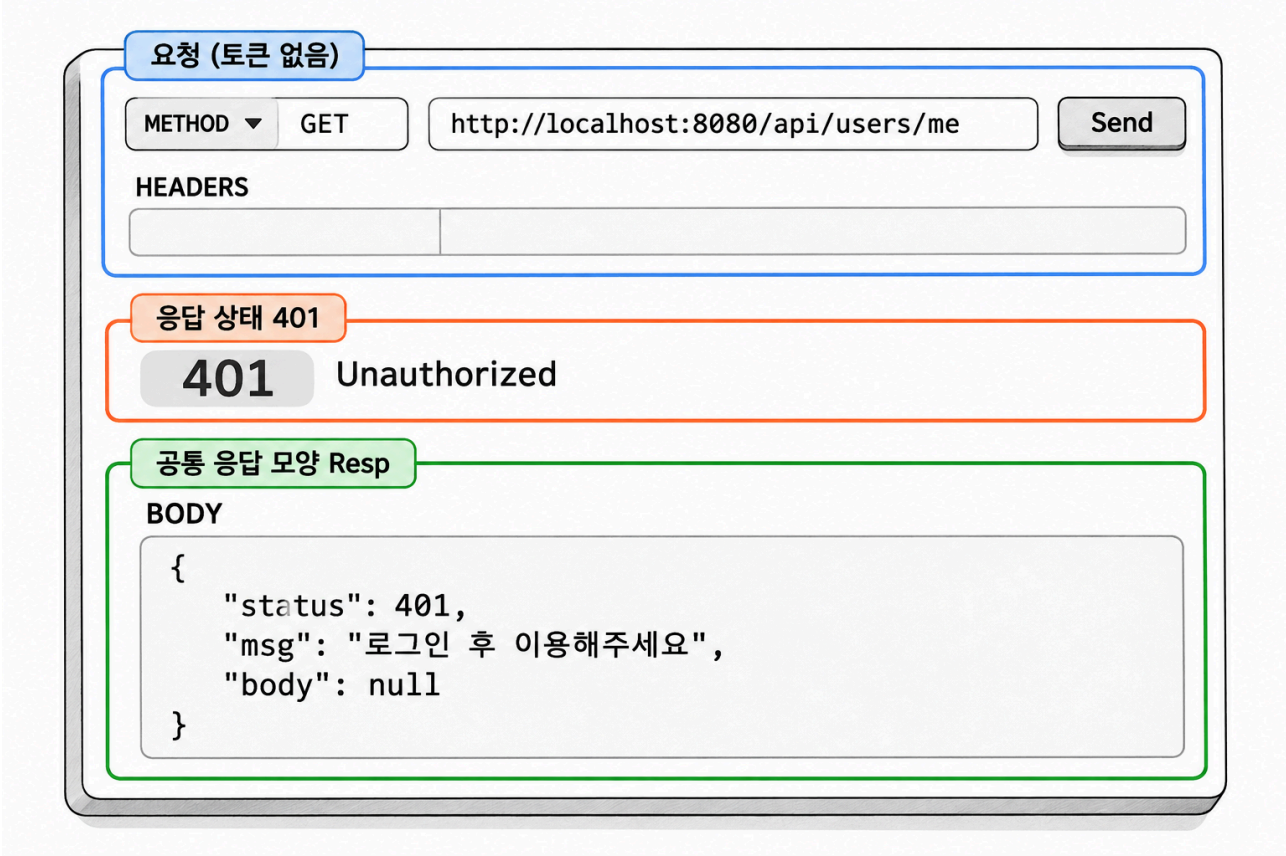
칸	값
METHOD	`GET`
URL	`http://localhost:8080/api/users/me`
HEADERS	없음

실행 결과

상태 코드는 `401`이고, 본문에는 다음 JSON이 옵니다.

JSON

```
{
  "status": 401,
  "msg": "로그인 후 이용해주세요",
  "body": null
}
```



Talend API Tester에서 토큰 없이 GET http://localhost:8080/api/users/me를 보낸 화면

생성형 예시 · 실제 실행 안 함

`/api/users/me`를 처리하는 컨트롤러는 8장에서 만듭니다. 그런데도 404가 아니라 401이 왔습니다. 요청이 컨트롤러를 찾기 전에 시큐리티 필터에서 막혔기 때문입니다. 요청은 다음 순서로 처리되었습니다.

1. CORS 필터가 요청의 `Origin` 헤더를 허용 목록과 비교합니다. Talend의 `chrome-extension://` 출처는 2.3에서 허용했으므로 통과합니다.
2. `JwtAuthorizationFilter`가 Authorization 헤더에서 토큰을 찾지 못하고, `SecurityContextHolder`를 비워 둔 채 다음 필터로 넘깁니다.

3. `AuthorizationFilter`가 이 요청이 `/api/**` 규칙(USER 또는 ADMIN)에 맞는데 인증 정보가 없다고 판단합니다.
4. `authenticationEntryPoint`가 `RespFilter.fail()`을 호출해 401 JSON을 응답에 씁니다.

결과가 다르면 다음 순서로 원인을 찾습니다. `Invalid CORS request`라는 글자와 함께 403이 오면 2.3의 허용 출처 설정을 다시 봅니다. 콤마 뒤 공백과 오타가 흔한 원인입니다. Resp 모양이 아닌 403이 오면 4.3의 `exceptionHandling()` 설정이 빠지지 않았는지 확인합니다.

7장. 회원 API 만들기

07

회원 API의 요청은 컨트롤러, 서비스, 리포지토리 순서로 지나갑니다. 컨트롤러는 요청을 받아 응답을 돌려줍니다. 서비스는 가입과 로그인 규칙을 처리하고, 리포지토리는 DB에 접근합니다. 이 장에서는 DB에 가까운 리포지토리부터 만들어 컨트롤러까지 올라갑니다.

계층	클래스	하는 일
컨트롤러	`AuthController`	요청 받기, `Resp` 로 응답하기
서비스	`UserService`	중복 확인, 비밀번호 암호화, 토큰 발급
리포지토리	`UserRepository`	`user_tb` 조회 · 저장

6장에서 `/join` , `/login` , `/check-username` 을 `permitAll()` 로 열어 두었습니다. 그래서 이 장의 API는 토큰 없이 요청할 수 있습니다.

---

1. UserRepository

`UserRepository` 는 `User` 엔티티를 저장하고 조회하는 인터페이스입니다. `JpaRepository` 를 상속하면 `save()` , `findById()` 같은 기본 메서드가 따라옵니다. 여기에 아이디로 회원을 찾는 메서드와 아이디가 있는지 확인하는 메서드를 더합니다.

예제 7-1 — UserRepository 만들기

이어서: 예제 6-7

파일명: `src/main/java/com/newtenco/blog/user/UserRepository.java`

Java

```java
package com.newtenco.blog.user;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Integer> {

    // select ... from user_tb where username = ?
    Optional<User> findByUsername(String username);

    // username 이 있으면 true, 없으면 false
    boolean existsByUsername(String username);
}
```

두 메서드에는 구현 코드가 없습니다. 스프링 데이터 JPA가 `findBy`, `existsBy` 뒤에 붙은 필드 이름을 읽고 쿼리를 만들어 주기 때문입니다. 이 쿼리는 서버가 시작될 때 만들어집니다. 그래서 `findByUsernam`처럼 필드 이름에 오타가 있으면 서버가 아예 시작되지 않습니다.

메서드	결과
`findByUsername("ssar")`	회원이 있으면 `User`가 담긴 `Optional`, 없으면 빈 `Optional`
`existsByUsername("ssar")`	회원이 있으면 `true`, 없으면 `false`

`findByUsername()`은 `Optional`을 돌려줍니다. 그래서 서비스 코드를 작성할 때 회원이 없는 경우를 빠뜨리기 어렵습니다. 5절의 로그인이 바로 이 경우를 처리합니다.

---

2. 요청 · 응답 DTO

2.1 record로 DTO 만들기

DTO(Data Transfer Object)는 계층 사이에서 데이터를 옮기는 객체입니다. 클라이언트가 보낸 JSON은 요청 DTO에 담기고, 응답 DTO는 JSON으로 바뀌어 클라이언트에 나갑니다. 엔티티를 그대로 주고받지 않는 이유는 2.3에서 설명합니다.

DTO는 4장의 `Resp`처럼 **record**로 만듭니다. 값은 `dto.username()`처럼 필드 이름과 같은 접근자로 꺼냅니다. record의 필드는 모두 `final`이라 한 번 만든 DTO의 값은 바뀌지 않습니다. 요청 값을 계

층 사이로 옮기기만 하는 DTO에는 이 성질이 잘 맞습니다.

이 프로젝트는 요청 DTO를 `AuthRequest` 클래스 안에, 응답 DTO를 `AuthResponse` 클래스 안에 모읍니다. 쓸 때는 `AuthRequest.JoinDTO`처럼 바깥 클래스 이름을 붙입니다. 이름만 봐도 요청용인지 응답용인지 알 수 있습니다.

2.2 JoinDTO 검증 어노테이션

요청 DTO의 필드에 검증 어노테이션을 붙이면 컨트롤러의 `@Valid`가 그 규칙으로 값을 검사합니다. 검사에 실패하면 4장의 전역 예외 처리기가 400 응답을 만듭니다.

예제 7-2 — 요청 DTO 만들기

이어서: 예제 7-1

파일명: `src/main/java/com/newtenco/blog/auth/AuthRequest.java`

Java

```java
package com.newtenco.blog.auth;

import com.newtenco.blog.user.User;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

import java.util.List;

public class AuthRequest {

    public record JoinDTO(
            @NotBlank(message = "유저네임을 입력해주세요")
            @Size(min = 4, max = 20, message = "유저네임은 4~20자여야 합니다")
            String username,

            @NotBlank(message = "비밀번호를 입력해주세요")
            @Size(min = 4, max = 20, message = "비밀번호는 4~20자여야 합니다")
            String password,

            @NotBlank(message = "이메일을 입력해주세요")
            @Email(message = "이메일 형식이 올바르지 않습니다")
            @Size(max = 30, message = "이메일은 30자 이하여야 합니다")
            String email) {

        // DTO -> Entity 변환 (비밀번호는 암호화된 값을 받는다)
        public User toEntity(String encPassword) {
            return User.builder()
                    .username(username)
                    .password(encPassword)
                    .email(email)
                    .roles(List.of("USER"))
                    .build();
        }
    }

    public record LoginDTO(
            @NotBlank(message = "유저네임을 입력해주세요") String username,
            @NotBlank(message = "비밀번호를 입력해주세요") String password) {
    }
}
```

검증 어노테이션	실패하는 값
`@NotBlank`	`null`, 빈 문자열 `""`, 공백만 있는 문자열 `" "`
`@Size(min, max)`	길이가 `min`보다 짧거나 `max`보다 긴 문자열(`min`을 생략하면 최대 길이만 검사)
`@Email`	이메일 형식이 아닌 문자열

`username`의 `@Size(max = 20)`과 `email`의 `@Size(max = 30)`은 2장 `User` 엔티티의 컬럼 길이 20, 30에 맞춘 값입니다. 이 검증이 없으면 21자 아이디나 31자 이메일은 DB에 저장할 때 오류가 납니다. 이 오류는 전역 예외 처리기를 거쳐 500 응답이 됩니다. 클라이언트가 잘못 보낸 값이므로 컨트롤러 입구에서 400으로 막는 편이 맞습니다.

DTO의 `password`는 사용자가 입력한 원래 비밀번호입니다. 이 값이 엔티티에 그대로 들어가면 안 되므로 `toEntity()`는 암호화한 비밀번호를 매개변수로 따로 받습니다. 원래 비밀번호를 엔티티에 넣는 실수를 메서드의 매개변수 구성으로 막는 셈입니다. 가입한 회원은 모두 `USER` 역할을 받고, `ADMIN`은 더미 데이터의 `cos`만 가집니다.

`LoginDTO`는 5절의 로그인에서 씁니다. 로그인할 때는 값이 비어 있는지만 검사합니다. 길이 규칙을 응답으로 알려 주면 공격자에게 힌트가 됩니다. 또 로그인 성공 여부는 어차피 DB에 저장된 값과 비교해야 정해집니다.

2.3 응답 DTO에서 비밀번호 빼기

컨트롤러가 `User` 엔티티를 그대로 돌려주면 스프링이 JSON 변환에 쓰는 Jackson은 getter가 있는 필드를 모두 JSON으로 바꿉니다. 이때 `password` 필드의 암호화된 비밀번호도 응답에 실립니다. 암호화된 값이라도 밖에 나가면 오프라인 대입 공격의 재료가 됩니다. 공격자가 해시를 가져가 자기 컴퓨터에서 비밀번호 후보를 하나씩 넣어 볼 수 있기 때문입니다. 그래서 응답에는 클라이언트에 필요한 필드만 골라 담은 DTO를 씁니다.

예제 7-3 — 응답 DTO 만들기

이어서: 예제 7-2

파일명: `src/main/java/com/newtenco/blog/auth/AuthResponse.java`

Java

```java
package com.newtenco.blog.auth;

import com.newtenco.blog.user.User;

import java.util.List;

public class AuthResponse {

    // 회원 정보 응답 (비밀번호는 넣지 않는다)
    public record DTO(Integer id, String username, String email, List<String> roles) {
        public DTO(User user) {
            this(user.getId(), user.getUsername(), user.getEmail(),
List.copyOf(user.getRoles()));
        }
    }

    // 로그인 성공 시 토큰 응답
    public record TokenDTO(String accessToken) {
    }

    // 유저네임 사용 가능 여부 응답
    public record AvailableDTO(boolean available) {
    }
}
```

`DTO`의 필드 목록에는 `password`가 없습니다. 필드가 없으니 응답 JSON에도 비밀번호가 실리지 않습니다. `User`를 받는 생성자는 엔티티에서 필요한 값만 꺼내 record의 표준 생성자 `this(...)`에 넘깁니다.

권한 목록은 `List.copyOf()`로 복사해 담았습니다. `user.getRoles()`는 JPA가 관리하는 엔티티의 컬렉션입니다. 이 컬렉션을 그대로 담으면 트랜잭션이 끝난 뒤 JSON으로 바꿀 때 JPA가 관리하는 컬렉션에 접근하게 됩니다. 복사해 두면 응답 DTO는 엔티티와 상관없는 일반 목록을 갖습니다.

`TokenDTO`는 5절의 로그인에서, `AvailableDTO`는 6절의 중복체크에서 씁니다.

---

3. BCrypt 비밀번호 암호화

3.1 PasswordEncoder와 BCrypt

PasswordEncoder는 스프링 시큐리티가 비밀번호를 암호화하고 비교할 때 쓰는 인터페이스입니다. 6장의 `SecurityConfig`에서 이 빈을 미리 등록해 두었습니다.

코드 조각

Java

```java
// SecurityConfig 안 (6장에서 등록)
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

반환 타입은 인터페이스 `PasswordEncoder`이고, 실제 객체는 BCrypt로 구현한 `BCryptPasswordEncoder`입니다. 서비스는 인터페이스 타입으로 주입받습니다. 그래서 나중에 암호화 방식을 바꾸더라도 이 메서드 한 곳만 고치면 됩니다.

BCrypt는 비밀번호를 저장하려고 만든 해시 함수입니다. **해시(hash)**는 입력값을 정해진 규칙으로 바꾼 결과입니다. 해시 결과에서 원래 값을 거꾸로 알아낼 수는 없습니다. 그래서 DB가 유출되어도 원래 비밀번호가 바로 드러나지 않습니다. BCrypt는 계산을 일부러 느리게 만듭니다. 공격자가 비밀번호 후보를 하나씩 대입해 볼 때 시간이 오래 걸리게 하려는 설계입니다.

`PasswordEncoder`에서 이 장이 쓰는 메서드는 두 개입니다.

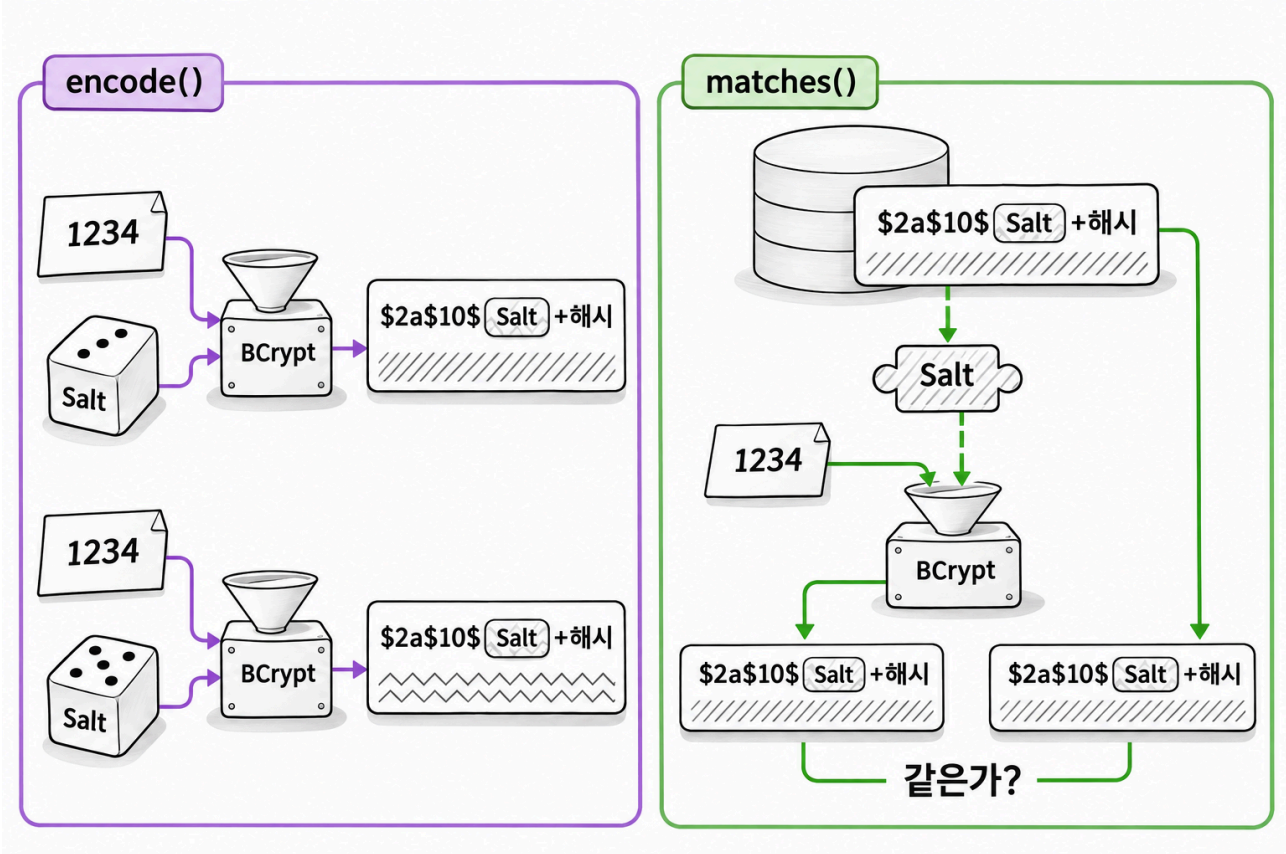
메서드	쓰는 곳	하는 일
`encode(원래비밀번호)`	회원가입	비밀번호를 BCrypt 해시로 바꿉니다
`matches(원래비밀번호, 저장된해시)`	로그인	입력한 비밀번호가 저장된 해시와 맞는지 확인합니다

3.2 Salt와 encode() · matches()

BCrypt는 해시를 만들 때마다 무작위 값인 **Salt**를 섞습니다. 그래서 같은 `"1234"`를 두 번 `encode()`해도 결과가 매번 다릅니다. Salt가 없으면 같은 비밀번호를 쓰는 회원들의 해시가 똑같아집니다. 공격자는 흔한 비밀번호의 해시를 미리 계산한 표로 그 회원들의 비밀번호를 한꺼번에 알아낼 수 있습니다.

`encode("1234")`를 한 번 실행하면 `$2a$10$eiDus5jn3LsNyfjng4/HxukQh.7EZmn1ZNL1g4s3kkqPYpwY5DiuO` 같은 값이 나옵니다. BCrypt 해시의 길이는 항상 60자입니다. 2장에서 `password` 컬럼 길이를 60으로 잡은 것도 이 때문입니다. 이 문자열은 네 부분으로 나뉩니다.

부분	값	뜻
`$2a$`	`2a`	BCrypt 버전
`10$`	`10`	비용(cost). 2의 10제곱 번 반복해 계산합니다
다음 22자	`eiDus5jn3LsNyfjng4/Hxu`	Salt
나머지 31자	`kQh.7EZmn1ZNL1g4s3kkqPYpwY5DiuO`	해시 결과



encode()와 matches()의 차이

Salt는 해시 문자열 안에 함께 저장됩니다. 그림 7-1처럼 `matches()`는 저장된 해시에서 Salt와 비용을 꺼냅니다. 그 값으로 입력 비밀번호를 다시 해시한 뒤 저장된 해시와 비교합니다.

로그인할 때 `encode(입력값).equals(저장된해시)`로 비교하면 안 됩니다. `encode()`는 매번 새 Salt를 쓰므로 비밀번호가 맞아도 저장된 해시와 다른 값을 돌려줍니다.

2장 `data.sql`에 넣은 `ssar`과 `cos`의 비밀번호도 `"1234"`를 BCrypt로 해시한 값입니다. 5절에서 `1234`로 로그인하면 `matches()`가 `true`를 돌려줍니다.

4. 회원가입 API

4.1 UserService의 회원가입

회원가입 메서드는 아이디 중복을 확인하고, 비밀번호를 암호화해 저장한 뒤, 비밀번호를 뺀 응답 DTO를 돌려줍니다.

예제 7-4 – 회원가입 서비스 만들기

이어서: 예제 7-3

파일명: `src/main/java/com/newtenco/blog/user/UserService.java`

Java

```java
package com.newtenco.blog.user;

import com.newtenco.blog.auth.AuthRequest;
import com.newtenco.blog.auth.AuthResponse;
import com.newtenco.blog.core.handler.errors.BadRequestException;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Transactional(readOnly = true)
@RequiredArgsConstructor
@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public AuthResponse.DTO 회원가입(AuthRequest.JoinDTO reqDTO) {
        if (userRepository.existsByUsername(reqDTO.username())) {
            throw new BadRequestException("이미 존재하는 유저네임입니다");
        }
        String encPassword = passwordEncoder.encode(reqDTO.password());
        User savedUser = userRepository.save(reqDTO.toEntity(encPassword));
        return new AuthResponse.DTO(savedUser);
    }
}
```

순서	코드	하는 일
1	`existsByUsername()`	아이디가 이미 있으면 `BadRequestException` (400)을 던집니다
2	`passwordEncoder.encode()`	비밀번호를 BCrypt 해시로 바꿉니다
3	`reqDTO.toEntity(encPassword)`	해시한 비밀번호로 `User` 엔티티를 만듭니다
4	`userRepository.save()`	`user_tb` 와 `user_role_tb` 에 저장합니다
5	`new AuthResponse.DTO(savedUser)`	비밀번호를 뺀 응답 DTO로 바꿉니다

1번의 `BadRequestException` 은 4장의 전역 예외 처리기가 받아 400 `Resp` 응답으로 바꿉니다. 서비스는 응답 모양을 신경 쓰지 않고 예외만 던집니다.

클래스에는 `@Transactional(readOnly = true)` 를 붙였습니다. 데이터를 바꾸는 `회원가입()` 에만 `@Transactional` 을 다시 붙였습니다. 메서드에 붙인 설정이 클래스 설정보다 우선합니다. 앞으로 추가할 조회 메서드는 클래스 설정대로 읽기 전용 트랜잭션에서 실행됩니다. 읽기 전용 트랜잭션에서는 JPA가 변경 감지(엔티티 값이 바뀌었는지 확인하는 작업)를 건너뛰므로 조금 가볍게 동작합니다.

스프링은 `@RequiredArgsConstructor` 가 만든 생성자로 `PasswordEncoder` 를 주입합니다. 3.1에서 본 `SecurityConfig` 의 빈이 이 자리에 들어옵니다.

4.2 AuthController의 /join

`AuthController` 는 JSON 요청을 `JoinDTO` 로 받아 검증하고, 서비스가 돌려준 결과를 `Resp.ok()` 로 감싸 응답합니다.

예제 7-5 — 회원가입 컨트롤러 만들기

이어서: 예제 7-4

파일명: `src/main/java/com/newtenco/blog/auth/AuthController.java`

Java

```java
package com.newtenco.blog.auth;

import com.newtenco.blog.core.util.Resp;
import com.newtenco.blog.user.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RequiredArgsConstructor
@RestController
public class AuthController {

    private final UserService userService;

    @PostMapping("/join")
    public ResponseEntity<?> join(@Valid @RequestBody AuthRequest.JoinDTO reqDTO) {
        return Resp.ok(userService.회원가입(reqDTO));
    }
}
```

`@RequestBody`는 요청 본문의 JSON을 `JoinDTO`로 바꿉니다. JSON의 키 이름과 record의 필드 이름이 같아야 값이 들어갑니다. `@Valid`는 만들어진 DTO를 검증 어노테이션의 규칙대로 검사합니다. 검사에 실패하면 컨트롤러 메서드는 실행되지 않고 `MethodArgumentNotValidException`이 발생합니다. 4장의 전역 예외 처리기가 이 예외를 `"필드명:메시지"` 모양의 400 응답으로 바꿉니다.

주소는 일부러 `/api/`로 시작하지 않게 했습니다. 6장의 접근 규칙에서 `/api/**`는 인증이 필요한 주소로 묶었습니다. 회원가입, 로그인, 중복체크는 아직 토큰이 없는 사람이 쓰는 기능이라 `/api/**` 바깥에 둡니다.

4.3 Talend API Tester로 회원가입 요청하기

서버를 실행하고 크롬에서 Talend API Tester를 엽니다. 요청 칸에는 다음 값을 넣습니다.

칸	값
METHOD	`POST`
URL	`http://localhost:8080/join`

칸	값
HEADERS	`Content-Type` : `application/json`
BODY	아래 JSON

코드 조각

JSON

```
{"username":"love","password":"1234","email":"love@nate.com"}
```

`Content-Type` 헤더가 빠지면 스프링은 본문을 JSON으로 읽지 않습니다. 이 경우 `@RequestBody` 변환에 실패해 415 응답이 돌아옵니다.

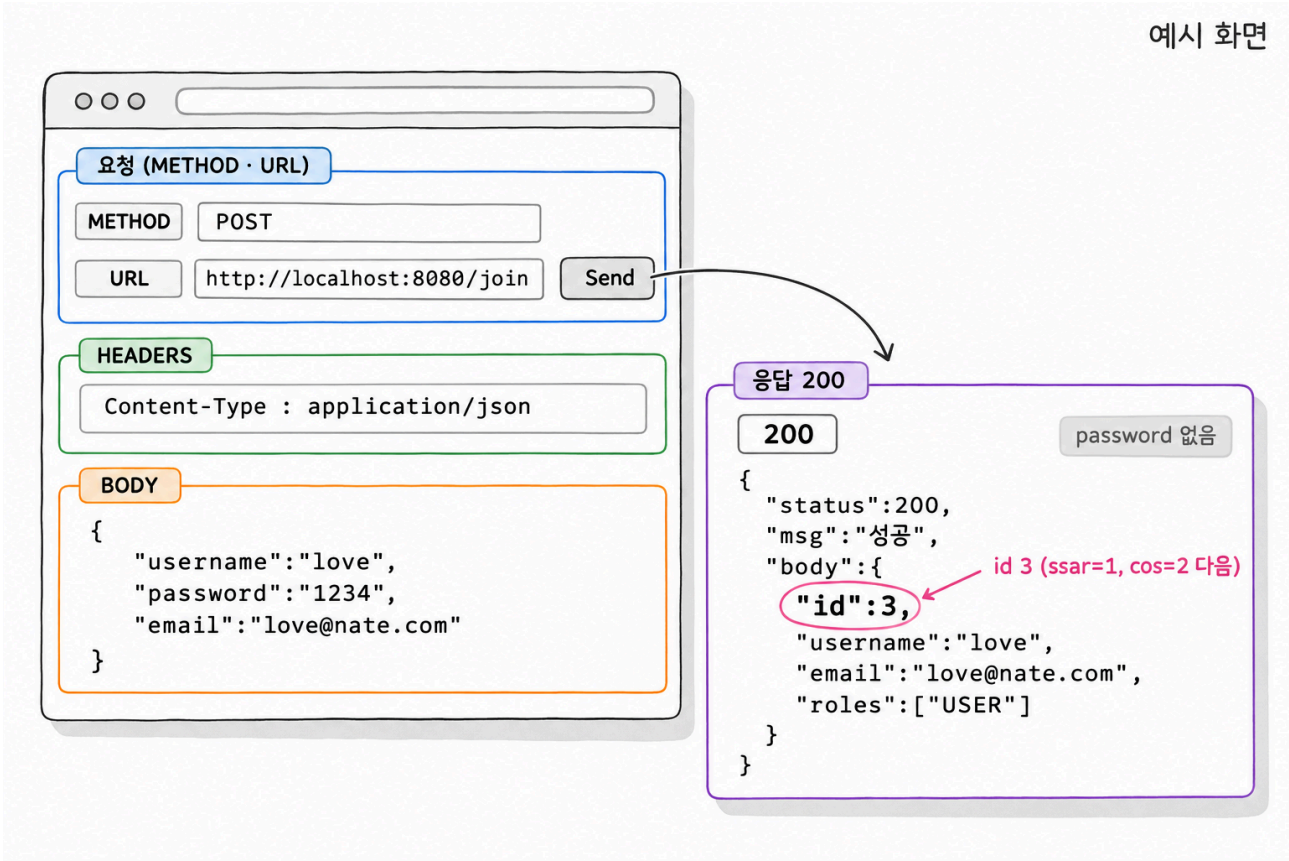
실행 결과

Send를 누르면 응답 상태 `200`과 다음 본문이 표시됩니다.

JSON

```
{"status":200,"msg":"성공","body":{"id":3,"username":"love","email":"love@nate.com","roles":["USER"]}}
```

`data.sql`의 `ssar`과 `cos`가 id 1, 2를 쓰고 있어서 새 회원은 id 3을 받습니다. 응답 본문에 `password`가 없다는 점도 확인합니다.



Talend API Tester에서 POST http://localhost:8080/join 요청을 보낸 화면

생성형 예시 · 실제 실행 안 함

같은 요청을 한 번 더 보내거나 값을 바꿔 보내면 실패 응답을 볼 수 있습니다.

요청 본문	응답 상태	응답 본문
love 회원을 다시 보냄	400	`{"status":400,"msg":"이미 존재하는 유저네임입니다","body":null}`
`"username":"ab"`	400	`{"status":400,"msg":"username:유저네임은 4~20자여야 합니다","body":null}`
`"email":"love"`	400	`{"status":400,"msg":"email:이메일 형식이 올바르지 않습니다","body":null}`

첫 번째 행은 서비스가 던진 `BadRequestException`에서 나온 응답입니다. 나머지 두 행은 `@Valid` 검증에 실패해 나온 응답입니다. 원인은 달라도 클라이언트는 같은 `Resp` 모양으로 받습니다.

H2 콘솔에서 `SELECT * FROM user_tb;` 를 실행하면 love 회원의 `password` 칸에서 `$2a$10$` 으로 시작하는 60자 해시를 볼 수 있습니다. H2가 메모리 모드라서 서버를 다시 시작하면 love 회원은 사라집니다.

---

5. 로그인 API와 JWT 발급

5.1 아이디 없음과 비밀번호 틀림을 같은 401로 응답하기

로그인 메서드는 아이디로 회원을 찾고 `matches()` 로 비밀번호를 비교합니다. 둘 다 통과하면 3장의 `JwtUtil.create()` 로 토큰을 만들어 돌려줍니다.

예제 7-6 — 로그인 API 만들기

이어서: 예제 7-5

파일명: `src/main/java/com/newtenco/blog/user/UserService.java`

Java

```java
package com.newtenco.blog.user;

import com.newtenco.blog.auth.AuthRequest;
import com.newtenco.blog.auth.AuthResponse;
import com.newtenco.blog.core.handler.errors.BadRequestException;
import com.newtenco.blog.core.handler.errors.UnauthorizedException;
import com.newtenco.blog.core.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Transactional(readOnly = true)
@RequiredArgsConstructor
@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public AuthResponse.DTO 회원가입(AuthRequest.JoinDTO reqDTO) {
        if (userRepository.existsByUsername(reqDTO.username())) {
            throw new BadRequestException("이미 존재하는 유저네임입니다");
        }
        String encPassword = passwordEncoder.encode(reqDTO.password());
        User savedUser = userRepository.save(reqDTO.toEntity(encPassword));
        return new AuthResponse.DTO(savedUser);
    }

    public AuthResponse.TokenDTO 로그인(AuthRequest.LoginDTO reqDTO) {
        // 아이디가 틀렸는지 비밀번호가 틀렸는지 알려주지 않는다
        User user = userRepository.findByUsername(reqDTO.username())
                .orElseThrow(() -> new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니
다"));

        if (!passwordEncoder.matches(reqDTO.password(), user.getPassword())) {
            throw new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니다");
        }
        return new AuthResponse.TokenDTO(JwtUtil.create(user));
    }
}
```

파일명: `src/main/java/com/newtenco/blog/auth/AuthController.java`

Java

```java
package com.newtenco.blog.auth;

import com.newtenco.blog.core.util.Resp;
import com.newtenco.blog.user.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RequiredArgsConstructor
@RestController
public class AuthController {

    private final UserService userService;

    @PostMapping("/join")
    public ResponseEntity<?> join(@Valid @RequestBody AuthRequest.JoinDTO reqDTO) {
        return Resp.ok(userService.회원가입(reqDTO));
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody AuthRequest.LoginDTO reqDTO) {
        return Resp.ok(userService.로그인(reqDTO));
    }
}
```

로그인은 두 곳에서 실패합니다. 아이디가 없으면 `orElseThrow()`에서, 비밀번호가 틀리면 `matches()` 검사에서 예외를 던집니다. 두 곳 모두 같은 `UnauthorizedException`에 같은 메시지를 씁니다.

메시지를 "없는 아이디입니다"와 "비밀번호가 틀렸습니다"로 나누면 사용자는 편합니다. 하지만 그 정보는 공격자에게도 그대로 전달됩니다. 공격자는 아무 아이디나 넣어 보며 가입된 아이디를 골라냅니다. 그다음 찾은 아이디에만 비밀번호를 대입하면 됩니다. 응답을 하나로 맞추면 공격자는 무엇이 틀렸는지 알 수 없습니다.

상태 코드는 401입니다. 로그인 실패는 누구인지 확인하지 못한 상황이라 5장에서 본 인증 실패에 해당합니다. 403은 누구인지는 알지만 권한이 없을 때 씁니다.

`로그인()`에는 `@Transactional`을 따로 붙이지 않았습니다. 회원을 조회하기만 하므로 클래스의 읽기 전용 트랜잭션으로 충분합니다.

`JwtUtil.create()` 는 앞에 `"Bearer "` 를 붙인 문자열을 돌려줍니다. 클라이언트는 받은 값을 그대로 `Authorization` 헤더에 넣으면 됩니다.

5.2 Talend API Tester로 토큰 받기

회원가입과 같은 방법으로 요청하고 주소와 본문만 바꿉니다.

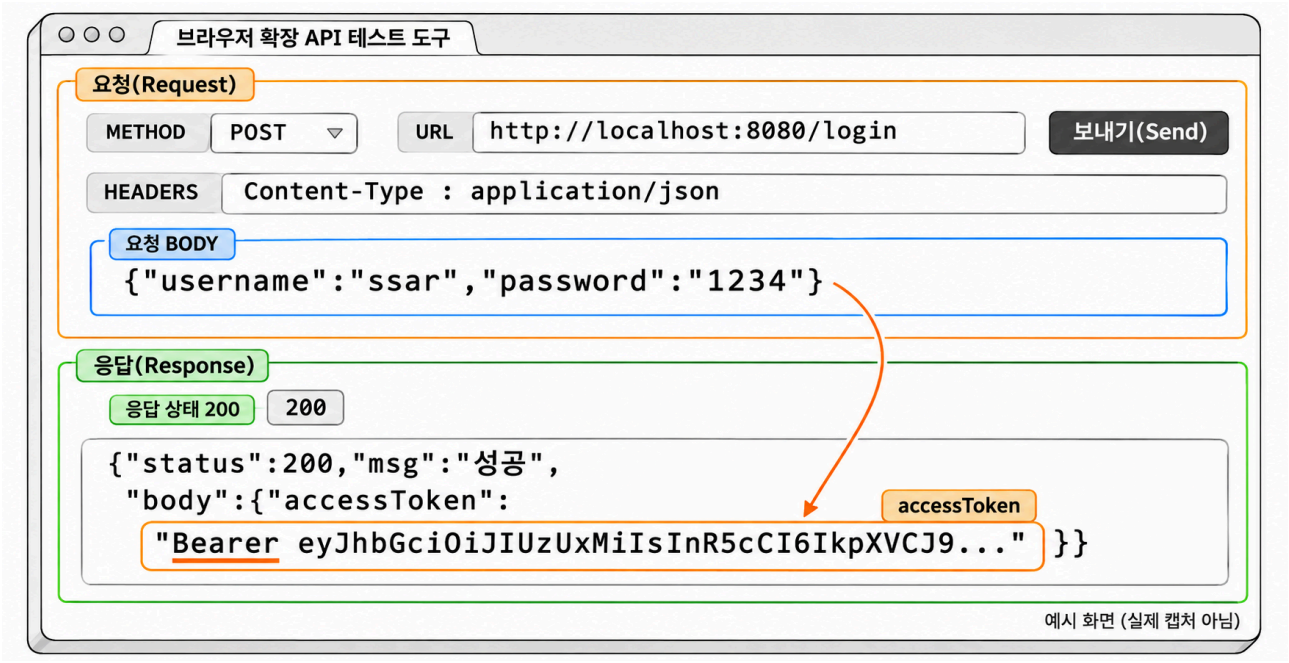
칸	값
METHOD	`POST`
URL	`http://localhost:8080/login`
HEADERS	`Content-Type` : `application/json`
BODY	`{"username":"ssar","password":"1234"}`

실행 결과

응답 상태 `200` 과 함께 토큰이 담긴 본문이 표시됩니다. 토큰에 만료 시각이 들어가므로 토큰 값은 로그인할 때마다 달라집니다.

JSON


```
{"status":200,"msg":"성공","body":{"accessToken":"Bearer
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9..."}}
```



Talend API Tester에서 POST http://localhost:8080/login 요청을 보낸 화면

생성형 예시 · 실제 실행 안 함

`accessToken` 값 전체를 `"Bearer "`까지 포함해 메모장에 복사해 둡니다. 8장에서 이 값을 `Authorization` 헤더에 넣어 요청합니다.

비밀번호를 `"5678"`로 바꾸거나 아이디를 `"nobody"`로 바꿔 보내면 두 경우 모두 같은 응답이 돌아옵니다.

실행 결과

JSON

```
{"status":401,"msg":"유저네임 혹은 비밀번호가 일치하지 않습니다","body":null}
```

두 응답은 한 글자도 다르지 않습니다. 클라이언트는 아이디와 비밀번호 가운데 어느 쪽이 틀렸는지 알 수 없습니다.

5.3 jwt.io에서 페이로드 확인하기

jwt.io는 JWT를 붙여 넣으면 헤더와 페이로드를 풀어 보여 주는 사이트입니다. 3장에서 페이로드는 Base64Url로 인코딩만 한 값이라 누구나 읽을 수 있다고 했습니다. 방금 받은 토큰으로 이 점을 직접 확인합니다.

복사한 값에서 앞의 `Bearer `(공백 포함 7글자)를 지우고 `eyJ`로 시작하는 부분만 붙여 넣습니다. `Bearer `는 HTTP 헤더에서 토큰 종류를 알리는 접두사이고, JWT의 일부가 아닙니다. 접두사가 남아

있으면 jwt.io가 헤더 부분을 풀지 못합니다.

실행 결과

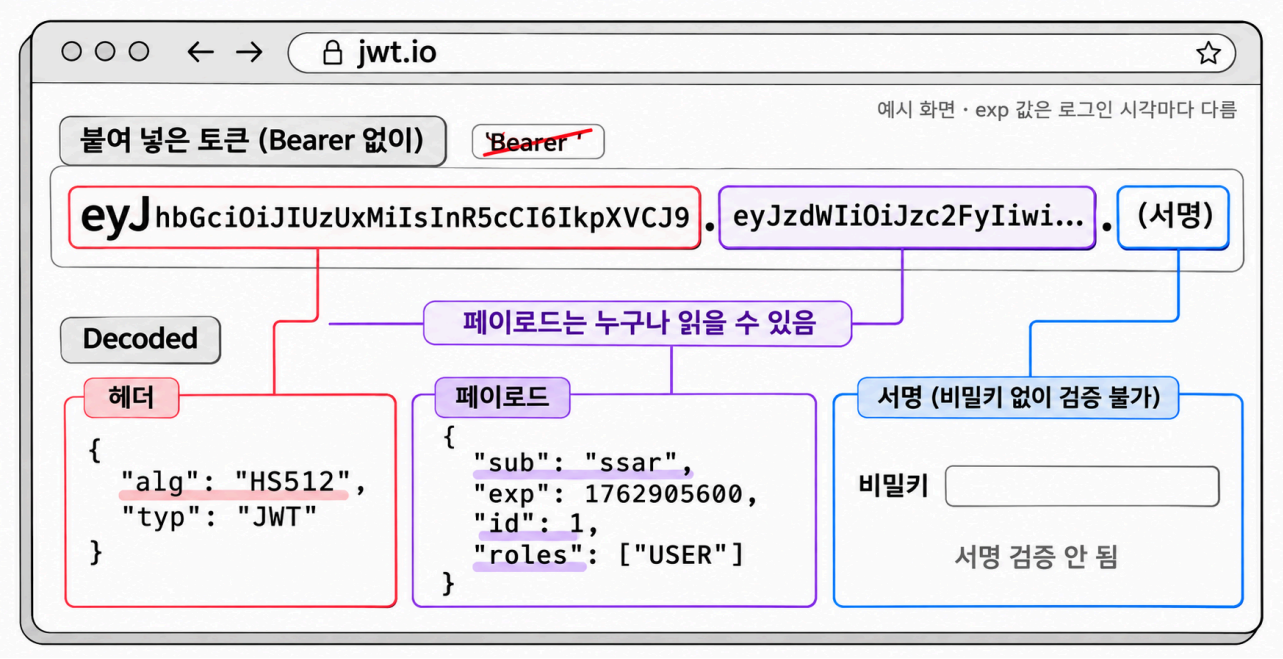
헤더와 페이로드 칸에 다음 내용이 표시됩니다. `exp` 값은 로그인한 시각에 따라 다릅니다.

JSON

```
{
  "alg": "HS512",
  "typ": "JWT"
}
```

JSON

```
{
  "sub": "ssar",
  "exp": 1762905600,
  "id": 1,
  "roles": ["USER"]
}
```



jwt.io에 ssar의 토큰을 Bearer 없이 붙여 넣은 브라우저 화면

생성형 예시 · 실제 실행 안 함

`JwtUtil.create()`에서 넣은 `sub`, `exp`, `id`, `roles`가 그대로 보입니다. 3장 코드에서 넣지 않은 `iat`는 없습니다. `exp`는 1970년 1월 1일부터 센 초 단위 시각입니다. 비밀키 없이도 누구나 이 내용을 읽

을 수 있으므로 페이로드에는 비밀번호 같은 민감한 정보를 넣지 않습니다.

jwt.io에 비밀키를 넣지 않았으므로 jwt.io는 서명이 올바른지 검증하지 못합니다. 토큰을 읽는 일은 누구나 할 수 있습니다. 그러나 내용을 고친 토큰에 맞는 서명은 서버의 비밀키 없이 만들 수 없습니다. 위조한 토큰을 보내면 401 응답이 돌아오는데, 이 결과는 8장에서 확인합니다.

---

6. 유저네임 중복체크 API

6.1 쿼리스트링으로 값 받기

중복체크 API는 사용자가 가입 버튼을 누르기 전에 아이디를 쓸 수 있는지 미리 알려 줍니다. 값을 조회하기만 하므로 GET 요청을 씁니다. GET 요청에는 본문을 쓰지 않습니다. 그래서 아이디는 주소 뒤에 붙는 **쿼리스트링(query string)** `?username=값`으로 보냅니다.

예제 7-7 — 유저네임 중복체크 API 만들기

이어서: 예제 7-6

파일명: `src/main/java/com/newtenco/blog/user/UserService.java`

Java

```java
package com.newtenco.blog.user;

import com.newtenco.blog.auth.AuthRequest;
import com.newtenco.blog.auth.AuthResponse;
import com.newtenco.blog.core.handler.errors.BadRequestException;
import com.newtenco.blog.core.handler.errors.UnauthorizedException;
import com.newtenco.blog.core.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Transactional(readOnly = true)
@RequiredArgsConstructor
@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public AuthResponse.DTO 회원가입(AuthRequest.JoinDTO reqDTO) {
        if (userRepository.existsByUsername(reqDTO.username())) {
            throw new BadRequestException("이미 존재하는 유저네임입니다");
        }
        String encPassword = passwordEncoder.encode(reqDTO.password());
        User savedUser = userRepository.save(reqDTO.toEntity(encPassword));
        return new AuthResponse.DTO(savedUser);
    }

    public AuthResponse.TokenDTO 로그인(AuthRequest.LoginDTO reqDTO) {
        // 아이디가 틀렸는지 비밀번호가 틀렸는지 알려주지 않는다
        User user = userRepository.findByUsername(reqDTO.username())
                .orElseThrow(() -> new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니
다"));

        if (!passwordEncoder.matches(reqDTO.password(), user.getPassword())) {
            throw new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니다");
        }
        return new AuthResponse.TokenDTO(JwtUtil.create(user));
    }

    public AuthResponse.AvailableDTO 유저네임중복체크(String username) {
        boolean available = !userRepository.existsByUsername(username);
        return new AuthResponse.AvailableDTO(available);
```

```
    }
}
```

파일명: `src/main/java/com/newtenco/blog/auth/AuthController.java`

Java

```java
package com.newtenco.blog.auth;

import com.newtenco.blog.core.util.Resp;
import com.newtenco.blog.user.UserService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RequiredArgsConstructor
@RestController
public class AuthController {

    private final UserService userService;

    @PostMapping("/join")
    public ResponseEntity<?> join(@Valid @RequestBody AuthRequest.JoinDTO reqDTO) {
        return Resp.ok(userService.회원가입(reqDTO));
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody AuthRequest.LoginDTO reqDTO) {
        return Resp.ok(userService.로그인(reqDTO));
    }

    // GET /check-username?username=ssar
    @GetMapping("/check-username")
    public ResponseEntity<?> checkUsername(@RequestParam("username") String username) {
        return Resp.ok(userService.유저네임중복체크(username));
    }
}
```

`@RequestParam("username")`은 쿼리스트링에서 `username` 키의 값을 꺼내 매개변수에 넣습니다.

`required` 속성의 기본값이 `true`라서 주소에 `?username=`이 아예 없으면

`MissingServletRequestParameterException`이 발생합니다. 4장의 전역 예외 처리기가 이 예외를 400 응답으로 바꿉니다.

서비스는 `existsByUsername()`의 결과를 뒤집어 돌려줍니다. 리포지토리는 아이디가 "있는가"를 묻지만, 클라이언트가 알고 싶은 것은 "쓸 수 있는가"이기 때문입니다.

DB에 아이디가	`existsByUsername()`	`available`
있음	`true`	`false` (사용 불가)
없음	`false`	`true` (사용 가능)

중복체크를 통과해도 가입이 보장되지는 않습니다. 사용자가 아이디를 확인한 뒤 가입 버튼을 누르기 전에, 다른 사람이 같은 아이디로 먼저 가입할 수 있습니다. 그래서 4.1의 `회원가입()`에도 중복 검사를 그대로 남겨 둡니다. 중복체크 API는 사용자에게 결과를 미리 알려 주는 용도입니다.

6.2 Talend API Tester로 확인하기

METHOD를 `GET`으로 바꾸고 주소에 쿼리스트링을 붙여 보냅니다. GET 요청이므로 HEADERS와 BODY는 비워 둡니다.

코드 조각

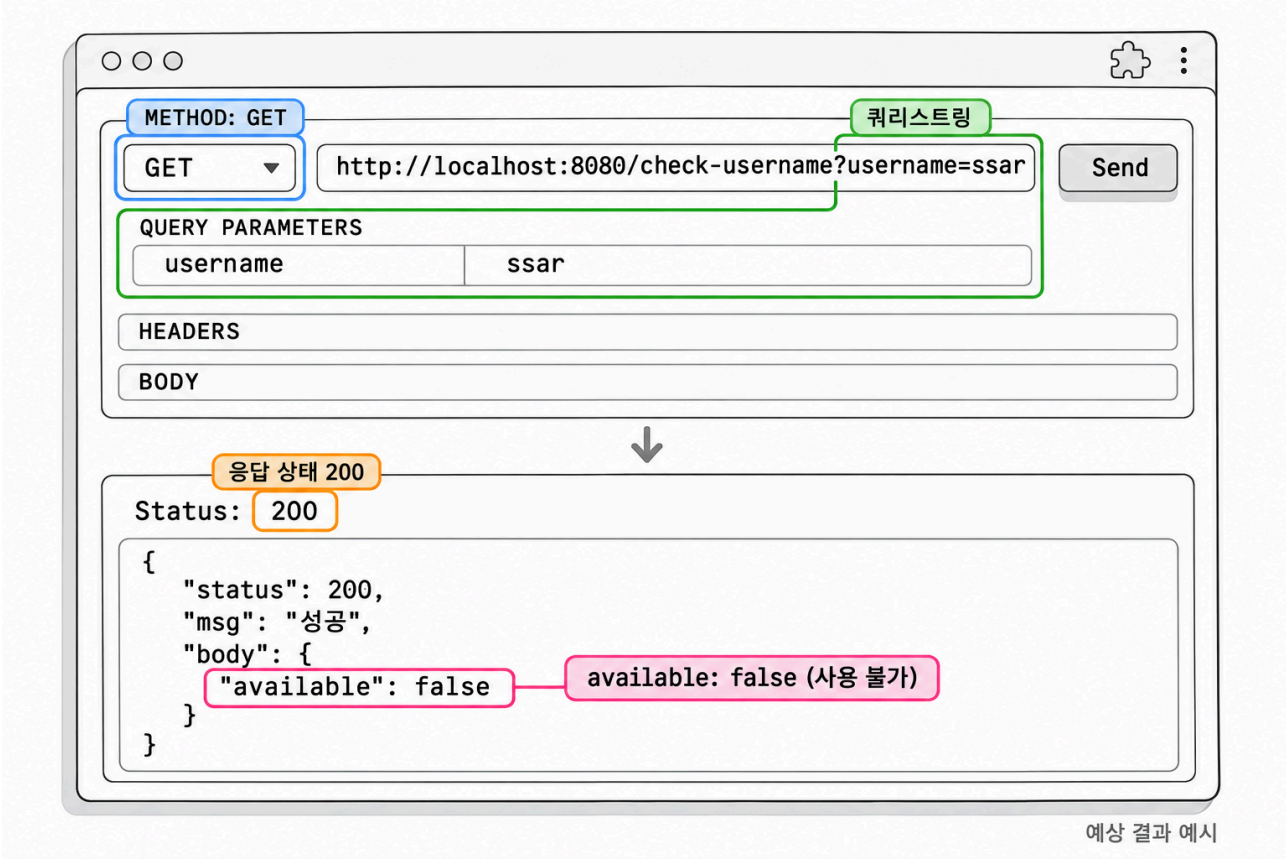
출력

```
http://localhost:8080/check-username?username=ssar
```

실행 결과

JSON

```
{"status":200,"msg":"성공","body":{"available":false}}
```



예상 결과 예시

Talend API Tester에서 GET http://localhost:8080/check-username?username=ssar 요청을

생성형 예시 · 실제 실행 안 함

쿼리스트링 값을 바꿔 보내면 응답이 다음과 같이 달라집니다.

요청 주소	응답 상태	응답 본문
`/check-username?username=ssar`	200	`{"status":200,"msg":"성공","body":{"available":false}}`
`/check-username?username=newuser`	200	`{"status":200,"msg":"성공","body":{"available":true}}`
`/check-username`	400	`{"status":400,"msg":"username:값을 입력해주세요","body":null}`

`ssar`은 `data.sql`로 넣어 둔 회원이라 쓸 수 없습니다. 4.3에서 가입한 `love`의 결과는 그 뒤에 서버를 다시 시작했는지에 따라 다릅니다. 다시 시작하지 않았다면 `false`가 나옵니다. 다시 시작했다면 메모리 DB에서 love 회원이 사라졌으므로 `true`가 나옵니다.

8장. JWT 인증 전체 흐름 확인하기

08

1. @AuthenticationPrincipal로 내 정보 조회

1.1 UserController와 /api/users/me

7장에서 만든 회원가입, 로그인, 중복체크는 모두 토큰 없이 부르는 공개 주소였습니다. 이 장에서는 토큰이 있어야만 들어갈 수 있는 API를 만듭니다. 그리고 토큰 발급, 토큰 검증, 컨트롤러에서 사용자 꺼내기까지 이어지는 흐름을 처음부터 끝까지 따라갑니다.

@AuthenticationPrincipal은 SecurityContextHolder에 저장된 Authentication 객체에서 principal을 꺼내 컨트롤러 매개변수에 넣어 주는 어노테이션입니다. principal은 인증된 사용자 객체를 말합니다. 5장의 JwtProvider는 토큰을 검증한 뒤 `UsernamePasswordAuthenticationToken(user, null, 권한 목록)`을 만들었습니다. JwtAuthorizationFilter는 이 객체를 SecurityContextHolder에 넣었습니다. 이 객체의 첫 번째 인자인 User가 principal입니다. 그래서 컨트롤러는 `@AuthenticationPrincipal User`라고만 써도 그 User를 받습니다.

기본 문법

Java

```java
@GetMapping("주소")
public ResponseEntity<?> 메서드명(@AuthenticationPrincipal 사용자클래스 변수명) {
    // 변수명으로 로그인한 사용자 정보를 쓴다
}
```

SSR 게시판에서는 `session.getAttribute("sessionUser")`로 로그인한 사용자를 꺼냈습니다. 지금은 이 어노테이션이 그 일을 맡습니다. 달라진 점은 사용자 정보를 세션 저장소가 아니라 이번 요청에 실려 온 토큰에서 얻는다는 것입니다. 변수 이름은 익숙하도록 SSR 게시판의 `sessionUser`를 그대로 씁니다.

예제 8-1 — 내 정보 조회 API 만들기

이어서: 예제 7-7

파일명: `src/main/java/com/newtenco/blog/user/UserService.java`

Java

```java
package com.newtenco.blog.user;

import com.newtenco.blog.auth.AuthRequest;
import com.newtenco.blog.auth.AuthResponse;
import com.newtenco.blog.core.handler.errors.BadRequestException;
import com.newtenco.blog.core.handler.errors.NotFoundException;
import com.newtenco.blog.core.handler.errors.UnauthorizedException;
import com.newtenco.blog.core.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Transactional(readOnly = true)
@RequiredArgsConstructor
@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    @Transactional
    public AuthResponse.DTO 회원가입(AuthRequest.JoinDTO reqDTO) {
        if (userRepository.existsByUsername(reqDTO.username())) {
            throw new BadRequestException("이미 존재하는 유저네임입니다");
        }
        String encPassword = passwordEncoder.encode(reqDTO.password());
        User savedUser = userRepository.save(reqDTO.toEntity(encPassword));
        return new AuthResponse.DTO(savedUser);
    }

    public AuthResponse.TokenDTO 로그인(AuthRequest.LoginDTO reqDTO) {
        // 아이디가 틀렸는지 비밀번호가 틀렸는지 알려주지 않는다
        User user = userRepository.findByUsername(reqDTO.username())
                .orElseThrow(() -> new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니
다"));

        if (!passwordEncoder.matches(reqDTO.password(), user.getPassword())) {
            throw new UnauthorizedException("유저네임 혹은 비밀번호가 일치하지 않습니다");
        }
        return new AuthResponse.TokenDTO(JwtUtil.create(user));
    }

    public AuthResponse.AvailableDTO 유저네임중복체크(String username) {
        boolean available = !userRepository.existsByUsername(username);
        return new AuthResponse.AvailableDTO(available);
```

```
    }

    // 토큰의 id로 DB를 다시 조회해 전체 회원 정보를 돌려준다
    public AuthResponse.DTO 내정보(Integer userId) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new NotFoundException("회원을 찾을 수 없습니다"));
        return new AuthResponse.DTO(user);
    }
}
```

파일명: `src/main/java/com/newtenco/blog/user/UserController.java`

Java

```
package com.newtenco.blog.user;

import com.newtenco.blog.core.util.Resp;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RequiredArgsConstructor
@RestController
public class UserController {

    private final UserService userService;

    // 로그인한 사용자만 접근 가능 (SecurityConfig: /api/** → USER 또는 ADMIN)
    @GetMapping("/api/users/me")
    public ResponseEntity<?> me(@AuthenticationPrincipal User sessionUser) {
        return Resp.ok(userService.내정보(sessionUser.getId()));
    }
}
```

실행 결과

서버를 다시 실행하면 오류 없이 뜹니다. 이 주소는 토큰이 있어야 들어갈 수 있으므로, 응답은 2절에서 Talend API Tester로 토큰을 넣어 확인합니다.

`me()`에는 로그인 여부를 검사하는 코드가 한 줄도 없습니다. 6장에서 `/api/**`에 `hasAnyRole("USER", "ADMIN")` 규칙을 걸어 두었기 때문입니다. 토큰이 없는 요청은 컨트롤러에 오기

전에 401로 끝납니다. 그러니 `me()`가 실행되었다면 `sessionUser`에는 반드시 토큰에서 복원한 User가 들어 있습니다.

`permitAll()`로 열어 둔 주소는 다릅니다. 토큰 없이 들어온 요청에서 `@AuthenticationPrincipal User`를 쓰면 `null`이 들어옵니다. 인증 정보가 없는 요청의 principal은 User 타입이 아니기 때문입니다. 공개 주소에서 사용자 정보를 쓸 때는 먼저 `null`인지 확인해야 합니다.

1.2 토큰에서 복원한 User와 DB 재조회

`sessionUser`는 DB에서 꺼낸 회원이 아닙니다. 3장의 `JwtUtil.verify()`가 토큰의 클레임만으로 새로 만든 User입니다. 토큰에 넣은 값은 `sub`(username), `id`, `roles` 세 가지뿐이므로, 복원한 User에도 이 세 값만 채워져 있습니다.

필드	토큰에서 복원한 User	DB에서 조회한 User
id	있음	있음
username	있음	있음
roles	있음	있음
email	`null`	있음
password	`null`	있음(BCrypt 값)

내 정보 화면에는 이메일도 보여야 합니다. 그래서 `내정보()`는 토큰의 id로 `findById()`를 다시 호출해 회원 정보 전체를 가져옵니다. 결과는 7장의 `AuthResponse.DTO`로 바꿔서 돌려주므로 비밀번호는 응답에 들어가지 않습니다.

DB를 다시 조회하면 Stateless의 장점이 사라지는 것처럼 보일 수 있습니다. 하지만 인증 자체는 여전히 DB 없이 토큰만으로 끝납니다. DB 조회는 토큰에 없는 정보가 필요한 API에서만 일어납니다. 토큰을 받은 뒤 회원이 탈퇴했다면 `findById()`가 회원을 찾지 못하고, 4장의 NotFoundException이 404 응답을 만듭니다.

토큰에 이메일까지 넣으면 DB 조회를 아낄 수 있을 것 같습니다. 그러나 3장에서 본 대로 페이로드는 누구나 디코딩해서 읽을 수 있습니다. 회원 정보가 바뀌어도 토큰은 만료될 때까지 예전 값을 그대로 담고 있다는 문제도 있습니다. 그래서 토큰에는 사용자를 알아보는 데 필요한 최소한의 값만 넣습니다.

1.3 관리자 전용 /api/admin/hello

역할에 따른 인가를 확인하려면 관리자만 들어갈 수 있는 API가 필요합니다. 6장 SecurityConfig에는 `/api/admin/**`에 대한 `hasRole("ADMIN")` 규칙이 이미 있으므로, 컨트롤러 메서드만 하나 더하면 됩니다.

예제 8-2 – 관리자 전용 API 추가하기

이어서: 예제 8-1

파일명: `src/main/java/com/newtenco/blog/user/UserController.java`

Java

```java
package com.newtenco.blog.user;

import com.newtenco.blog.core.util.Resp;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RequiredArgsConstructor
@RestController
public class UserController {

    private final UserService userService;

    // 로그인한 사용자만 접근 가능 (SecurityConfig: /api/** → USER 또는 ADMIN)
    @GetMapping("/api/users/me")
    public ResponseEntity<?> me(@AuthenticationPrincipal User sessionUser) {
        return Resp.ok(userService.내정보(sessionUser.getId()));
    }

    // ADMIN 역할만 접근 가능 (SecurityConfig: /api/admin/** → ADMIN)
    @GetMapping("/api/admin/hello")
    public ResponseEntity<?> adminHello(@AuthenticationPrincipal User sessionUser) {
        return Resp.ok("관리자 " + sessionUser.getUsername() + "님 환영합니다");
    }
}
```

실행 결과

서버가 오류 없이 다시 실행됩니다. 응답은 2.3에서 USER 토큰으로, 2.4에서 ADMIN 토큰으로 나누어 확인합니다.

`adminHello()` 는 DB를 조회하지 않습니다. 이 메서드에 필요한 값은 username 하나이고, username은 토큰에 들어 있습니다. 1.2에서 본 `me()` 와 견주어 보면 기준이 분명해집니다. 토큰에 없는 값이 필요할 때만 DB를 다시 조회합니다.

관리자인지도 컨트롤러가 검사하지 않습니다. 토큰의 `roles` 가 `["USER","ADMIN"]` 이면 2장의 `getAuthorities()` 가 `ROLE_USER` 와 `ROLE_ADMIN` 을 돌려줍니다. 시큐리티는 `hasRole("ADMIN")` 규칙에서 `ROLE_ADMIN` 권한이 있는지를 봅니다. 2장 data.sql 기준으로 ssar는 USER 역할만 있고, cos는 USER와 ADMIN 역할이 모두 있습니다.

---

2. Talend API Tester로 401 · 403 · 200 확인

요청이 들어와서 응답이 나가기까지 거치는 단계는 5장과 6장에서 설명한 그대로입니다. 다만 요청이 어느 단계에서 끝나는지는 토큰 상태와 역할에 따라 달라집니다.

JwtAuthorizationFilter는 토큰이 없거나 잘못되어도 요청을 막지 않고, 인증 정보를 넣지 않은 채로 다음 필터에 넘깁니다. 요청을 막는 필터는 6장의 URL 규칙을 검사하는 AuthorizationFilter입니다. 규칙을 통과하지 못한 요청에 401과 403 중 무엇을 돌려줄지는 ExceptionTranslationFilter가 정합니다.

거부된 요청에 어느 처리기가 응답하는지 표로 정리하면 다음과 같습니다.

거부 당시 상태	호출되는 처리기	응답
인증 정보 없음(토큰 없음 · 위조 · 만료)	`authenticationEntryPoint`	401 "로그인 후 이용해주세요"
인증은 됐지만 필요한 역할이 없음	`accessDeniedHandler`	403 "권한이 없습니다"

두 처리기는 6장에서 RespFilter로 JSON 응답을 쓰도록 만들어 두었습니다. 요청 다섯 개를 차례로 보내서 표대로 응답이 오는지 확인합니다.

2.1 Authorization 헤더에 토큰 넣기

먼저 7장과 같은 방법으로 ssar 계정의 토큰을 받습니다.

칸	입력값
METHOD	`POST`
URL	`http://localhost:8080/login`

칸	입력값
HEADERS	`Content-Type: application/json`
BODY	`{"username":"ssar","password":"1234"}`

실행 결과

출력

```
200
{"status":200,"msg":"성공","body":{"accessToken":"Bearer
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9..."}}
```

`accessToken` 값을 통째로 복사합니다. jwt.io에 붙여 넣을 때는 `Bearer ` 를 지웠지만, 요청 헤더에 넣을 때는 `Bearer ` 까지 그대로 넣어야 합니다. 5장의 `resolveToken()` 은 헤더 값이 `Bearer ` 로 시작할 때만 토큰을 꺼냅니다. 접두사가 빠지면 토큰이 없는 요청과 똑같이 처리됩니다.

새 요청을 만들고 HEADERS에 헤더 하나를 추가합니다. 이름 칸에는 `Authorization` , 값 칸에는 복사한 토큰을 넣습니다.

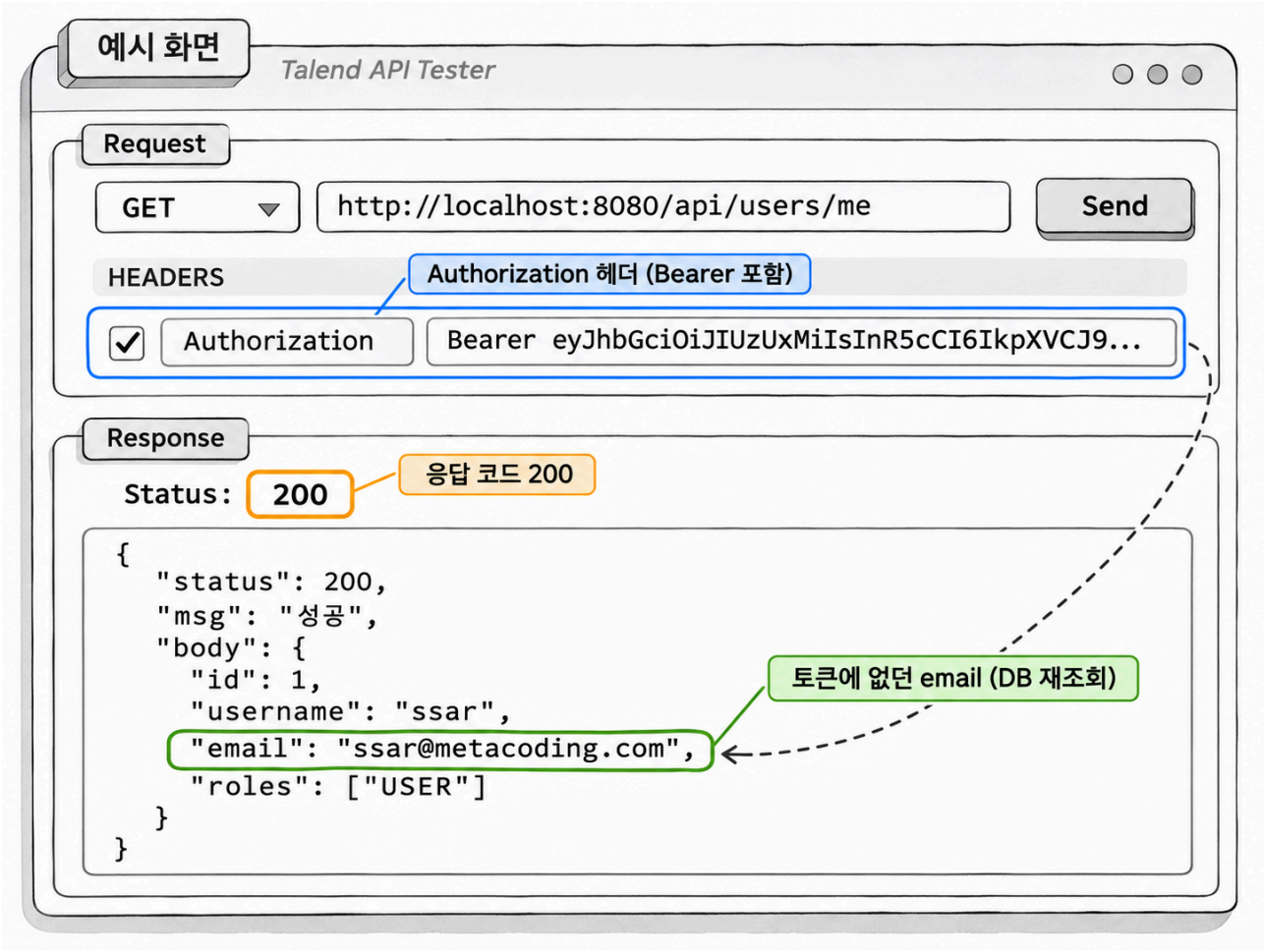
칸	입력값
METHOD	`GET`
URL	`http://localhost:8080/api/users/me`
HEADERS	`Authorization: Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9...`

실행 결과

출력

```
200
{"status":200,"msg":"성공","body":
{"id":1,"username":"ssar","email":"ssar@metacoding.com","roles":["USER"]}}
```

토큰에 없던 `email` 이 응답에 들어 있습니다. 1.2에서 설명한 DB 재조회가 일어났기 때문입니다. `password` 는 응답 DTO에서 뺐으므로 보이지 않습니다.



Talend API Tester에서 Authorization 헤더를 넣고 GET /api/users/me를 보낸 화면

생성형 예시 · 실제 실행 안 함

2.2 토큰 없음 · 위조 → 401

2.1과 같은 요청에서 Authorization 헤더의 체크를 끄거나 헤더를 지우고 다시 보냅니다.

실행 결과

출력

```
401
{"status":401,"msg":"로그인 후 이용해주세요","body":null}
```

이번에는 헤더를 다시 켜고 토큰 값 맨 끝에 `abc` 를 붙여서 보냅니다. 서명 부분이 바뀌었으니 위조된 토큰입니다.

실행 결과

출력

```
401
{"status":401,"msg":"로그인 후 이용해주세요","body":null}
```

두 응답이 똑같습니다. 토큰이 없으면 `resolveToken()`이 `null`을 돌려줍니다. 위조된 토큰이면 `JwtUtil.verify()`가 예외를 던지고, `getAuthentication()`이 `null`을 돌려줍니다. 어느 경우든 SecurityContextHolder는 비어 있습니다. AuthorizationFilter는 인증 정보가 없는 요청을 `/api/**` 규칙에서 거부하고, ExceptionTranslationFilter가 `authenticationEntryPoint`를 불러 401 응답을 씁니다. 토큰이 없을 때와 위조되었을 때를 나누는 코드가 없는데도 둘 다 401이 나오는 이유가 여기에 있습니다.

2.3 USER 토큰으로 관리자 API 요청 → 403

2.1에서 받은 ssar 토큰을 그대로 두고 URL만 바꿉니다.

칸	입력값
METHOD	`GET`
URL	`http://localhost:8080/api/admin/hello`
HEADERS	`Authorization: Bearer (ssar 토큰)`

실행 결과

출력

```
403
{"status":403,"msg":"권한이 없습니다","body":null}
```

토큰이 유효하므로 JwtAuthorizationFilter는 ssar의 인증 정보를 SecurityContextHolder에 넣었습니다. 서버는 요청한 사람이 ssar라는 것까지 압니다. 하지만 ssar의 권한은 `ROLE_USER`뿐이라서 `hasRole("ADMIN")` 규칙을 통과하지 못합니다. 이번에는 인증된 사용자가 거부되었으므로 ExceptionTranslationFilter가 `accessDeniedHandler`를 불러 403을 씁니다.

2.2와 나란히 놓으면 5장에서 배운 401과 403의 차이가 보입니다. 401은 누가 보냈는지 모르는 요청이고, 403은 누가 보냈는지는 알지만 그 일을 할 권한이 없는 요청입니다.

2.4 ADMIN 토큰으로 관리자 API 요청 → 200

cos 계정으로 로그인해서 새 토큰을 받습니다.

칸	입력값
METHOD	`POST`
URL	`http://localhost:8080/login`
HEADERS	`Content-Type: application/json`
BODY	`{"username":"cos","password":"1234"}`

받은 `accessToken` 을 Authorization 헤더에 넣고 `GET http://localhost:8080/api/admin/hello` 를 다시 보냅니다.

실행 결과

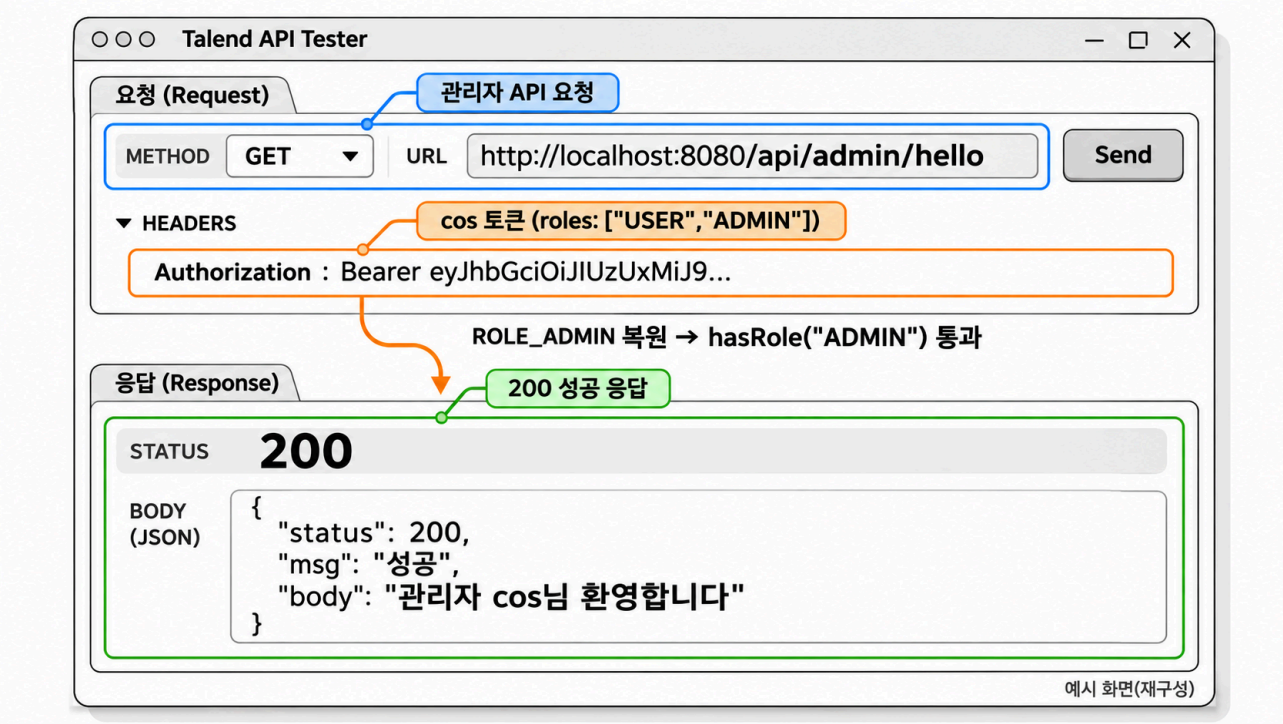
출력

```
200
{"status":200,"msg":"성공","body":"관리자 cos님 환영합니다"}
```

cos의 토큰에는 `roles` 가 `["USER","ADMIN"]` 으로 들어 있으므로 `ROLE_ADMIN` 권한이 복원됩니다. 같은 토큰으로 `/api/users/me` 를 요청해도 200과 함께 cos의 정보가 돌아옵니다. `/api/**` 규칙이 USER와 ADMIN을 모두 허용하기 때문입니다.

지금까지 보낸 요청의 결과를 모으면 다음과 같습니다.

요청	응답	응답을 만든 곳
토큰 없이 /api/users/me	401 "로그인 후 이용해주세요"	RespFilter(`authenticationEntryPoint`)
위조 토큰으로 /api/users/me	401 "로그인 후 이용해주세요"	RespFilter(`authenticationEntryPoint`)
ssar 토큰으로 /api/users/me	200, ssar 정보	UserController
ssar 토큰으로 /api/admin/hello	403 "권한이 없습니다"	RespFilter(`accessDeniedHandler`)
cos 토큰으로 /api/admin/hello	200 "관리자 cos님 환영합니다"	UserController



Talend API Tester에서 cos 토큰으로 GET /api/admin/hello를 보낸 화면

생성형 예시 · 실제 실행 안 함

---

3. 연습문제 — 만료된 토큰으로 401 확인하기

2.2에서는 위조된 토큰이 401로 막히는 것을 봤습니다. 이번에는 만료된 토큰도 같은 401로 막히는지 확인합니다. 지금 유효기간은 1일이라서 만료될 때까지 기다릴 수 없습니다. 그래서 유효기간을 잠시 짧게 줄여서 실험합니다.

요구사항

1. JwtUtil의 유효기간을 10초로 줄입니다.
2. 서버를 다시 실행하고 ssar로 로그인해 새 토큰을 받습니다.
3. 받은 토큰으로 `/api/users/me`를 10초 안에 한 번, 10초가 지난 뒤 한 번 요청합니다.
4. 확인이 끝나면 유효기간을 1일로 되돌립니다.

완료 조건

- 10초 안에 보낸 요청에는 200과 ssar 정보가 응답됩니다.
- 10초가 지난 뒤 같은 토큰으로 보낸 요청에는 401 "로그인 후 이용해주세요"가 응답됩니다.

3.1 EXPIRATION_TIME 줄이기

유효기간은 `JwtUtil`의 `EXPIRATION_TIME` 상수에 밀리초 단위로 들어 있습니다. 지금 값 `1000L * 60 * 60 * 24`는 1000밀리초 × 60 × 60 × 24이므로 1일입니다. 이 줄을 다음처럼 바꿉니다.

코드 조각

Java

```java
public static final long EXPIRATION_TIME = 1000L * 10; // 실험용: 유효기간 10초
```

이 상수는 `create()`에서 `exp` 클레임을 계산할 때만 쓰입니다. 값을 바꿔도 이미 발급된 토큰의 만료 시간은 달라지지 않고, 값을 바꾼 뒤에 새로 발급한 토큰부터 유효기간이 10초가 됩니다.

3.2 서버 재시작 후 새 토큰 받기

상수 값은 컴파일된 코드에 들어가므로 서버를 다시 실행해야 바뀐 값이 적용됩니다. 재시작한 뒤 2.1처럼 ssar로 로그인해 새 `accessToken`을 받습니다.

서버를 재시작해도 2절에서 받은 예전 토큰은 무효가 되지 않습니다. 서버는 발급한 토큰을 어디에도 저장하지 않습니다. 비밀키가 그대로라면 예전 토큰도 서명 검증을 통과하고, 그 토큰의 `exp`도 여전히 1일 뒤입니다. 세션 방식에서는 서버를 껐다 켜면 메모리의 세션이 사라져 다시 로그인해야 했습니다. 이 차이가 3장에서 배운 Stateless입니다. 예전 토큰으로 실험하면 401이 나오지 않으니, 반드시 재시작한 뒤 새로 받은 토큰을 씁니다.

3.3 만료 시간이 지난 뒤 요청해 401 확인하기

새 토큰을 Authorization 헤더에 넣고 곧바로 `GET http://localhost:8080/api/users/me`를 보내면 200이 나옵니다. 15초쯤 기다렸다가 같은 요청을 다시 보냅니다. `exp`는 초 단위로 저장되므로 10초보다 넉넉하게 기다립니다.

실행 결과

출력

```
401
{"status":401,"msg":"로그인 후 이용해주세요","body":null}
```

만료된 토큰은 `JwtUtil.verify()`에서 예외를 일으킵니다. java-jwt는 만료를 TokenExpiredException으로 알리는데, 이 예외는 위조 때와 같은 JWTVerificationException의 하위 예외입니다. `getAuthentication()`이 두 경우 모두 `null`을 돌려주므로 그 뒤 흐름은 2.2와 똑같이 401로 끝납니다.

실험을 마치면 `EXPIRATION_TIME`을 `1000L * 60 * 60 * 24`로 되돌리고 서버를 다시 실행합니다.